

CAIRO UNIVERSITY

Faculty of Computers and Artificial Intelligence

CS213 — Object-Oriented Programming

Assignment 2 • Final Submmision



VOLE MACHINE SIMULATOR

Design Report

Instructor: Mohamed El-Ramly | Sections: S15, S16

,17,18

Report date: 9 November 2026

Team Members

1. Team Leader: Laila Mohamed Abd El-Mohsen | 20250508 | 15,16 |Phone: 01111368601
2. Ereany Eid Eweda | 20250102 |15,16
3. George Mina Zakaria | 20251111|17,18
4. Roqaia Mohamed Ismael | 20251149 |15,16

Work Breakdown

Team Members	Work Done
Laila Mohamed Abd El-Mohsen	**CPU.h, CPU.cpp, Memory.h, Memory.cpp, GUI part1 , testing program , GUI Video**
Ereany Eid Eweda	**ALU.h, ALU.cpp, GUI_part2, Document part3**
George Mina Zakaria	**Machine.h, Machine.cpp, old-A2-S15_16-2025149-20251111-20250508-20250102.cpp, Document part2**
Roqaia Mohamed Ismael	**Utilities.h, Utilities.cpp,document part1**

Report Contents

- 1. Introduction
- 2. Initial Design
- 3. AI Designs and Comparison
- 4. Final Design — Selected DeepSeek Design
- 5. Example Vole Programs
- 6. Video Link
- Appendix A. Full DeepSeek Conversation
- Appendix B. Full ChatGPT Conversation
- Appendix C. Full Gemini Conversation
- Appendix D. Vole Simulator Chat Transcript
- Program testing Screenshots
- GitHub sceenshots

1. Introduction

The Vole Machine Simulator models a small register-oriented computer. It loads hexadecimal machine-language programs into byte-addressed memory and executes them through a fetch–decode–execute cycle. The simulator separates the machine state from program loading, register storage, memory access, instruction decoding, arithmetic and logic, and hexadecimal conversion.

The architecture has 256 memory cells, each holding an 8-bit byte, addressed from 00h through FFh, and 16 general-purpose 8-bit registers. Each instruction is 16 bits long and occupies two consecutive memory cells. The 8-bit Program Counter (PC) identifies the next instruction address, and the 16-bit Instruction Register (IR) holds the instruction being processed. A normal instruction advances PC by two; a conditional jump replaces PC with its target address.

During execution, the CPU fetches the two instruction bytes from memory and combines them into IR. The instruction decoder separates the opcode and operands and checks whether the instruction format is valid. The CPU then performs the operation using Memory, RegisterFile, and the ALU as needed. Programs may run continuously or one instruction at a time. Address FFh is mapped to screen output when a byte is stored there, and memory can be displayed as a 16 × 16 grid.

The instruction set includes memory and immediate loads, stores, register moves, integer and floating-point addition, bitwise OR/AND/XOR, circular right rotation, a conditional branch, and HALT. The precise floating-point encoding and boundary behavior should follow the course specification.

2. Initial Design

The initial proposal in the conversation records begins with four explicitly described classes: Machine, CPU, RegisterFile, and Memory. Machine also refers to a ProgramLoader object, but its detailed attributes and methods were not yet specified. At this stage, hexadecimal conversion, program loading, control flow, and many arithmetic operations are concentrated in Machine or RegisterFile; there is not yet a separate ALU or instruction decoder.

2.1 Classes and responsibilities

- Machine: Coordinates the simulator and initially contains CPU, Memory, ProgramLoader, and RegisterFile. Its listed methods include program loading, run and step execution, reset, halt, load, jump, start-address validation, and hexadecimal conversion.
- CPU: Stores PC and IR and exposes setters/getters. Fetch, decode, validation, and execution responsibilities are not yet fully assigned in the initial proposal.
- RegisterFile: Stores 16 registers and initially includes operations such as immediate load, move, integer/float addition, rotation, bitwise operations, display, and equality checking.
- Memory: Stores 256 cells and provides get, set, and display operations.
- ProgramLoader: Referenced as a Machine member, but its responsibility and methods are not defined in the initial class list.

This initial design is a useful starting point, but the later proposals improve separation of concerns by moving file parsing to ProgramLoader, instruction validation/decoding to InstructionDecoder, and computation to ALU.

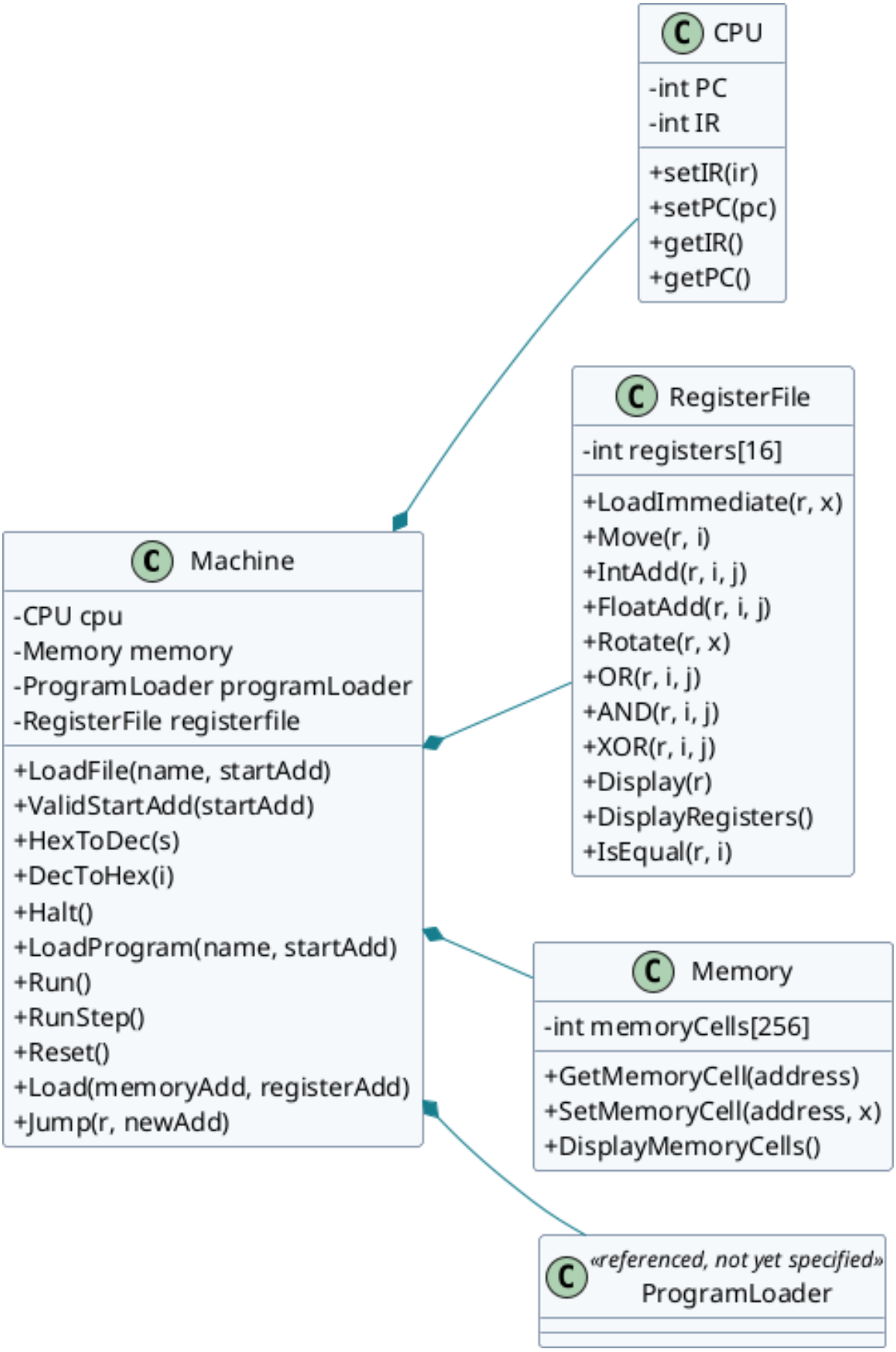

CPU
-int PC
-int IR
+setIR(ir)
+setPC(pc)
+getIR()
+getPC()
RegisterFile
-int registers[16]
+LoadImmediate(r, x)
+Move(r, i)
+IntAdd(r, i, j)
+FloatAdd(r, i, j)
+Rotate(r, x)
+OR(r, i, j)
+AND(r, i, j)
+XOR(r, i, j)
+Display(r)
+DisplayRegisters()
+IsEqual(r, i)
Machine
-CPU cpu
-Memory memory
-ProgramLoader programLoader
-RegisterFile registerfile
+LoadFile(name, startAdd)
+ValidStartAdd(startAdd)
+HexToDec(s)
+DecToHex(i)
+Halt()
+LoadProgram(name, startAdd)
+Run()
+RunStep()
+Reset()
+Load(memoryAdd, registerAdd)
+Jump(r, newAdd)
Memory
-int memoryCells[256]
+GetMemoryCell(address)
+SetMemoryCell(address, x)
+DisplayMemoryCells()
«referenced, not yet specified»
ProgramLoader

Figure 1. Initial class structure reconstructed from the first class proposal in the supplied conversations.

PlantUML source — Initial Design

```
@startuml
left to right direction
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
skinparam shadowing false
skinparam class {
 BackgroundColor #F6F9FC
 BorderColor #17365D
 ArrowColor #167D8D
}
class Machine {
  -CPU cpu
  -Memory memory
  -ProgramLoader programLoader
  -RegisterFile registerfile
  +LoadFile(name, startAdd)
  +ValidStartAdd(startAdd)
  +HexToDec(s)
  +DecToHex(i)
  +Halt()
  +LoadProgram(name, startAdd)
  +Run()
  +RunStep()
  +Reset()
  +Load(memoryAdd, registerAdd)
  +Jump(r, newAdd)
}
class CPU {
  -int PC
  -int IR
  +setIR(ir)
  +setPC(pc)
  +getIR()
  +getPC()
}
class RegisterFile {
  -int registers[16]
  +LoadImmediate(r, x)
  +Move(r, i)
  +IntAdd(r, i, j)
  +FloatAdd(r, i, j)
  +Rotate(r, x)
  +OR(r, i, j)
  +AND(r, i, j)
  +XOR(r, i, j)
  +Display(r)
  +DisplayRegisters()
  +IsEqual(r, i)
}
class Memory {
  -int memoryCells[256]
  +GetMemoryCell(address)
  +SetMemoryCell(address, x)
  +DisplayMemoryCells()
}
class ProgramLoader <<referenced, not yet specified>>
Machine *-- CPU
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- ProgramLoader
@enduml
```

3. AI Designs and Comparison

The three design alternatives compared in this section are the designs recorded in the DeepSeek, ChatGPT, and Gemini conversations. Claude’s later conversation is included in Appendix D as the design-review discussion used to compare and select among alternatives. The group selected the DeepSeek design for the final report.

3.1 ChatGPT design

ChatGPT's final class list uses five main classes: Machine, CPU, RegisterFile, Memory, and ProgramLoader, plus an ExecutionStatus enumeration. It emphasizes CPU references to Memory and RegisterFile and includes step results for running, halted, invalid instruction, and memory error. It keeps screen-output coordination in Machine and retains many arithmetic operations on RegisterFile.

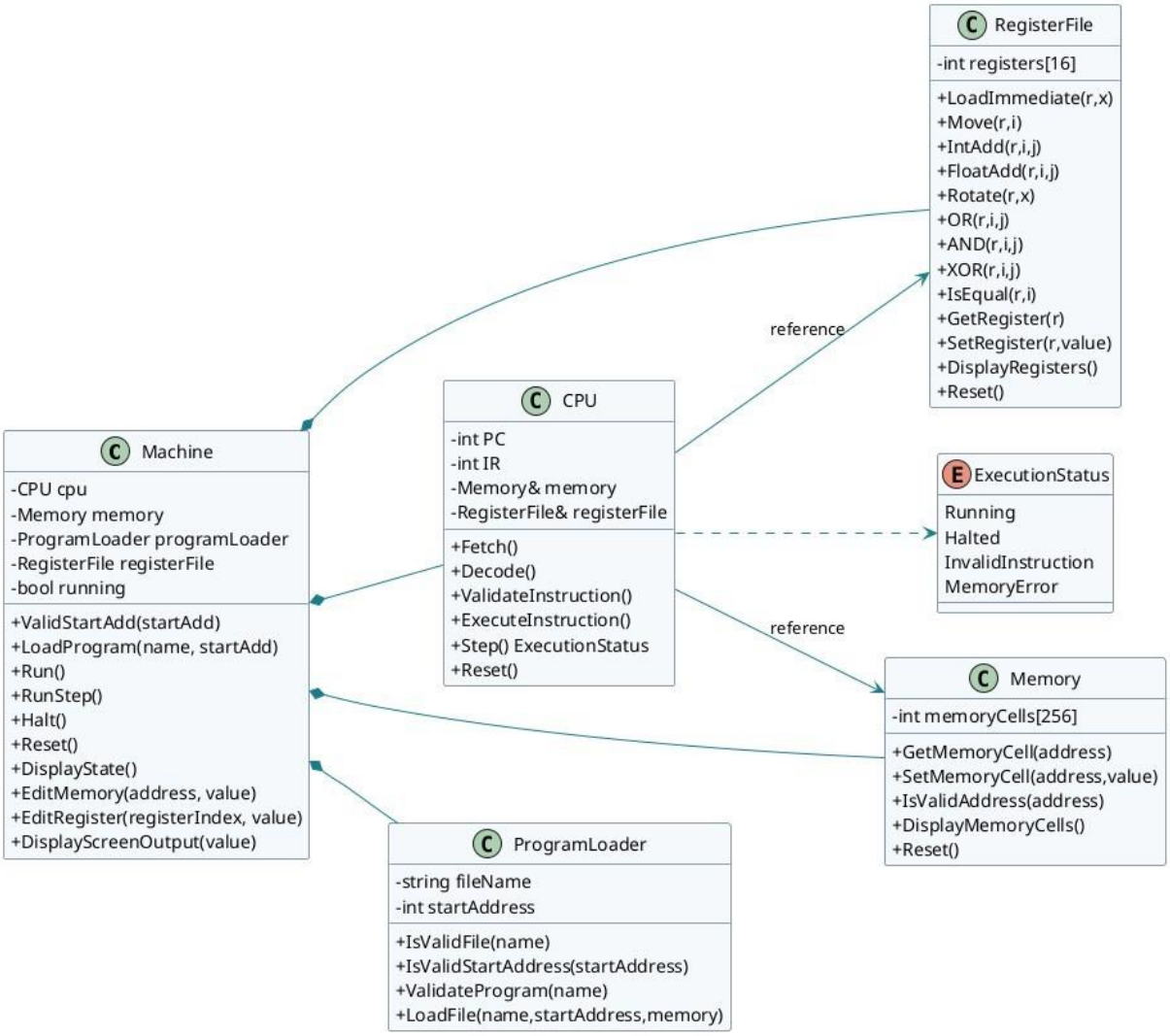


Figure 2. UML class diagram reconstructed from the final class list in the supplied ChatGPT transcript.

PlantUML source — ChatGPT Design

```
@startuml
left to right direction
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam class {
  BackgroundColor #F6F9FC
  BorderColor #17365D
}
skinparam ArrowColor #167D8D
class Machine {
  -CPU cpu
  -Memory memory
  -ProgramLoader programLoader
  -RegisterFile registerFile
```

```
  -bool running
  +ValidStartAdd(startAdd)
  +LoadProgram(name, startAdd)
  +Run()
  +RunStep()
  +Halt()
  +Reset()
  +DisplayState()
  +EditMemory(address, value)
  +EditRegister(registerIndex, value)
  +DisplayScreenOutput(value)
}
class CPU {
  -int PC
  -int IR
  -Memory& memory
  -RegisterFile& registerFile
  +Fetch()
  +Decode()
  +ValidateInstruction()
  +ExecuteInstruction()
  +Step() ExecutionStatus
  +Reset()
}
class RegisterFile {
  -int registers[16]
  +LoadImmediate(r,x)
  +Move(r,i)
  +IntAdd(r,i,j)
  +FloatAdd(r,i,j)
  +Rotate(r,x)
  +OR(r,i,j)
  +AND(r,i,j)
  +XOR(r,i,j)
  +IsEqual(r,i)
  +GetRegister(r)
  +SetRegister(r,value)
  +DisplayRegisters()
  +Reset()
}
class Memory {
  -int memoryCells[256]
  +GetMemoryCell(address)
  +SetMemoryCell(address,value)
  +IsValidAddress(address)
  +DisplayMemoryCells()
  +Reset()
}
class ProgramLoader {
  -string fileName
  -int startAddress
  +IsValidFile(name)
  +IsValidStartAddress(startAddress)
  +ValidateProgram(name)
  +LoadFile(name,startAddress,memory)
}
enum ExecutionStatus {
  Running
  Halted
  InvalidInstruction
  MemoryError
}
Machine *-- CPU
Machine *-- Memory
Machine *-- ProgramLoader
Machine *-- RegisterFile
CPU --> Memory : reference
CPU --> RegisterFile : reference
CPU ..> ExecutionStatus
@enduml
```

3.2 Gemini design

Gemini first preserves the original core classes and adds an ALU and a file-oriented ProgramLoader. Its later proposal introduces an Instruction structure to hold parsed fields and suggests error handling and machine-state flags. The final supplied diagram shows Machine, CPU, Instruction, RegisterFile, Memory, ALU, and ProgramLoader.

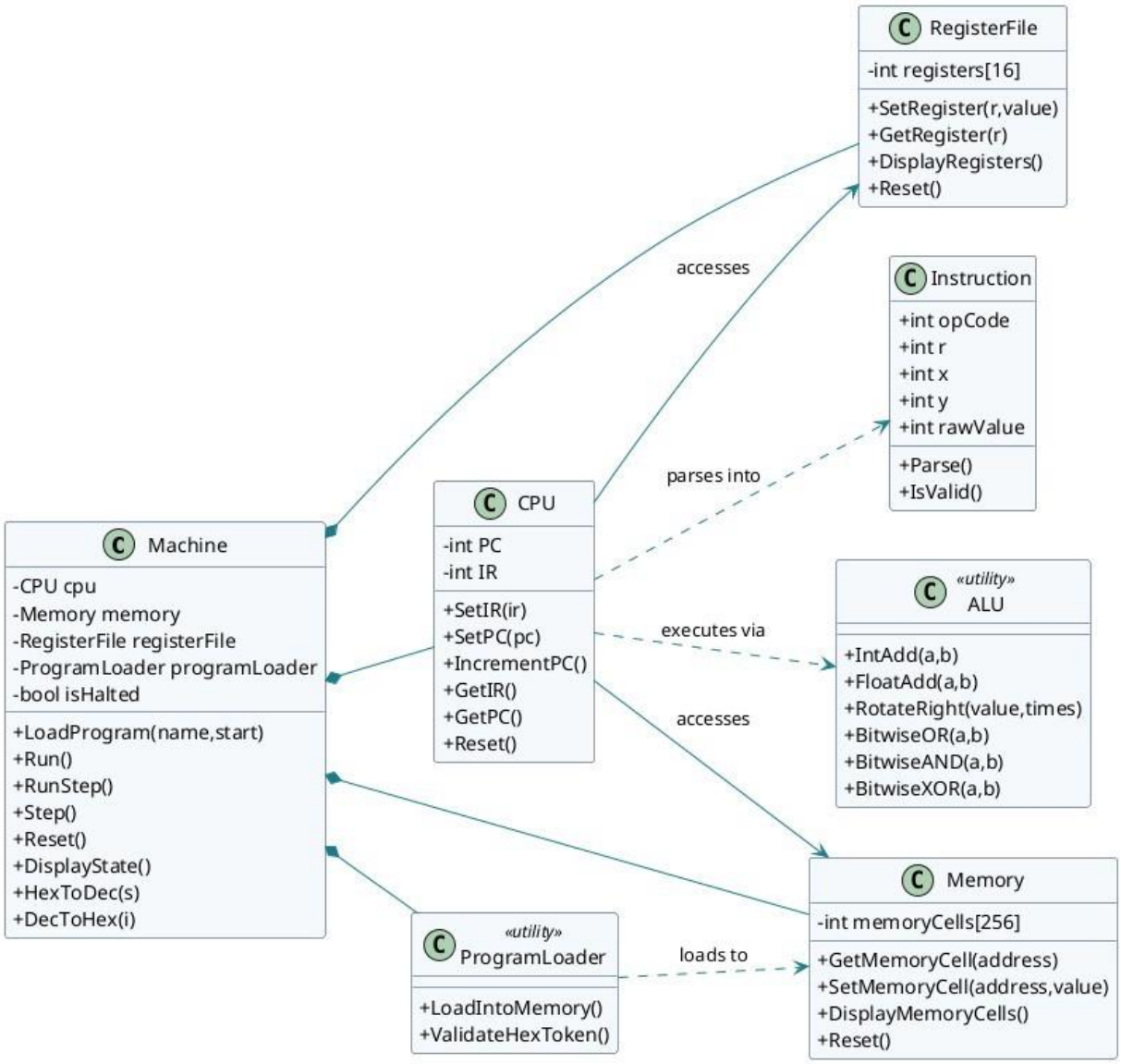


Figure 3. UML class diagram reconstructed from the final supplied Gemini design image and conversation.

PlantUML source — Gemini Design

```
@startuml
left to right direction
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam class {
  BackgroundColor #F6F9FC
  BorderColor #17365D
}
skinparam ArrowColor #167D8D
class Machine {
  -CPU cpu
  -Memory memory
  -RegisterFile registerFile
  -ProgramLoader programLoader
  -bool isHalted
  +LoadProgram(name,start)
  +Run()
  +RunStep()
  +Step()
  +Reset()
  +DisplayState()
  +HexToDec(s)
  +DecToHex(i)
}
class CPU {
```

```
  -int PC
  -int IR
  +SetIR(ir)
  +SetPC(pc)
  +IncrementPC()
  +GetIR()
  +GetPC()
  +Reset()
}
class Instruction {
  +int opCode
  +int r
  +int x
  +int y
  +int rawValue
  +Parse()
  +IsValid()
}
class RegisterFile {
  -int registers[16]
  +SetRegister(r,value)
  +GetRegister(r)
  +DisplayRegisters()
  +Reset()
}
class Memory {
  -int memoryCells[256]
  +GetMemoryCell(address)
  +SetMemoryCell(address,value)
  +DisplayMemoryCells()
  +Reset()
}
class ALU <<utility>> {
  +IntAdd(a,b)
  +FloatAdd(a,b)
  +RotateRight(value,times)
  +BitwiseOR(a,b)
  +BitwiseAND(a,b)
  +BitwiseXOR(a,b)
}
class ProgramLoader <<utility>> {
  +LoadIntoMemory()
  +ValidateHexToken()
}
Machine *-- CPU
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- ProgramLoader
CPU ..> Instruction : parses into
CPU ..> ALU : executes via
CPU --> Memory : accesses
CPU --> RegisterFile : accesses
ProgramLoader ..> Memory : loads to
@enduml
```

3.3 DeepSeek design

DeepSeek’s latest agreed class list has eight classes: HexUtils, InstructionDecoder, ALU, RegisterFile, Memory, ProgramLoader, CPU, and Machine. It merges decoded-instruction fields into InstructionDecoder and handles the screen character within Memory, while keeping HexUtils separate. The full supplied DeepSeek dialogue is reproduced in Appendix A.

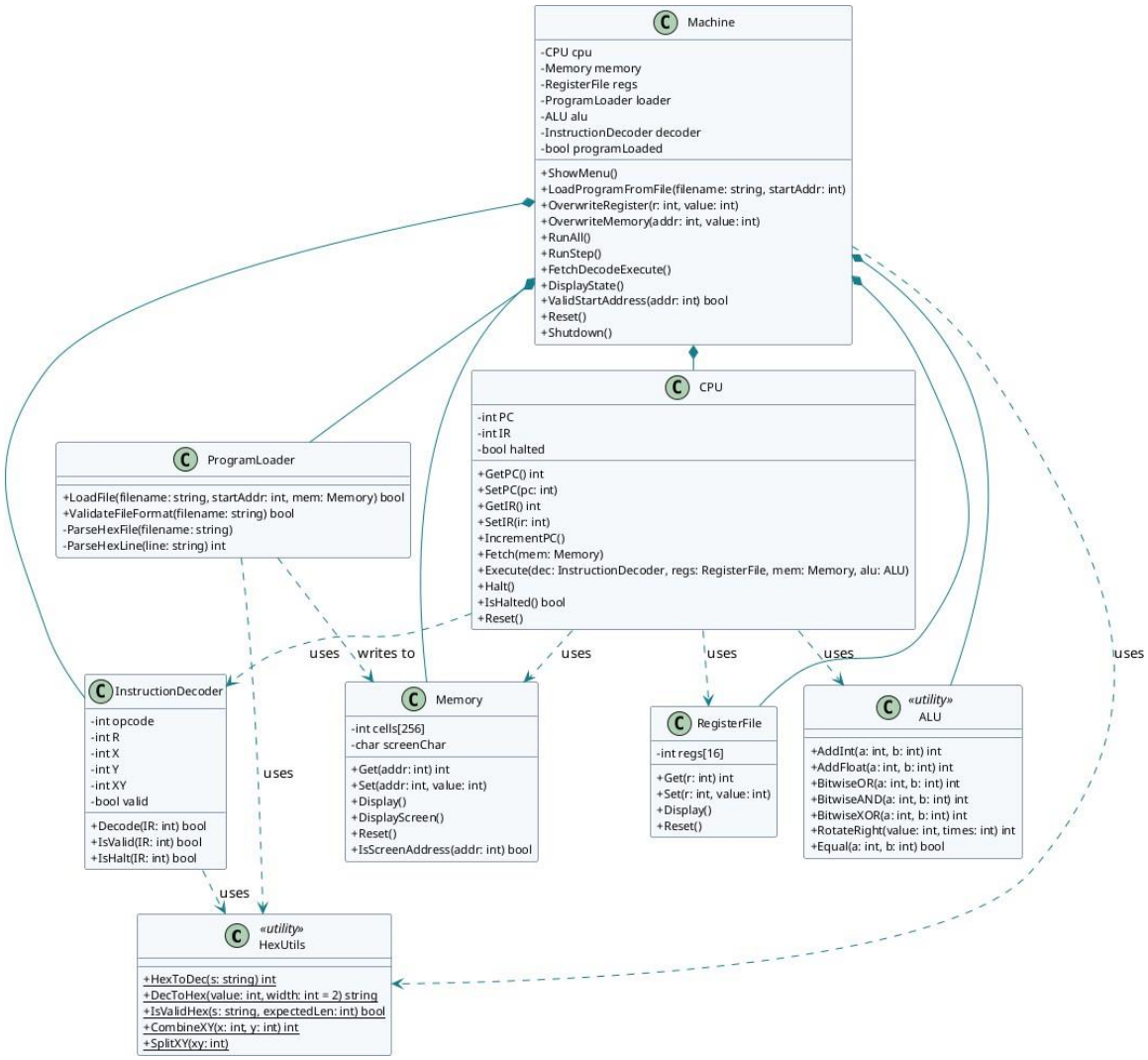


Figure 4. DeepSeek's final eight-class UML design, represented in PlantUML for this report.

PlantUML source — DeepSeek Design

```
@startuml
top to bottom direction
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
skinparam shadowing false
skinparam classFontSize 13
skinparam classAttributeFontSize 11
skinparam classMethodFontSize 11
skinparam nodesep 18
skinparam ranksep 22
skinparam class {
  BackgroundColor #F6F9FC
  BorderColor #17365D
}
skinparam ArrowColor #167D8D
class HexUtils <<utility>> {
  {static} +HexToDec(s: string) int
  {static} +DecToHex(value: int, width: int = 2) string
  {static} +IsValidHex(s: string, expectedLen: int) bool
  {static} +CombineXY(x: int, y: int) int
  {static} +SplitXY(xy: int)
}
class InstructionDecoder {
  -int opcode
  -int R
  -int X
  -int Y
  -int XY
```

```
  -bool valid
  +Decode(IR: int) bool
  +IsValid(IR: int) bool
  +IsHalt(IR: int) bool
}
class ALU <<utility>> {
  +AddInt(a: int, b: int) int
  +AddFloat(a: int, b: int) int
  +BitwiseOR(a: int, b: int) int
  +BitwiseAND(a: int, b: int) int
  +BitwiseXOR(a: int, b: int) int
  +RotateRight(value: int, times: int) int
  +Equal(a: int, b: int) bool
}
class RegisterFile {
  -int regs[16]
  +Get(r: int) int
  +Set(r: int, value: int)
  +Display()
  +Reset()
}
class Memory {
  -int cells[256]
  -char screenChar
  +Get(addr: int) int
  +Set(addr: int, value: int)
  +Display()
  +DisplayScreen()
  +Reset()
  +IsScreenAddress(addr: int) bool
}
class ProgramLoader {
  +LoadFile(filename: string, startAddr: int, mem: Memory) bool
  +ValidateFileFormat(filename: string) bool
  -ParseHexFile(filename: string)
  -ParseHexLine(line: string) int
}
class CPU {
  -int PC
  -int IR
  -bool halted
  +GetPC() int
  +SetPC(pc: int)
  +GetIR() int
  +SetIR(ir: int)
  +IncrementPC()
  +Fetch(mem: Memory)
  +Execute(dec: InstructionDecoder, regs: RegisterFile, mem: Memory, alu: ALU)
  +Halt()
  +IsHalted() bool
  +Reset()
}
class Machine {
  -CPU cpu
  -Memory memory
  -RegisterFile regs
  -ProgramLoader loader
  -ALU alu
  -InstructionDecoder decoder
  -bool programLoaded
  +ShowMenu()
  +LoadProgramFromFile(filename: string, startAddr: int)
  +OverwriteRegister(r: int, value: int)
  +OverwriteMemory(addr: int, value: int)
  +RunAll()
  +RunStep()
  +FetchDecodeExecute()
  +DisplayState()
  +ValidStartAddress(addr: int) bool
  +Reset()
  +Shutdown()
}
Machine *-- CPU
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- ProgramLoader
Machine *-- ALU
Machine *-- InstructionDecoder
CPU ..> InstructionDecoder : uses
CPU ..> RegisterFile : uses
CPU ..> Memory : uses
CPU ..> ALU : uses
ProgramLoader ..> Memory : writes to
ProgramLoader ..> HexUtils : uses
InstructionDecoder ..> HexUtils : uses
```

```
Machine ..> HexUtils : uses
@enduml
```

3.4 Comparison and selection

The alternatives make different trade-offs. ChatGPT keeps the class count low, but its proposed RegisterFile still contains many opcode operations, while Machine handles conversion and screen coordination. Gemini improves the division of computation by introducing ALU and adds an Instruction structure, but its supplied final diagram does not show a separate decoder or hexadecimal utility. Claude's review suggests additional implementation safeguards and a more detailed architecture; the full review is preserved in Appendix D.

The selected DeepSeek design offers a balanced class structure for the assignment. Machine orchestrates; CPU runs the execution cycle; InstructionDecoder validates and extracts instruction fields; ALU performs arithmetic and logic; RegisterFile and Memory encapsulate state; ProgramLoader handles files; and HexUtils provides stateless conversion helpers. Memory also handles the FFh screen mapping. This keeps responsibilities distinct without introducing more classes than the team needs for its selected design.

The review raised useful implementation considerations—such as validating file size, preventing out-of-range fetches, and making screen output explicit. These are considerations for implementation and testing; they do not change the selected six-class architecture or the original conversation text.

4. Final Design

The final design below represents the Vole Machine Simulator architecture adopted by the team. It separates machine control, CPU execution, memory and register storage, program loading, utility conversions, and arithmetic/logic operations.

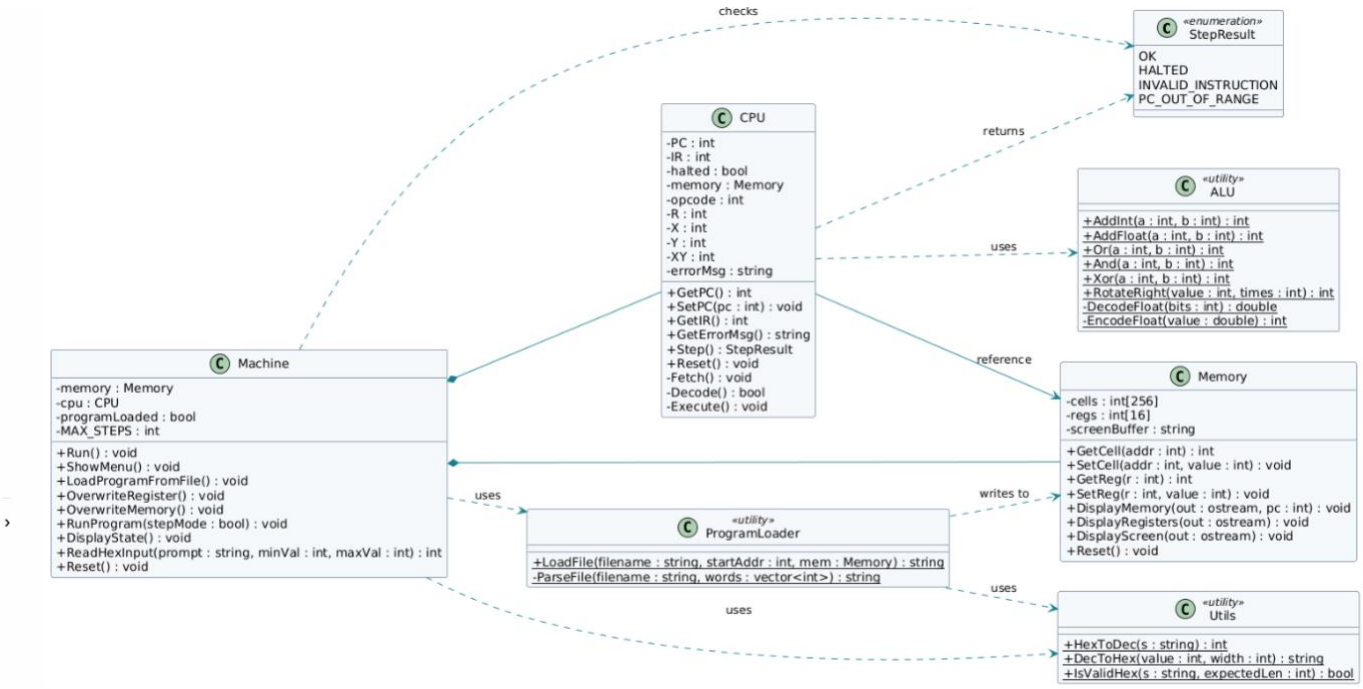


Figure: Final design — UML class diagram (PlantUML source below).

PlantUML source — Final Design

```
@startuml
left to right direction
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam class {
  BackgroundColor #F6F9FC
  BorderColor #17365D
}
skinparam ArrowColor #167D8D

'==================== ENUM ====================
class StepResult <<enumeration>> {
  OK
  HALTED
  INVALID_INSTRUCTION
  PC_OUT_OF_RANGE
}

'==================== UTILS ====================
class Utils <<utility>> {
  {static} +HexToDec(s : string) : int
  {static} +DecToHex(value : int, width : int) : string
```

```
  {static} +IsValidHex(s : string, expectedLen : int) : bool
}

'==================== ALU ====================
class ALU <<utility>> {
  {static} +AddInt(a : int, b : int) : int
  {static} +AddFloat(a : int, b : int) : int
  {static} +Or(a : int, b : int) : int
  {static} +And(a : int, b : int) : int
  {static} +Xor(a : int, b : int) : int
  {static} +RotateRight(value : int, times : int) : int
  {static} -DecodeFloat(bits : int) : double
  {static} -EncodeFloat(value : double) : int
}

'==================== MEMORY ====================
class Memory {
  -cells : int[256]
  -regs : int[16]
  -screenBuffer : string
  +GetCell(addr : int) : int
  +SetCell(addr : int, value : int) : void
  +GetReg(r : int) : int
  +SetReg(r : int, value : int) : void
  +DisplayMemory(out : ostream, pc : int) : void
  +DisplayRegisters(out : ostream) : void
  +DisplayScreen(out : ostream) : void
  +Reset() : void
}

'==================== PROGRAM LOADER ====================
class ProgramLoader <<utility>> {
  {static} +LoadFile(filename : string, startAddr : int, mem : Memory) : string
  {static} -ParseFile(filename : string, words : vector<int>) : string
}

'==================== CPU ====================
class CPU {
  -PC : int
  -IR : int
  -halted : bool
  -memory : Memory
```

```
  -opcode : int
  -R : int
  -X : int
  -Y : int
  -XY : int
  -errorMsg : string
  +GetPC() : int
  +SetPC(pc : int) : void
  +GetIR() : int
  +GetErrorMsg() : string
  +Step() : StepResult
  +Reset() : void
  -Fetch() : void
  -Decode() : bool
  -Execute() : void
}

'==================== MACHINE ====================
class Machine {
  -memory : Memory
  -cpu : CPU
  -programLoaded : bool
  -MAX_STEPS : int
  +Run() : void
  +ShowMenu() : void
  +LoadProgramFromFile() : void
  +OverwriteRegister() : void
  +OverwriteMemory() : void
  +RunProgram(stepMode : bool) : void
  +DisplayState() : void
  +ReadHexInput(prompt : string, minVal : int, maxVal : int) : int
  +Reset() : void
}

'==================== RELATIONSHIPS ====================
Machine *-- Memory
Machine *-- CPU
CPU --> Memory : reference
CPU ..> ALU : uses
CPU ..> StepResult : returns
ProgramLoader ..> Memory : writes to
ProgramLoader ..> Utils : uses
```

```
Machine ..> ProgramLoader : uses
Machine ..> Utils : uses
Machine ..> StepResult : checks

@enduml
```

4.1 Final classes and responsibilities

Machine: Coordinates the simulator and owns the other components. Holds `Memory` and `CPU` (in that declaration order), tracks `programLoaded` and `MAX_STEPS`, and provides the menu, program loading, register/memory overwrite, full-run and step-by-step execution, state display, hex input reading, and reset.

CPU: Maintains `PC`, `IR`, halt status, decoded instruction fields (`opcode`, `R`, `X`, `Y`, `XY`), and `errorMsg`. Performs fetch-decode-execute through `Step()`, which returns a `StepResult`. Private helpers `Fetch()`, `Decode()`, and `Execute()` keep the cycle cohesive. Holds a reference to `Memory`.

Memory: Centralizes all storage — 256 memory cells, 16 general-purpose registers, and the screen buffer. All writes go through `SetCell` / `SetReg`, which mask values to 8 bits; a write to address `FF` is appended to `screenBuffer` instead of a cell. Provides display functions and reset.

ProgramLoader: Static utility that reads a program file, validates the start address (even, ≤ FE), validates every token as 4 hex digits, ensures the program fits before `FF`, and only then writes it into `Memory` — two bytes per instruction (high byte first). Returns an empty string on success or an error message on failure.

ALU: Stateless utility that performs integer and floating-point addition, bitwise OR/AND/XOR, and circular right rotation. Contains private `DecodeFloat` / `EncodeFloat` helpers for the 8-bit Vole float format (1 sign bit, 3 exponent bits excess-4, 4 mantissa bits, truncation).

Utils: Stateless utility for hexadecimal-to-decimal conversion, decimal-to-hex formatting with fixed width, and hex string validation. Used by `Machine` (input/display) and `ProgramLoader` (file parsing).

StepResult: Enumeration describing the outcome of a CPU step: `OK`, `HALTED`, `INVALID_INSTRUCTION`, or `PC_OUT_OF_RANGE`. Allows `RunAll` and `RunStep` to share one loop.

4.2 Design rationale

The 6-class design keeps each responsibility in one place without over-fragmenting the architecture. `Machine` acts as the composition root and user interface; `CPU` owns the fetch-decode-execute cycle and the decoded instruction fields, removing the need for a separate `Instruction` struct or `InstructionDecoder` class; `Memory` consolidates memory cells, registers, and the memory-mapped screen, which matches the specification's statement that address `FF` is a screen mapping; `ProgramLoader` remains the only class that reads files; `ALU` isolates all arithmetic and logic including

the 8-bit float helpers; `Utils` provides stateless hex helpers used by both `Machine` and `ProgramLoader`.

Merging `RegisterFile` into `Memory` and `Screen` into `Memory` reduces the class count while preserving clear boundaries, because all state that must stay consistent (cells, registers, screen) is written through one pair of setters (`SetCell`, `SetReg`) that enforce the 8-bit invariant. Merging `InstructionDecoder` into `CPU` avoids a class that only held transient decoded fields; the CPU keeps those fields as private attributes for display and execution. `StepResult` makes CPU outcomes explicit without requiring a larger `ExecutionStatus` enum.

5. Example Vole Machine Programs

The following two Vole Machine programs are provided for evaluating the simulator. Each four-digit instruction occupies two consecutive memory cells.

Example 1 — Swap two register values using XOR

```
;Vole Machine Program
;Purpose :
;Swap the values stored in 2 registers
;using bitwise 'XOR' operation
;The XOR trick: a = (a XOR b) XOR b
;(This works because XOR is reversible and
;self-inverse)
;By applying XOR three times, we swap
;a and b without a temp.
2005    ;R0 = 5
2107    ;R1 = 7
9201    ;R2 = R0 xor R1 (R2 = 2)
9002    ;R0 = R0 xor R2 (R0 = 7)
9112    ;R1 = R1 xor R2 (R1 = 5)
C000    ;Halt
```

Example 2 — Multiply the first number by 2 and subtract the second

```
; vole machine sample program 2
; purpose : load two number multiply the first by 2 ,
; and subtract the second number from the first after multiplying it,(by adding the two's complement),
; dispaly the result on screen.
2106 ;load the first number (06) into register 1
5211 ;multiply it by 2,R2=R1+R1 (6+6=0C)=12
2303 ; load the second number (03) into register 3
24FD ; load the two's complement of -3 (FD) into register 4
5624 ; R6 = R2 + R4 (12+(-3))= 09
36FF ; dispaly the result
C000 ; HALT execution
```

6. Video Link

GITHUB: https://github.com/roq4i4/oop-assignment2

VIDEO_DESIGN: https://youtu.be/2-UUeZFESg8?si=rEcIKeTebyg5VIwT

VIDEO_GUI: https://youtu.be/3gviknstlVM

VOLE MACHINE SIMULATOR DESIGN

DEEPSEEK TRANSCRIPT

USER: I have to make a Vole machine simulator that can execute the instructions below (in hexadecimal) using C++. Each instruction has 4 hexadecimal digits and each memory cell holds two hexadecimal digits.

The simulator will:

1. Offer a menu of choices as needed.
2. Allow the user to load a program from a file into a specific even memory address (00, 02, 04, …, F0, F2, …) below FFh.
3. Allow the user to overwrite a register or memory cell.
4. Set the Program Counter (PC) to the program's start memory location.
5. Display memory as a matrix, together with IR and the 16 general-purpose registers.
6. Fetch each instruction into IR, validate it, decode it, execute it, and update PC.
7. Allow execution all at once or step by step.
8. If running all at once, display memory, PC, IR, registers, and screen after execution finishes.
9. If running step by step, display the state after each step and wait for Enter before continuing.
10. Ask whether the user wants to exit or load another program.

Machine Language (2026/27 version)

Op-code	Operand	Description
1	RXY	Load register R from memory cell XY. Example: 14A3 loads memory[A3] into register 4.
2	RXY	Load immediate value XY into register R. Example: 20A3 loads A3 into register 0.
3	RXY	Store register R in memory cell XY. Example: 35B1 stores register 5 at B1.
3	RFF	Store to FF, which is memory-mapped to the screen.
4	0RS	Move the value in register R to register S.
5	RST	Add registers S and T as two's-complement integers; store result in R.
6	RST	Add registers S and T as floating-point values; store result in R.
7	RST	Bitwise OR registers S and T; store result in R.
8	RST	Bitwise AND registers S and T; store result in R.
9	RST	Bitwise XOR registers S and T; store result in R.
A	R0X	Rotate register R right by X bits.
B	RXY	Jump to XY if register R equals register 0.
C	000	Halt execution.

Now I just want to determine the classes, their attributes and methods — no code at all.

Initial design

Class Machine

- CPU cpu
- Memory memory
- ProgramLoader programLoader
- RegisterFile registerfile

- bool LoadFile(string Name, int StartAdd)
- bool ValidStartAdd(int startAdd)
- int HexToDec(string s)
- string DecToHex(int i)
- void Halt()
- void LoadProgram(string Name, int StartAdd)
- void Run()
- void RunStep()
- void Reset()
- void Load(int MemoryAdd, int RegisterAdd)
- void Jump(int r, int NewAdd)

Class CPU

- int PC
- int IR
- void setIR(int ir)
- void setPC(int pc)
- int getIR()
- int getPC()

Class RegisterFile

- int registers[16]
- void LoadImmediate(int r, int x)
- void Move(int r, int i)
- void IntAdd(int r, int i, int j)
- void FloatAdd(int r, int i, int j)
- void Rotate(int r, int x)
- void OR(int r, int i, int j)
- void AND(int r, int i, int j)
- void XOR(int r, int i, int j)
- void Display(int r)
- void DisplayRegisters()
- bool IsEqual(int r, int i)

Class Memory

- int memoryCells[256]
- int GetMemoryCell(int address)
- void SetMemoryCell(int address, int x)
- void DisplayMemoryCells()

This is what I reached now. Help me in the design of the simulator.

ASSISTANT:

Vole Simulator Design Review & Refinement

Your initial class breakdown is a good start. Below is a structured critique and a refined design. This is design only, with no code.

1. Overall Architectural Concerns

Concerns with your current split

Issue	Why it matters
Instruction decoding lives nowhere	RunStep() in Machine implies it decodes, but decode logic has no dedicated class or method.
RegisterFile does ALU work	IntAdd, FloatAdd, AND, OR, XOR, and Rotate are ALU operations; RegisterFile should primarily store and read registers.

Machine does too much	Machine currently combines file loading, validation, hex conversion, CPU driving, and instruction execution.
No Screen abstraction	Address FF is memory-mapped to the screen; this needs a clear handling point.
Opcode helpers on Machine	Load() and Jump() are instruction implementations rather than machine-level services.
ProgramLoader is undefined	Machine refers to ProgramLoader, but its attributes and methods are not defined.
IR is a raw int	Decoding should expose structured fields such as opcode, R, X, Y, and XY.
No explicit instruction validation	The requirements call for validation before execution.

Recommended top-level classes

- Machine — coordinates the run loop, menu, and program lifecycle.
- CPU — manages PC, IR, and the fetch/decode/execute pipeline.
- RegisterFile — stores the 16 general-purpose registers.
- Memory — stores 256 cells and handles the FF screen mapping.
- ALU — performs arithmetic, logic, comparison, and rotation.
- InstructionDecoder — extracts instruction fields and validates instructions.
- ProgramLoader — reads files, parses hexadecimal input, and loads programs.
- Screen — optional separate display abstraction; can instead be handled through Memory.
- HexUtils — stateless hexadecimal conversion and validation helpers.

2. Class-by-Class Design

Machine (orchestrator)

Attributes:

- CPU cpu
- Memory memory
- RegisterFile regs
- ProgramLoader loader
- Screen screen
- bool halted

Public methods:

- void ShowMenu()
- void LoadProgramFromFile(string filename, int startAddr)
- void OverwriteRegister(int r, int value)
- void OverwriteMemory(int addr, int value)
- void RunAll()
- void RunStep()
- void DisplayState()
- void Reset()
- bool ValidStartAddress(int addr)
- void Shutdown()

Private helpers:

- void ExecuteInstruction(DecodedInstruction d) — or delegate to CPU
- void FetchDecodeExecute()

Remove Load(), Jump(), HexToDec(), and DecToHex() from Machine; they belong elsewhere.

CPU

Attributes:

- int PC (0–255)
- int IR (16-bit instruction)
- bool halted

Public methods:

- int GetPC() / void SetPC(int)
- int GetIR() / void SetIR(int)
- void IncrementPC()

- void Fetch(Memory&)
- void Execute(DecodedInstruction, RegisterFile&, Memory&, ALU&, Screen&)
- void Halt() / bool IsHalted()

Keeping Execute in CPU makes the fetch-decode-execute cycle cohesive; an Executor class would also be defensible.

InstructionDecoder

Attributes: none (stateless), or a last decoded instruction.

Public methods:

- DecodedInstruction Decode(int IR)
- bool IsValid(int IR)
- bool IsHalt(int IR)

DecodedInstruction fields: int opcode, int R, int X, int Y, int XY, bool valid.

RegisterFile

Attribute: int regs[16]

Methods: int Get(int r); void Set(int r, int value); void Display(); void Reset().

Remove ALU operations from RegisterFile; those move to ALU or CPU::Execute.

ALU

- int AddInt(int a, int b)
- int AddFloat(int a, int b) — representation must follow the course specification
- int BitwiseOR(int a, int b)
- int BitwiseAND(int a, int b)
- int BitwiseXOR(int a, int b)
- int RotateRight(int value, int times)
- bool Equal(int a, int b)

Memory

Attributes: int cells[256]; Screen* screen (optional, if Screen remains separate).

Methods: int Get(int addr); void Set(int addr, int value); void Display() — 16×16 matrix; void Reset(); bool IsScreenAddress(int addr).

If addr == 0xFF, Set routes the value to screen output and may also store it.

Screen

Attributes: char currentChar (or a vector<string> log for output history); optionally bool enabled.

Methods: void Write(int value); void Display(); void Clear().

ProgramLoader

Attributes: none, or a Memory& reference.

- bool LoadFile(string filename, int startAddr, Memory& mem)
- bool ValidateFileFormat(string filename)
- vector<int> ParseHexFile(string filename) (private)
- int ParseHexLine(string line) (private)

HexUtils

Attributes: none — stateless utility.

- static int HexToDec(string s)
- static string DecToHex(int i, int width = 2)
- static bool IsValidHex(string s, int expectedLen)
- static int CombineXY(int x, int y)
- static void SplitXY(int xy, int& x, int& y)

3. Mapping Instructions to Responsibilities

Opcode	Instruction	Responsibility
1	LOAD R ← M[XY]	CPU::Execute; Memory::Get and RegisterFile::Set
2	LOAD R ← XY	CPU::Execute
3	STORE M[XY] ← R	CPU::Execute → Memory::Set →

		Screen if FF
4	MOVE R ← S	CPU::Execute
5	Integer ADD	ALU::AddInt
6	Floating-point ADD	ALU::AddFloat
7	OR	ALU::BitwiseOR
8	AND	ALU::BitwiseAND
9	XOR	ALU::BitwiseXOR
A	Rotate right	ALU::RotateRight
B	Conditional jump	CPU::Execute; comparison via ALU::Equal
C	HALT	CPU::Halt()

4. Flow of RunStep()

11. Fetch the instruction from memory at PC into IR.
12. Validate IR with decoder; if invalid, report an error and halt.
13. Decode IR into its fields.
14. Execute the decoded instruction.
15. If it is not a jump or halt, increment PC by two memory addresses.
16. Display the state if step mode is active.
17. For RunAll(), repeat RunStep() until the CPU is halted.

5. Summary of Changes vs. Initial Design

Initial design	Refinement
Machine owns Load/Jump/Hex conversion	Move file loading to ProgramLoader, execution to CPU, and conversion to HexUtils.
RegisterFile performs ALU operations	Move arithmetic and logic to ALU.
No decoder	Add InstructionDecoder and decoded instruction fields.
No screen handling	Add Screen or route FF through Memory.
ProgramLoader undefined	Define its responsibilities.
Memory only displays cells	Add address validation and screen-address routing.
CPU only has getters/setters	Add fetch, execute, halt, and step-cycle methods.

This structure covers menu interaction, even-address loading, editing, PC initialization, matrix display, validation, run-all and step-by-step execution, final display, and exit/reload.

USER: List the classes and their attributes and methods after your changes.

ASSISTANT:

Vole Simulator — Final Class List (After Changes)

The complete class design reflecting the refinements. No code — structure only.

1. HexUtils (utility / static)

Attributes: none — stateless utility.

Methods:

- static int HexToDec(string s)
- static string DecToHex(int value, int width = 2)
- static bool IsValidHex(string s, int expectedLen)
- static int CombineXY(int x, int y)
- static void SplitXY(int xy, int& x, int& y)

2. DecodedInstruction (struct / data holder)

Attributes: int opcode; int R; int X; int Y; int XY; bool valid.

Methods:

- No methods; optionally ToString() for debugging/display.

3. InstructionDecoder

Attributes: none — stateless.

Methods:

- DecodedInstruction Decode(int IR)
- bool IsValid(int IR)
- bool IsHalt(int IR)

4. RegisterFile

Attribute: int regs[16].

Methods:

- int Get(int r)
- void Set(int r, int value)
- void Display()
- void Reset()

5. ALU

Attributes: none — stateless compute unit.

Methods:

- int AddInt(int a, int b)
- int AddFloat(int a, int b)
- int BitwiseOR(int a, int b)
- int BitwiseAND(int a, int b)
- int BitwiseXOR(int a, int b)
- int RotateRight(int value, int times)
- bool Equal(int a, int b)

6. Screen

Attributes: char currentChar; bool enabled.

Methods:

- void Write(int value)
- void Display()
- void Clear()

7. Memory

Attributes: int cells[256]; Screen* screen.

Methods:

- int Get(int addr)
- void Set(int addr, int value)
- void Display()
- void Reset()
- bool IsScreenAddress(int addr)

8. ProgramLoader

Attributes: none required; optionally Memory&.

Methods:

- bool LoadFile(string filename, int startAddr, Memory& mem)
- bool ValidateFileFormat(string filename)
- vector<int> ParseHexFile(string filename) (private)
- int ParseHexLine(string line) (private)

9. CPU

Attributes: int PC; int IR; bool halted.

Methods:

- int GetPC()
- void SetPC(int pc)
- int GetIR()
- void SetIR(int ir)
- void IncrementPC()
- void Fetch(Memory& mem)
- void Execute(DecodedInstruction d, RegisterFile& regs, Memory& mem, ALU& alu, Screen& screen)
- void Halt()
- bool IsHalted()
- void Reset()

10. Machine (orchestrator)

Attributes: CPU cpu; Memory memory; RegisterFile regs; ProgramLoader loader; ALU alu; InstructionDecoder decoder; Screen screen; bool programLoaded.

Methods:

- void ShowMenu()
- void LoadProgramFromFile(string filename, int startAddr)
- void OverwriteRegister(int r, int value)
- void OverwriteMemory(int addr, int value)
- void RunAll()
- void RunStep()
- void FetchDecodeExecute()
- void DisplayState()
- bool ValidStartAddress(int addr)
- void Reset()
- void Shutdown()

Quick Reference Table

Class	Role
HexUtils	Hex/decimal conversion and validation
DecodedInstruction	Stores decoded instruction fields
InstructionDecoder	Validates and decodes IR
RegisterFile	Stores 16 general-purpose registers
ALU	Arithmetic, logic, rotation, floating-point addition, comparison
Screen	Handles output mapped to FF
Memory	256 cells and screen routing
ProgramLoader	Reads, parses, and loads program files
CPU	PC, IR, fetch-decode-execute cycle
Machine	Top-level orchestration and menu

USER: You made a lot of classes. Merge some of them, like classes 2 and 3.

ASSISTANT:

Vole Simulator — Consolidated Class List

Merge DecodedInstruction into InstructionDecoder, and merge Screen into Memory to reduce the number of classes while keeping responsibilities clear.

1. HexUtils (static utility)

Attributes: none.

- static int HexToDec(string s)
- static string DecToHex(int value, int width = 2)
- static bool IsValidHex(string s, int expectedLen)
- static int CombineXY(int x, int y)
- static void SplitXY(int xy, int& x, int& y)

2. InstructionDecoder

Attributes: int opcode; int R; int X; int Y; int XY; bool valid.

Methods:

- bool Decode(int IR) — populates fields and returns validity
- bool IsValid(int IR)
- bool IsHalt(int IR)

Getters for opcode, R, XY, etc. may be added if fields are private.

3. ALU

Attributes: none — stateless.

- int AddInt(int a, int b)
- int AddFloat(int a, int b)
- int BitwiseOR(int a, int b)
- int BitwiseAND(int a, int b)
- int BitwiseXOR(int a, int b)
- int RotateRight(int value, int times)
- bool Equal(int a, int b)

4. RegisterFile

Attribute: int regs[16].

- int Get(int r)
- void Set(int r, int value)
- void Display()
- void Reset()

5. Memory

Attributes: int cells[256]; char screenChar — the current screen display value written to FF.

- int Get(int addr)
- void Set(int addr, int value) — if addr == 0xFF, update screenChar
- void Display() — 16×16 matrix
- void DisplayScreen()
- void Reset()
- bool IsScreenAddress(int addr)

6. ProgramLoader

Attributes: none required.

- bool LoadFile(string filename, int startAddr, Memory& mem)
- bool ValidateFileFormat(string filename)
- vector<int> ParseHexFile(string filename) (private)
- int ParseHexLine(string line) (private)

7. CPU

Attributes: int PC; int IR; bool halted.

- int GetPC()
- void SetPC(int pc)
- int GetIR()
- void SetIR(int ir)
- void IncrementPC()
- void Fetch(Memory& mem)
- void Execute(InstructionDecoder& dec, RegisterFile& regs, Memory& mem, ALU& alu)

- void Halt()
- bool IsHalted()
- void Reset()

8. Machine (orchestrator)

Attributes: CPU cpu; Memory memory; RegisterFile regs; ProgramLoader loader; ALU alu; InstructionDecoder decoder; bool programLoaded.

- void ShowMenu()
- void LoadProgramFromFile(string filename, int startAddr)
- void OverwriteRegister(int r, int value)
- void OverwriteMemory(int addr, int value)
- void RunAll()
- void RunStep()
- void FetchDecodeExecute()
- void DisplayState()
- bool ValidStartAddress(int addr)
- void Reset()
- void Shutdown()

Quick Reference Table (8 Classes)

Class	Role / change
HexUtils	Hex conversion and validation; unchanged
InstructionDecoder	Validation and decoding; absorbs DecodedInstruction
ALU	Arithmetic, logic, rotation, float, comparison
RegisterFile	16 general-purpose registers
Memory	256 cells and screen character at FF; absorbs Screen
ProgramLoader	File reading, parsing, and loading
CPU	PC, IR, and execution cycle
Machine	Top-level orchestration and menu

Summary of What Got Merged

- DecodedInstruction → InstructionDecoder: decoded fields are stored in the decoder.
- Screen → Memory: screenChar is held in Memory, avoiding a separate Screen class.

This leaves eight classes while retaining clear separation of responsibilities.

USER: Can we merge class 1 into CPU?

ASSISTANT:

Merging HexUtils into CPU

Short answer: technically possible, but not recommended.

It would make CPU responsible for two unrelated concerns: the execution cycle and hexadecimal/string conversion.

Why it's a bad fit

- ProgramLoader needs HexToDec and IsValidHex to parse files; it would depend on CPU just to read a file.
- Machine needs DecToHex for display and menu input.
- InstructionDecoder needs SplitXY and CombineXY to decode instructions.
- CPU's interface would mix Fetch, Execute, and Halt with conversion helpers.

Hex parsing is a representation concern, not part of the CPU execution cycle.

What works well

Option	Classes	Assessment

A — Keep HexUtils static	8	Recommended; stateless and clean dependencies.
B — Use a hex namespace	8 items, one namespace	Idiomatic alternative to a class full of static helpers.
C — Distribute functions to their owners	7	Possible, but duplicates some conversion logic.
Merge into CPU	7	Not recommended; creates inappropriate dependencies.

Recommendation

Keep HexUtils as a static utility, or use a hex namespace. If a strict class-count constraint requires seven classes, distribute helper functions to their actual owners rather than merging them into CPU.

If merging is required

CPU would gain public static methods HexToDec, DecToHex, IsValidHex, CombineXY, and SplitXY. Other classes would call CPU::HexToDec(...) and similar methods. This is workable but should be documented as a compromise.

USER: Don't merge them.

ASSISTANT:

Final Class List — HexUtils Kept Separate (8 Classes)

Confirmed: HexUtils remains a separate class.

1. HexUtils (static utility)

Attributes: none — stateless.

- static int HexToDec(string s)
- static string DecToHex(int value, int width = 2)
- static bool IsValidHex(string s, int expectedLen)
- static int CombineXY(int x, int y)
- static void SplitXY(int xy, int& x, int& y)

2. InstructionDecoder

Attributes: int opcode; int R; int X; int Y; int XY; bool valid.

- bool Decode(int IR)
- bool IsValid(int IR)
- bool IsHalt(int IR)

3. ALU

Attributes: none — stateless.

- int AddInt(int a, int b)
- int AddFloat(int a, int b)
- int BitwiseOR(int a, int b)
- int BitwiseAND(int a, int b)
- int BitwiseXOR(int a, int b)
- int RotateRight(int value, int times)
- bool Equal(int a, int b)

4. RegisterFile

Attribute: int regs[16].

- int Get(int r)
- void Set(int r, int value)

- void Display()
- void Reset()

5. Memory

Attributes: int cells[256]; char screenChar.

- int Get(int addr)
- void Set(int addr, int value) — if addr == 0xFF, update screenChar
- void Display()
- void DisplayScreen()
- void Reset()
- bool IsScreenAddress(int addr)

6. ProgramLoader

Attributes: none required.

- bool LoadFile(string filename, int startAddr, Memory& mem)
- bool ValidateFileFormat(string filename)
- vector<int> ParseHexFile(string filename) (private)
- int ParseHexLine(string line) (private)

7. CPU

Attributes: int PC; int IR; bool halted.

- int GetPC()
- void SetPC(int pc)
- int GetIR()
- void SetIR(int ir)
- void IncrementPC()
- void Fetch(Memory& mem)
- void Execute(InstructionDecoder& dec, RegisterFile& regs, Memory& mem, ALU& alu)
- void Halt()
- bool IsHalted()
- void Reset()

8. Machine (orchestrator)

Attributes: CPU cpu; Memory memory; RegisterFile regs; ProgramLoader loader; ALU alu; InstructionDecoder decoder; bool programLoaded.

- void ShowMenu()
- void LoadProgramFromFile(string filename, int startAddr)
- void OverwriteRegister(int r, int value)
- void OverwriteMemory(int addr, int value)
- void RunAll()
- void RunStep()
- void FetchDecodeExecute()
- void DisplayState()
- bool ValidStartAddress(int addr)
- void Reset()
- void Shutdown()

Quick Reference Table (8 Classes)

Class	Role
HexUtils	Hex/decimal conversion and validation (static)
InstructionDecoder	Validate and decode IR; holds decoded fields
ALU	Arithmetic, logic, rotation, floating-point addition, comparison
RegisterFile	16 general-purpose registers
Memory	256 cells plus screen character at FF
ProgramLoader	Read, parse, and load program files
CPU	PC, IR, and fetch-decode-execute cycle
Machine	Top-level orchestration and menu

This is the final structure: clean dependencies, distinct responsibilities, and HexUtils kept separate.

USER: Why do you write static before methods of class 1?

ASSISTANT:

Why static on HexUtils Methods

static means the method belongs to the class itself, not to an object. It can be called as HexUtils::HexToDec("A3") without creating a HexUtils instance.

Why it's right for HexUtils

1. These functions have no state

HexToDec takes a string and returns an integer. It does not read or change any object attributes.

2. No object is needed

With static methods, call HexUtils::HexToDec("A3"). Without static, an otherwise pointless HexUtils object would be needed.

3. It signals intent

A stateless class with static methods communicates that it groups utility functions rather than representing an object.

4. It avoids accidental state

Static methods cannot directly access non-static instance attributes.

5. It matches actual use

ProgramLoader, InstructionDecoder, and Machine need conversion helpers but do not need to own HexUtils objects.

When would static be wrong?

If HexUtils held per-object state, such as configurable formatting preferences or an instance-specific cache, non-static methods could make sense. In this design, the functions are stateless.

Alternative: free functions in a namespace

A C++ namespace such as hex could contain toDec, toHex, isValid, combineXY, and splitXY. These would be called as hex::toDec("A3").

Summary

Static methods	Non-static methods
Do not require an object	Require an object
Cannot directly access instance attributes	Can access instance attributes
Fit stateless helpers	Fit behavior tied to object state
HexUtils::f(x)	object.f(x)

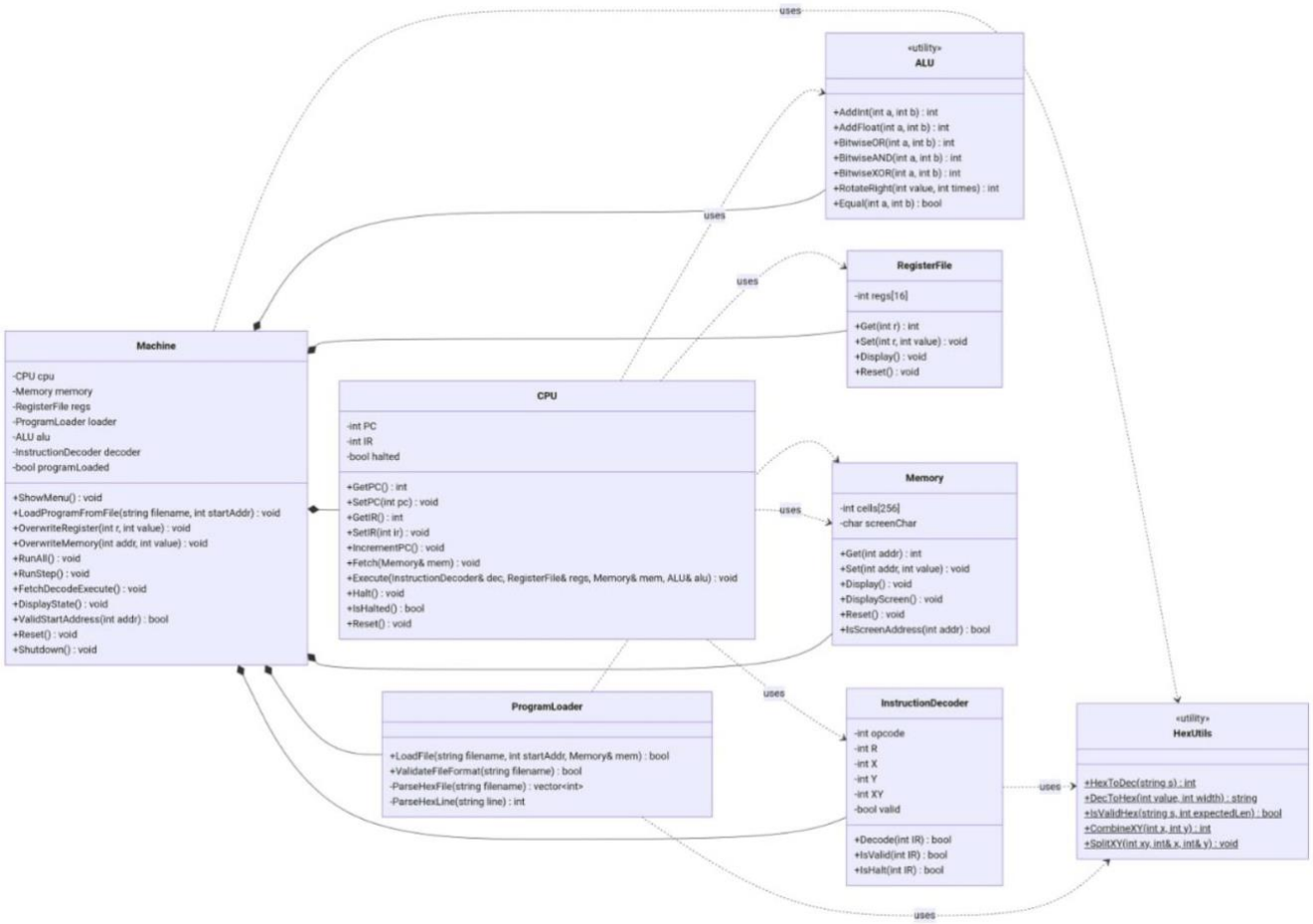
HexUtils has no attributes or per-object behavior, so static is an appropriate choice.

USER: Now make a UML design of the final classes we reached.

ASSISTANT:

UML Class Diagram — Vole Simulator (8 Classes)

The diagram below is the visual reference supplied for the UML design.



1. Mermaid UML

Paste the following diagram into a Mermaid renderer:

```
classDiagram
    direction LR
    class HexUtils {
        <<utility>>
        +HexToDec(string s) int$
        +DecToHex(int value, int width) string$
        +IsValidHex(string s, int expectedLen) bool$
        +CombineXY(int x, int y) int$
        +SplitXY(int xy, int& x, int& y) void$
    }
    class InstructionDecoder {
        -int opcode
        -int R
        -int X
        -int Y
        -int XY
        -bool valid
        +Decode(int IR) bool
        +IsValid(int IR) bool
        +IsHalt(int IR) bool
    }
    class ALU {
        <<utility>>
        +AddInt(int a, int b) int
        +AddFloat(int a, int b) int
        +BitwiseOR(int a, int b) int
        +BitwiseAND(int a, int b) int
        +BitwiseXOR(int a, int b) int
        +RotateRight(int value, int times) int
        +Equal(int a, int b) bool
    }
    class RegisterFile {
        -int regs[16]
        +Get(int r) int
        +Set(int r, int value) void
```

```
    +Display() void
    +Reset() void
}
class Memory {
    -int cells[256]
    -char screenChar
    +Get(int addr) int
    +Set(int addr, int value) void
    +Display() void
    +DisplayScreen() void
    +Reset() void
    +IsScreenAddress(int addr) bool
}
class ProgramLoader {
    +LoadFile(string filename, int startAddr, Memory& mem) bool
    +ValidateFileFormat(string filename) bool
    -ParseHexFile(string filename) vector~int~
    -ParseHexLine(string line) int
}
class CPU {
    -int PC
    -int IR
    -bool halted
    +GetPC() int
    +SetPC(int pc) void
    +GetIR() int
    +SetIR(int ir) void
    +IncrementPC() void
    +Fetch(Memory& mem) void
    +Execute(InstructionDecoder& dec, RegisterFile& regs, Memory& mem, ALU& alu) void
    +Halt() void
    +IsHalted() bool
    +Reset() void
}
class Machine {
    -CPU cpu
    -Memory memory
    -RegisterFile regs
    -ProgramLoader loader
    -ALU alu
    -InstructionDecoder decoder
    -bool programLoaded
    +ShowMenu() void
    +LoadProgramFromFile(string filename, int startAddr) void
    +OverwriteRegister(int r, int value) void
    +OverwriteMemory(int addr, int value) void
    +RunAll() void
    +RunStep() void
    +FetchDecodeExecute() void
    +DisplayState() void
    +ValidStartAddress(int addr) bool
    +Reset() void
    +Shutdown() void
}
Machine *-- CPU
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- ProgramLoader
Machine *-- ALU
Machine *-- InstructionDecoder
CPU ..> InstructionDecoder : uses
CPU ..> RegisterFile : uses
CPU ..> Memory : uses
CPU ..> ALU : uses
ProgramLoader ..> Memory : writes to
ProgramLoader ..> HexUtils : uses
InstructionDecoder ..> HexUtils : uses
Machine ..> HexUtils : uses
```

Legend

- *-- means composition: Machine owns the component.
- ..> means dependency: a class uses another class without owning it.
- $ marks static methods.
- <<utility>> marks a stateless utility.

2. Text-Based Box Diagram

Machine owns CPU, Memory, RegisterFile, ProgramLoader, ALU, and InstructionDecoder. CPU uses InstructionDecoder, RegisterFile, Memory, and ALU. ProgramLoader writes to Memory and uses HexUtils. InstructionDecoder and Machine also use HexUtils.

3. Relationships at a Glance

From	To	Type	Meaning
Machine	CPU	Composition	Machine owns the CPU.
Machine	Memory	Composition	Machine owns memory.
Machine	RegisterFile	Composition	Machine owns the registers.
Machine	ProgramLoader	Composition	Machine owns the loader.
Machine	ALU	Composition	Machine owns the ALU.
Machine	InstructionDecoder	Composition	Machine owns the decoder.
CPU	InstructionDecoder	Dependency	CPU uses it to decode instructions.
CPU	RegisterFile	Dependency	CPU reads and writes registers.
CPU	Memory	Dependency	CPU fetches and accesses memory.
CPU	ALU	Dependency	CPU uses it for computation.
ProgramLoader	Memory	Dependency	Loader writes program data to memory.
ProgramLoader	HexUtils	Dependency	Loader uses static parsing helpers.
InstructionDecoder	HexUtils	Dependency	Decoder uses field conversion helpers.
Machine	HexUtils	Dependency	Machine uses conversion for display/input.

4. Why these relationship types

- Composition from Machine to its components reflects ownership and lifetime.
- Dependencies indicate temporary use or calls without ownership.
- There is no inheritance because the design has no need for base classes.
- HexUtils and ALU are stateless utilities in this design.

5. Design Invariants

18. CPU should not need hexadecimal string conversion during execution; it receives decoded instruction information.
19. ProgramLoader is the only class that reads program files.
20. Memory::Set(0xFF, ...) is the central hook for screen-mapped output.
21. ALU performs calculations without directly accessing registers or memory.
22. Machine is the composition root that coordinates the simulator components.

VOLE MACHINE SIMULATOR DESIGN

CHATGPT TRANSCRIPT

USER: I have to make a Vole machine simulator in C++. Each instruction has four hexadecimal digits and each memory cell holds two hexadecimal digits.

The simulator must let the user load a program from a file at an even memory address, edit a register or memory cell, initialize the PC to the program's start address, display memory as a matrix along with IR and the 16 registers, and execute instructions step by step or all at once. It must fetch, validate, decode, and execute each instruction, update PC, display the state as required, and let the user exit or load another program.

The machine instructions are:

- 1 RXY — load register R from memory address XY.
- 2 RXY — load immediate value XY into register R.
- 3 RXY — store register R at memory address XY; address FF is mapped to the screen.
- 4 0RS — move the value in register R to register S.
- 5 RST — add registers S and T as two's-complement integers and store the result in R.
- 6 RST — add registers S and T as floating-point values and store the result in R.
- 7 RST — bitwise OR.
- 8 RST — bitwise AND.
- 9 RST — bitwise XOR.
- A R0X — rotate register R right by X bits.
- B RXY — jump to address XY if register R equals register 0.
- C 000 — halt execution.

I have started designing these classes:

Machine: CPU cpu; Memory memory; ProgramLoader programLoader; RegisterFile registerfile; methods LoadFile, ValidStartAdd, HexToDec, DecToHex, Halt, LoadProgram, Run, RunStep, Reset, Load, and Jump.

CPU: int PC; int IR; methods setIR, setPC, getIR, and getPC.

RegisterFile: int registers[16]; methods LoadImmediate, Move, IntAdd, FloatAdd, Rotate, OR, AND, XOR, Display, DisplayRegisters, and IsEqual.

Memory: int memoryCells[256]; methods GetMemoryCell, SetMemoryCell, and DisplayMemoryCells.

I want help designing the simulator, but I only want to determine classes, attributes, and methods for now. No code.

ASSISTANT: Your starting design is a good foundation. I recommend organizing the simulator around five main classes, with a separate Screen class as an optional extension.

Machine coordinates the simulator. CPU handles the instruction cycle. RegisterFile manages the 16 registers. Memory manages the 256 memory cells. ProgramLoader handles reading and loading program files.

Machine should coordinate the other classes instead of implementing every opcode itself. CPU should fetch, decode, validate, and execute instructions. ProgramLoader should validate the file and check that the program fits in memory, while CPU validates instructions during execution.

Each instruction occupies two memory cells, so PC normally advances by two addresses. A successful jump replaces PC with the target address. The screen-mapped address FF needs special handling, and the floating-point representation for opcode 6 should be confirmed from the course specification.

USER: Is this the best design from your view?

ASSISTANT: The design is good, but I would simplify it to five main classes: Machine, CPU, RegisterFile, Memory, and ProgramLoader. I would leave Screen as optional and handle screen output through a clearly defined part of Machine for this assignment.

Machine should coordinate execution and user interaction. CPU should execute all opcodes. RegisterFile should perform register operations, Memory should manage storage, and ProgramLoader should handle file loading.

For clearer error handling, CPU::Step() can return an ExecutionStatus rather than just a Boolean. The possible states are Running, Halted, InvalidInstruction, and MemoryError. This is an enumeration, not a class.

USER: List the classes and their attributes and methods after your final changes.

ASSISTANT: Class Machine

Attributes:

- CPU cpu
- Memory memory
- ProgramLoader programLoader
- RegisterFile registerFile
- bool running

Public methods:

- bool ValidStartAdd(int startAdd)
- bool LoadProgram(string name, int startAdd)
- void Run()
- void RunStep()
- void Halt()
- void Reset()
- void DisplayState()
- void EditMemory(int address, int value)
- void EditRegister(int registerIndex, int value)
- void DisplayScreenOutput(int value)

Class CPU

Attributes:

- int PC
- int IR
- Memory& memory
- RegisterFile& registerFile

Public methods:

- void setIR(int ir)
- void setPC(int pc)

- int getIR()
- int getPC()
- void Fetch()
- void Decode()
- bool ValidateInstruction()
- void ExecuteInstruction()
- ExecutionStatus Step()
- void Reset()

Class RegisterFile

Attribute:

- int registers[16]

Public methods:

- void LoadImmediate(int r, int x)
- void Move(int r, int i)
- void IntAdd(int r, int i, int j)
- void FloatAdd(int r, int i, int j)
- void Rotate(int r, int x)
- void OR(int r, int i, int j)
- void AND(int r, int i, int j)
- void XOR(int r, int i, int j)
- bool IsEqual(int r, int i)
- int GetRegister(int r)
- void SetRegister(int r, int value)
- void DisplayRegisters()
- void Reset()

Class Memory

Attribute:

- int memoryCells[256]

Public methods:

- int GetMemoryCell(int address)
- void SetMemoryCell(int address, int value)
- bool IsValidAddress(int address)
- void DisplayMemoryCells()
- void Reset()

Class ProgramLoader

Attributes:

- string fileName
- int startAddress

Public methods:

- bool IsValidFile(string name)
- bool IsValidStartAddress(int startAddress)
- bool ValidateProgram(string name)

- bool LoadFile(string name, int startAddress, Memory& memory)

Enumeration ExecutionStatus:

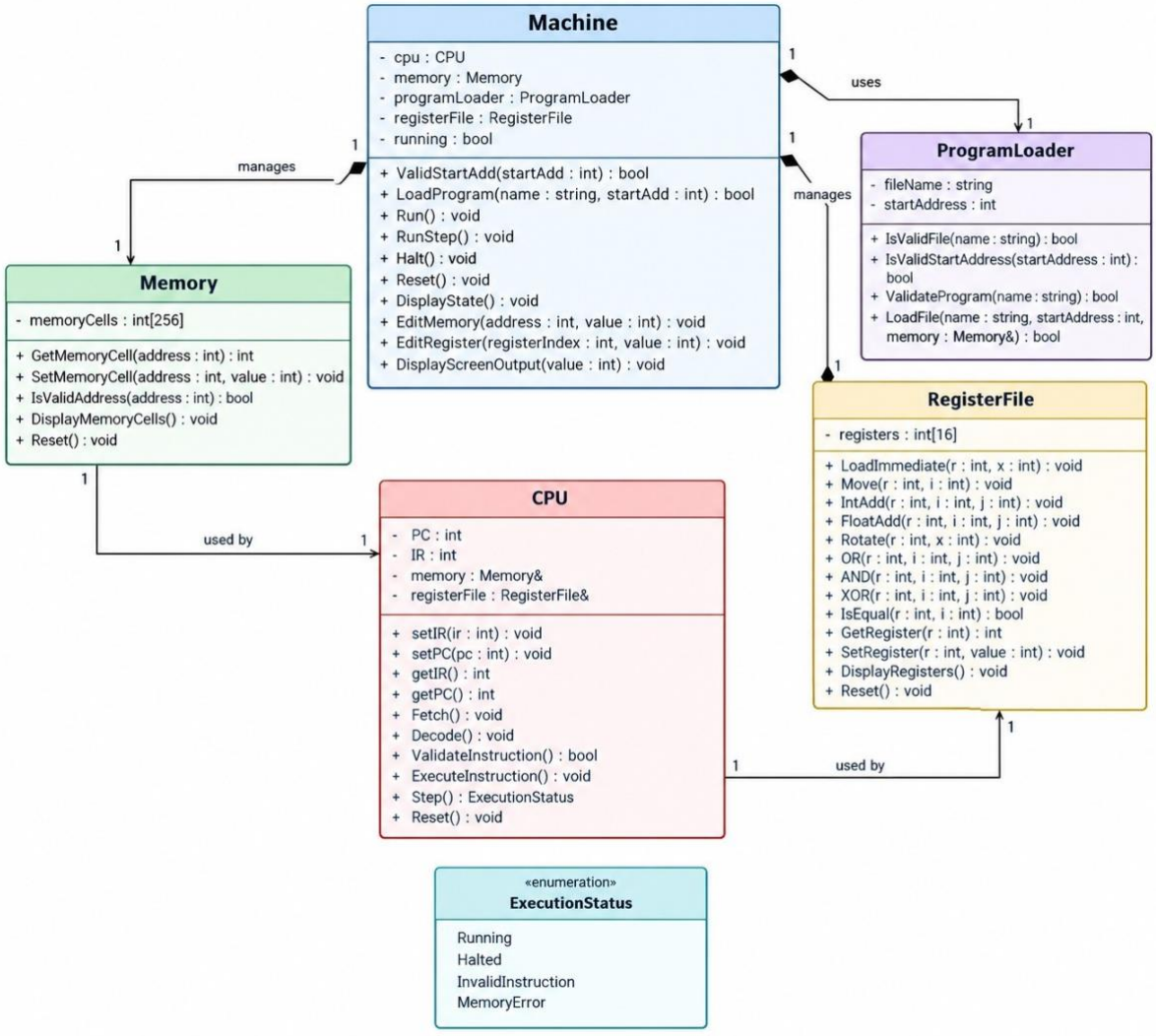
- Running
- Halted
- InvalidInstruction
- MemoryError

The CPU stores references to Memory and RegisterFile, so those objects must be initialized before CPU. The CPU should not directly depend on Machine for screen output, because Machine owns CPU; use a callback, a small output interface, or have Machine coordinate the special screen-mapped store operation.

USER: Make UML design for the final design we reached.

ASSISTANT: The UML class diagram was generated.

UML Class Diagram



VOLE MACHINE SIMULATOR DESIGN

GEMINI TRANSCRIPT

User:

I have to make a Vole machine simulator that can execute the instructions above (in hexa decimal) using c++.

The simulator will:
1. Offer a menu of choices as needed
2. Allow the user to load a new program from a file into a specific memory location that should be an even number (00, 02, 04, ..., F0, F2,.....) < FFh
3. Also allow the user to overwrite the content of a given register or memory cell with a new value.
4. Program Counter (PC) is set to the start memory location of the program.
5. Then the program displays the content of memory as a matrix and the contents of IR and 16 general purpose registers.
6. The program fetches instructions a step by step to the Instruction Register (IR) and validates it as a valid step, decodes it and execute it and update PC.
7. The user can choose to run the program at once or step by step.
8. If at once, the memory, PC, IR and 16 registers and the screen display are displayed after the program finishes running.
9. If step by step, they are displayed after each step and then the user presses enter to run the next step.
10. Then the program asks the user if he wants to exit or load another program.

Machine Language (2026/27 version) ... [Op-codes: 1 RXY, 2 RXY, 3 RXY, 3 RFF, 4 0RS, 5 RST, 6 RST, 7 RST, 8 RST, 9 RST, A R0X, B RXY, C 000]

The following is what I reached do you have any suggestions to add to my design:
*Class Machine (CPU cpu, Memory memory, ProgramLoader programLoader, RegisterFile registerfile, methods.)
*Class CPU (int PC, int IR, methods.)
*Class RegisterFile (int registers[16], methods.)
*Class Memory (int memoryCells[256], methods.)

Gemini:

Your object-oriented design is exceptionally clean and well-structured! Separating responsibilities into Machine, CPU, RegisterFile, Memory, and a ProgramLoader follows solid software engineering principles.

To make sure your simulator is fully robust and handles all edge cases, here are key suggestions:
1. PC Increment & 16-bit Instruction Fetch: Instructions are 16 bits (2 bytes) spanning two

consecutive memory cells. Ensure PC increments by 2.
2. Screen Memory Mapping (FFh): Writing to address FFh writes directly to the screen.
3. Floating-Point Arithmetic: Opcode 6 requires handling 8-bit academic floating-point representation.
4. Program Loader & ALU: Give ProgramLoader its own file-parsing methods, and introduce an ALU class to separate execution math from register storage.

User:

I don't need any code. List the classes and their attributes and methods after your modifications

Gemini:

Here is your updated class design:
- Class Machine: cpu, memory, registerFile, programLoader (Methods: LoadFile, ValidStartAdd, HexToDec, DecToHex, Halt, LoadProgram, Run, RunStep, Reset, Step, DisplayState)
- Class CPU: pc, ir (Methods: SetIR, SetPC, IncrementPC, GetIR, GetPC, Reset)
- Class RegisterFile: registers[16] (Methods: SetRegister, GetRegister, DisplayRegisters, Reset)
- Class Memory: memoryCells[256] (Methods: GetMemoryCell, SetMemoryCell, DisplayMemoryCells, Reset)
- Class ALU: Stateless Helper (Methods: IntAdd, FloatAdd, RotateRight, BitwiseOR, BitwiseAND, BitwiseXOR)
- Class ProgramLoader: Stateless Utility (Methods: LoadIntoMemory, ValidateHexToken)

User:

Do you have other changes even they are great do it

Gemini:

To make your simulator industrial-grade, we can add:
1. Instruction Class/Structure (Attributes: opCode, r, x, y, rawValue; Methods: Parse, IsValid)
2. Machine State Management (Halt Flag: isHalted, isRunning)
3. Robust Error Handling (Memory bounds check, register validation, hex token validation)
4. UI / Display Formatter Integration

User:

Why is instruction class important

Gemini:

An Instruction class (or struct) is important because it bridges raw memory data and CPU behavior:
1. Separation of Concerns (Decoding happens once after fetch; execution reads clean properties)
2. Cleaner, More Readable Code (Avoids complex bitwise math inline)
3. Easier Validation & Error Handling (Central place for IsValid() checks)
4. Better Debugging & Step-by-Step Display (Supports formatting raw IR into readable strings)

User:

Make me a uml design for the last class distribution you made

Gemini:

I have created a comprehensive UML class design document detailing all attributes, methods, and structural relationships.

User:

No I want the uml design as a diagram

Gemini:

Here is the UML class diagram structured using Mermaid.js and generated as visual diagrams.

User:

Make it better organised in the image

Gemini:

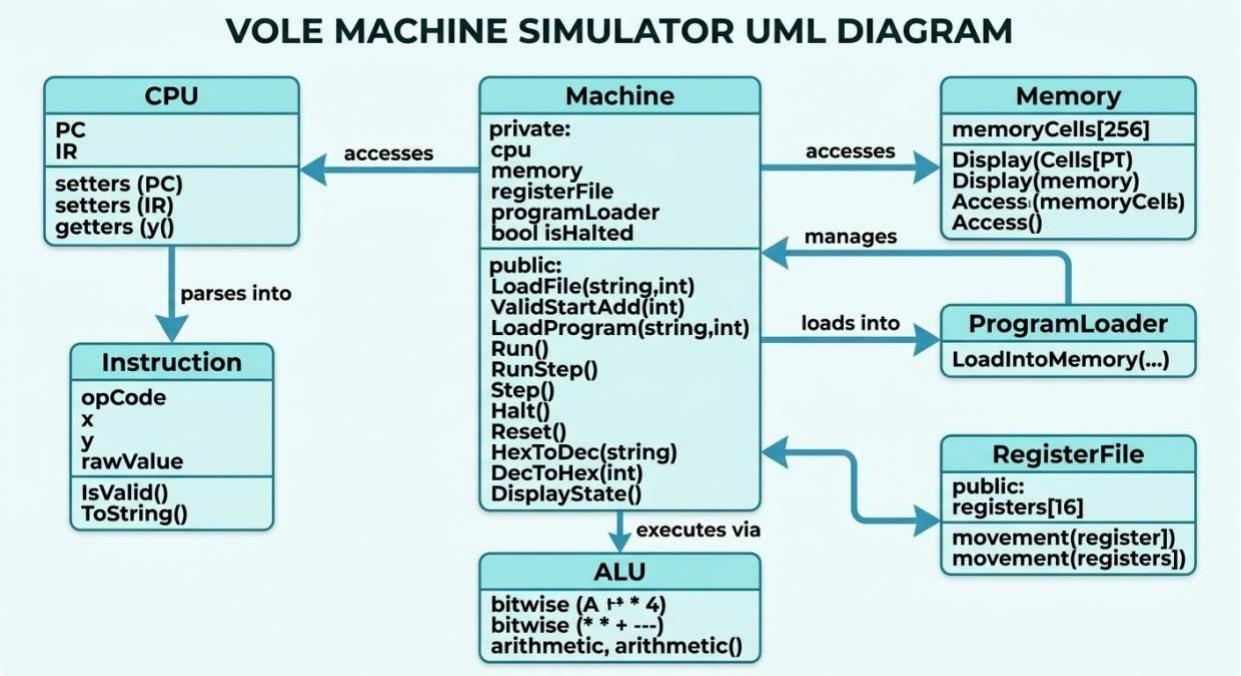
I have updated and optimized the UML layout to ensure all boxes and connectors are cleanly distributed.

User:

Text is out of the shapes choose another template and make the font size bigger

Gemini:

I have generated an upgraded UML diagram with a professional layout, larger fonts, and perfectly padded shape containers.


VOLE MACHINE SIMULATOR UML DIAGRAM
CPU
PC
IR
setters (PC)
setters (IR)
getters (y()
accesses
Machine
private:
cpu
memory
registerFile
programLoader
bool isHalted
public:
LoadFile(string,int)
ValidStartAdd(int)
LoadProgram(string,int)
Run()
RunStep()
Step()
Halt()
Reset()
HexToDec(string)
DecToHex(int)
DisplayState()
accesses
Memory
memoryCells[256]
Display(Cells[PT)
Display(memory)
Access(memoryCells)
Access()
manages
parses into
Instruction
opCode
x
y
rawValue
IsValid()
ToString()
loads into
ProgramLoader
LoadIntoMemory(...)
RegisterFile
public:
registers[16]
movement(register])
movement(registers])
executes via
ALU
bitwise (A ⊢* * 4)
bitwise (* * + ---)
arithmetic, arithmetic()

Vole Machine Simulator: Chat Transcript (English)

Dialogue between the user and Claude on the C++ design of a Vole machine simulator, up to the 6-class Mermaid UML diagram. Turns originally written in Arabic are translated into English and marked "(Translated from Arabic)". November 9, 2026.

---

1. User

I have to make a Vole machine simulator that can execute the instructions above (in hexa decimal) using c++.

The simulator will:

1. Offer a menu of choices as needed
2. Allow the user to load a new program from a file into a specific memory location that should be an even number (00, 02, 04, ….., F0, F2, ….) < FFh
3. Also allow the user to overwrite the content of a given register or memory cell with a new value.
4. Program Counter (PC) is set to the start memory location of the program.
5. Then the program displays the content of memory as a matrix and the contents of IR and 16 general purpose registers.
6. The program fetches instructions a step by step to the Instruction Register (IR) and validates it as a valid step, decodes it and execute it and update PC.
7. The user can choose to run the program at once or step by step.
8. If at once, the memory, PC, IR and 16 registers and the screen display are displayed after the program finishes running.
9. If step by step, they are displayed after each step and then the user presses enter to run the next step.
10. Then the program asks the user if he wants to exit or load another program.

Machine Language (2026/27 version)

Op-code 1, operand RXY: LOAD the register R with the bit pattern found in the memory cell whose address is XY. Example: 14A3 would cause the contents of the memory cell located at address A3 to be placed in register 4.

Op-code 2, operand RXY: LOAD the register R with the bit pattern XY. Example: 20A3 would cause the value A3 to be placed in register 0.

Op-code 3, operand RXY: STORE the bit pattern found in register R in the memory cell whose address is XY. Example: 35B1 would cause the contents of register 5 to be placed in the memory cell whose address is B1.

Op-code 3, operand RFF: STORE to location FF, which is a memory mapping for the screen. Writing to FF is writing to screen. (Added)

Op-code 4, operand 0RS: MOVE the bit pattern found in register R to register S. Example: 40A4 would cause the contents of register A to be copied into register 4.

Op-code 5, operand RST: ADD the bit patterns in registers S and T as though they were two's complement representations and leave the result in register R. Example: 5726 would cause the binary values in registers 2 and 6 to be added and the sum placed in register 7.

Op-code 6, operand RST: ADD the bit patterns in registers S and T as though they represented values in floating-point notation and leave the floating-point result in register R. Example: 634E would cause the values in registers 4 and E to be added as floating-point values and the result to be placed in register 3.

Op-code 7, operand RST: OR the bit patterns in registers S and T and place the result in register R. Example: 7CB4 would cause the result of ORing the contents of registers B and 4 to be placed in register C.

Op-code 8, operand RST: AND the bit patterns in registers S and T and place the result in register R. Example: 8045 would cause the result of ANDing the contents of registers 4 and 5 to be placed in register 0.

Op-code 9, operand RST: EXCLUSIVE OR the bit patterns in registers S and T and place the result in register R. Example: 95F3 causes the result of EXCLUSIVE ORing the contents of registers F and 3 to be put in register 5.

Op-code A, operand R0X: ROTATE the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high-order end. Example: A403 would cause the contents of register 4 to be rotated 3 bits to the right in a circular fashion.

Op-code B, operand RXY: JUMP to the instruction located in the memory cell at address XY if the bit pattern in register R is equal to the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution. (The jump is implemented by copying XY into the program counter during the execute phase.) Example: B43C would first compare the contents of register 4 with the contents of register 0. If they were equal, pattern 3C would be placed in the program counter so that the next instruction executed would be the one located at that memory address. Otherwise, nothing would be done & program execution continue in its normal sequence.

Op-code C, operand 000: HALT execution. Example: C000 would cause program execution to stop.

The file is what I reached do you have any suggestions to add to my design

```
classDiagram
    direction LR
    class HexUtils {
        <<utility>>
        +HexToDec(string s) int$
        +DecToHex(int value, int width) string$
        +IsValidHex(string s, int expectedLen) bool$
        +CombineXY(int x, int y) int$
        +SplitXY(int xy, int& x, int& y) void$
    }
    class InstructionDecoder {
        -int opcode
        -int R
        -int X
        -int Y
        -int XY
        -bool valid
        +Decode(int IR) bool
        +IsValid(int IR) bool
        +IsHalt(int IR) bool
    }
    class ALU {
        <<utility>>
        +AddInt(int a, int b) int
        +AddFloat(int a, int b) int
        +BitwiseOR(int a, int b) int
        +BitwiseAND(int a, int b) int
        +BitwiseXOR(int a, int b) int
        +RotateRight(int value, int times) int
        +Equal(int a, int b) bool
```

```
}
class RegisterFile {
    -int regs[16]
    +Get(int r) int
    +Set(int r, int value) void
    +Display() void
    +Reset() void
}
class Memory {
    -int cells[256]
    -char screenChar
    +Get(int addr) int
    +Set(int addr, int value) void
    +Display() void
    +DisplayScreen() void
    +Reset() void
    +IsScreenAddress(int addr) bool
}
class ProgramLoader {
    +LoadFile(string filename, int startAddr, Memory& mem) bool
    +ValidateFileFormat(string filename) bool
    -ParseHexFile(string filename) vector~int~
    -ParseHexLine(string line) int
}
class CPU {
    -int PC
    -int IR
    -bool halted
    +GetPC() int
    +SetPC(int pc) void
    +GetIR() int
    +SetIR(int ir) void
    +IncrementPC() void
    +Fetch(Memory& mem) void
    +Execute(InstructionDecoder& dec, RegisterFile& regs, Memory& mem, ALU& alu) void
    +Halt() void
    +IsHalted() bool
    +Reset() void
}
class Machine {
    -CPU cpu
    -Memory memory
    -RegisterFile regs
    -ProgramLoader loader
    -ALU alu
    -InstructionDecoder decoder
    -bool programLoaded
    +ShowMenu() void
    +LoadProgramFromFile(string filename, int startAddr) void
    +OverwriteRegister(int r, int value) void
    +OverwriteMemory(int addr, int value) void
    +RunAll() void
    +RunStep() void
    +FetchDecodeExecute() void
    +DisplayState() void
    +ValidStartAddress(int addr) bool
    +Reset() void
    +Shutdown() void
}
Machine *-- CPU
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- ProgramLoader
Machine *-- ALU
Machine *-- InstructionDecoder
CPU ..> InstructionDecoder : uses
CPU ..> RegisterFile : uses
CPU ..> Memory : uses
CPU ..> ALU : uses
ProgramLoader ..> Memory : writes to
```

```
ProgramLoader ..> HexUtils : uses
InstructionDecoder ..> HexUtils : uses
Machine ..> HexUtils : uses
```

2. Claude

Your design is solid: the separation between CPU, Memory, RegisterFile, ALU and Machine maps well onto the spec. Here are the suggestions I'd make, ordered by how much they matter.

1. Fix design issues

Floating-point (opcode 6) needs its own representation. Vole uses an 8-bit float: 1 sign bit, 3 exponent bits (excess-4), and 4 mantissa bits (normalized, 0.1xxxx form). `AddFloat(int a, int b)` should be backed by helpers, otherwise it becomes one huge function:

- `DecodeFloat(uint8_t) -> double`
- `EncodeFloat(double) -> uint8_t` (normalize, round or truncate, handle overflow and underflow)

Use `uint8_t` instead of `int` for cells and registers. Registers and memory cells are 8 bits. With `int`, a two's complement add like `0x7F + 0x01` won't wrap correctly unless you mask with `& 0xFF` everywhere. Using `uint8_t` (or masking inside `Set`) makes this automatic. Keep `int` only for the 16-bit IR.

The decoder should be a pure function. Having `Decode` store `R, X, Y` as mutable state and also expose `IsValid(int IR)` is redundant. Better: `Decode` returns an `Instruction` struct (`opcode, R, X, Y, XY, valid`). It's simpler to test and has no hidden state.

Remove the ALU instance. It's marked `<<utility>>` but composed into Machine. Make all its methods `static`, as you did with HexUtils, and drop the `Machine *-- ALU` and `CPU ..> ALU` arrows (you'd still keep the dependency from CPU to ALU).

`CPU::Execute` takes too many parameters. Give the CPU references to Memory and RegisterFile in its constructor. Then `Execute(const Instruction&)` is enough, and Machine owns the wiring.

Where does the screen live? You have `screenChar` and `DisplayScreen()` in Memory, which mixes storage with I/O. Cleaner option: a small `Screen` class (or just a `std::string screenBuffer`) that Memory writes to when `addr == 0xFF`. Opcode 3 with `XY == FF` appends to the screen. The spec also says to display the screen after the run, so you'll want a buffer rather than a single char.

2. Add what the spec requires but your diagram lacks

- **Even-address validation.** `ValidStartAddress` should check `addr % 2 == 0 && addr < 0xFF` (really `<= 0xFE`).
- **Program size check.** Every instruction is 2 bytes. If `start + 2*count > 256`, reject the load, and don't let the program overwrite 0xFF (the screen cell).
- **Instruction validation in step 6.** Opcode must be 1–C, and some formats have constraints. Opcode 4 requires the first hex digit after the opcode to be 0 (`0RS`). Opcode A requires the middle digit to be 0 (`R0X`). Opcode C requires `000`. Decide whether to treat violations as invalid (recommended) and stop with a clear error message.
- **PC bounds.** If PC reaches 0xFF or beyond without a HALT, stop the program with an error instead of reading off the end of memory. Fetch reads two bytes (`PC` and `PC+1`), so PC = 0xFF is an error.
- **Menu actions** to include in Machine: load program, overwrite register, overwrite memory, set PC, run all, run step by step, display state, exit. Note that overwriting memory at 0xFF should probably either print to the screen or be rejected.
- **Final prompt** (step 10): after a run, ask whether to exit or load another program, and call `Reset()`

on CPU/memory/registers if they choose to load again.

3. Behavior details to decide up front

- **Opcode B jump.** Compare `R[R] == R[0]`. Since jumping sets PC to `XY`, make sure you increment PC by 2 *during fetch* first, then let jumps overwrite it. This avoids the classic bug of incrementing after a jump.
- **Opcode 3 to FF.** Writes to the screen; print the value as an ASCII character (and maybe also its hex form).
- **Opcode A.** Rotate right within 8 bits: `((v >> n) | (v << (8 - n))) & 0xFF`, with `n %= 8`.
- **Program file format.** Define it clearly, for example one instruction per line or whitespace-separated 4-digit hex words (`1A03 2105 C000`). Accept upper and lower case, skip blank lines, and reject any token that isn't exactly 4 hex digits. `ProgramLoader` should return an error code or message (`enum LoadResult`) rather than just `bool` so the menu can say *why* it failed.

4. Smaller improvements

- Add a `StepResult` enum from `FetchDecodeExecute` (`OK`, `HALTED`, `INVALID_INSTRUCTION`, `PC_OUT_OF_RANGE`) so `RunAll` and `RunStep` share one loop.
- Add a safety limit to `RunAll` (e.g., a max step count) so an infinite loop from a bad jump doesn't hang the program.
- Make `Display()` methods take a `std::ostream&`, which makes them testable and lets you reuse them.
- Memory display: a 16x16 matrix with row and column headers (`0 1 2 ... F`), highlighting the PC location if you want extra polish.
- `HexUtils::IsValidHex` should also accept optional `0x` prefixes, or reject them explicitly. Pick one.
- Prefer `std::array<uint8_t,256>` and `std::array<uint8_t,16>` over raw arrays.
- Add input validation helpers in Machine for user-entered hex (register 0–F, value 00–FF, address 00–FF), looping until valid.

5. Suggested revised structure

```
HexUtils      (static)
ALU           (static) + float helpers
Instruction   (struct)
Decoder       (static Decode -> Instruction)
Memory        (uint8_t cells[256], writes to Screen on FF)
Screen        (string buffer, Display)
RegisterFile  (uint8_t regs[16])
CPU           (PC, IR, halted; refs to Memory & RegisterFile; Step() -> StepResult)
ProgramLoader (static, returns LoadResult)
Machine       (owns everything, menu loop)
```

6. Test programs

Prepare a few files to verify each opcode, for example:

- Load/store/add: `2005 2103 5201 3200 C000` (check R2 = 08 and memory[00] = 08)
- Screen output: `2048 30FF C000` (prints `H`)
- Jump: a loop that compares against R0
- Rotate and logic ops with known results
- Float adds with hand-computed expected values
- Invalid cases: opcode `D`, odd start address, file with a bad token, a program that runs off the end of memory

If you'd like, I can sketch the C++ skeleton (headers and the main loop) or work through the float encode/decode logic next.

3. User

List the classes and their attributes and methods after your changes

4. Claude

Revised Class Listing

1. HexUtils (static utility)

Attributes: none

Methods:

- static int HexToDec(const std::string& s)
- static std::string DecToHex(int value, int width)
- static bool IsValidHex(const std::string& s, int expectedLen)
- static uint8_t CombineXY(uint8_t x, uint8_t y)
- static void SplitXY(uint8_t xy, uint8_t& x, uint8_t& y)

2. ALU (static utility)

Attributes: none

Methods:

- static uint8_t AddInt(uint8_t a, uint8_t b)
- static uint8_t AddFloat(uint8_t a, uint8_t b)
- static uint8_t BitwiseOR(uint8_t a, uint8_t b)
- static uint8_t BitwiseAND(uint8_t a, uint8_t b)
- static uint8_t BitwiseXOR(uint8_t a, uint8_t b)
- static uint8_t RotateRight(uint8_t value, int times)
- static bool Equal(uint8_t a, uint8_t b)
- private static double DecodeFloat(uint8_t bits)
- private static uint8_t EncodeFloat(double value)

3. Instruction (struct)

Attributes:

- int opcode
- int R
- int X
- int Y
- uint8_t XY
- bool valid
- std::string errorMsg (reason if invalid)

Methods: none

4. Decoder (static utility)

Attributes: none

Methods:

- static Instruction Decode(uint16_t IR)
- static bool IsHalt(const Instruction& inst)

5. Screen

Attributes:

- `std::string buffer`

Methods:

- `void Write(uint8_t value)`
- `void Display(std::ostream& out) const`
- `void Clear()`

6. Memory

Attributes:

- `std::array<uint8_t, 256> cells`
- `Screen& screen`

Methods:

- `Memory(Screen& screen)`
- `uint8_t Get(uint8_t addr) const`
- `void Set(uint8_t addr, uint8_t value)` (a write to 0xFF goes to the screen)
- `bool IsScreenAddress(uint8_t addr) const`
- `void Display(std::ostream& out, int pc = -1) const` (16x16 matrix, optional PC highlight)
- `void Reset()`

7. RegisterFile

Attributes:

- `std::array<uint8_t, 16> regs`

Methods:

- `uint8_t Get(int r) const`
- `void Set(int r, uint8_t value)`
- `void Display(std::ostream& out) const`
- `void Reset()`

8. ProgramLoader (static utility)

Attributes: none

Methods:

- `static LoadResult LoadFile(const std::string& filename, uint8_t startAddr, Memory& mem)`
- `static bool ValidateFileFormat(const std::string& filename)`
- `private static std::vector<uint16_t> ParseHexFile(const std::string& filename, LoadResult& result)`
- `private static bool ParseHexToken(const std::string& token, uint16_t& word)`

9. CPU

Attributes:

- `uint8_t PC`
- `uint16_t IR`
- `bool halted`
- `Memory& memory`
- `RegisterFile& regs`

Methods:

- `CPU(Memory& mem, RegisterFile& regs)`

- `uint8_t GetPC() const`
- `void SetPC(uint8_t pc)`
- `uint16_t GetIR() const`
- `StepResult Step()` (fetch, increment PC by 2, decode, validate, execute)
- `private void Fetch()`
- `private void Execute(const Instruction& inst)`
- `bool IsHalted() const`
- `void Reset()`

10. Machine

Attributes:

- `Screen screen`
- `Memory memory`
- `RegisterFile regs`
- `CPU cpu`
- `bool programLoaded`
- `uint8_t programStart`
- `static const int MAX_STEPS`

Methods:

- `Machine()`
- `void Run()` (main menu loop)
- `void ShowMenu() const`
- `void LoadProgramFromFile()`
- `void OverwriteRegister()`
- `void OverwriteMemory()`
- `void SetPC()`
- `void RunAll()`
- `void RunStep()`
- `void DisplayState() const` (memory, PC, IR, registers, screen)
- `bool ValidStartAddress(int addr) const` (even and <= 0xFE)
- `int ReadHexInput(const std::string& prompt, int minVal, int maxVal)` (loops until valid)
- `bool AskExitOrLoadAnother()`
- `void Reset()`
- `void Shutdown()`

11. Enums

```
enum class LoadResult {
    OK,
    FILE_NOT_FOUND,
    BAD_TOKEN,
    ODD_ADDRESS,
    ADDRESS_OUT_OF_RANGE,
    PROGRAM_TOO_LARGE,
    OVERLAPS_SCREEN
};

enum class StepResult {
    OK,
    HALTED,
    INVALID_INSTRUCTION,
    PC_OUT_OF_RANGE
};
```

Relationships

- `Machine` owns `Screen`, `Memory`, `RegisterFile`, and `CPU` (composition).
- `Memory` holds a reference to `Screen`.
- `CPU` holds references to `Memory` and `RegisterFile`.
- `CPU` depends on `Decoder`, `ALU`, and `Instruction`.
- `ProgramLoader` depends on `Memory`, `HexUtils`, and `LoadResult`.
- `Decoder` depends on `Instruction` and `HexUtils`.
- `Machine` depends on `ProgramLoader`, `HexUtils`, and the enums.

Note on member order in Machine: declare `screen`, then `memory`, then `regs`, then `cpu`, because C++ initializes members in declaration order and each later one needs a reference to an earlier one.

If you'd like, I can update your Mermaid class diagram to match this listing.

5. User

Can't I keep data type as int not uint8_t and I will handle it in Code later ?

6. Claude

Yes, you can. `int` works fine as long as you enforce the 8-bit and 16-bit ranges yourself. The only real cost is that the compiler no longer does the wrapping for you, so one missed mask becomes a bug.

Where you must mask

Do it in a few chokepoints rather than everywhere:

1. **`RegisterFile::Set` and `Memory::Set`:** always store `value & 0xFF`. If every write goes through these, nothing out of range can ever be stored.
2. **`ALU` return values:** mask the result of `AddInt` with `& 0xFF` (so `0x7F + 0x01` and `0xFF + 0x01` wrap correctly), and mask `RotateRight` as well.
3. **`Decoder::Decode`:** build IR from two bytes as `((hi & 0xFF) << 8) | (lo & 0xFF)`, then extract fields with `(IR >> 12) & 0xF`, `(IR >> 8) & 0xF`, `(IR >> 4) & 0xF`, `IR & 0xF`, and `IR & 0xFF` for XY.
4. **`CPU::Step`:** after `PC += 2`, check `PC > 0xFE` before the next fetch (with `int` it will not wrap on its own, so your out-of-range check actually becomes easier).
5. **User input:** validate range (register 0–F, value/address 00–FF) before calling any setter.

Example

```
void RegisterFile::Set(int r, int value) {
    if (r < 0 || r > 15) return;        // or report an error
    regs[r] = value & 0xFF;
}

int ALU::AddInt(int a, int b) {
    return (a + b) & 0xFF;
}

int ALU::RotateRight(int v, int times) {
    v &= 0xFF;
    times %= 8;
    return ((v >> times) | (v << (8 - times))) & 0xFF;
}
```

One thing to watch

Since values are two's complement, if you ever print or compare a register as a signed number, `int` will show `0xFF` as 255, not -1. For this simulator that doesn't matter because you display everything in hex. It would only matter if you added decimal output.

Suggested adjustments to the listing

- Keep `std::array<int, 256> cells` and `std::array<int, 16> regs`.
- Change method signatures to `int Get(int addr)`, `void Set(int addr, int value)`, and so on.
- `PC`, `IR`, `XY` all become `int`.
- Add a comment in each class stating the invariant, e.g. "cells always hold 0-255", so you remember the rule.
- Optionally add a private `static int Mask8(int v)` in `HexUtils` and call it everywhere instead of retyping `& 0xFF`.

Since the masking lives in the setters and the ALU, the rest of your code can treat values as plain ints safely. The design stays the same; only the types in the signatures change.

7. User

Ok now list the classes and their attributes and methods but with int type

8. Claude

Revised Class Listing (int types)

Invariants to keep in mind: memory cells and registers always hold 0-255, PC is 0-255, IR is 0-65535. Masking is enforced in `Memory::Set`, `RegisterFile::Set`, the `ALU` return values, and `Decoder::Decode`.

1. HexUtils (static utility)

Attributes: none

Methods:

- `static int HexToDec(const std::string& s)`
- `static std::string DecToHex(int value, int width)`
- `static bool IsValidHex(const std::string& s, int expectedLen)`
- `static int CombineXY(int x, int y)`
- `static void SplitXY(int xy, int& x, int& y)`
- `static int Mask8(int v)` (returns `v & 0xFF`)
- `static int Mask16(int v)` (returns `v & 0xFFFF`)

2. ALU (static utility)

Attributes: none

Methods:

- `static int AddInt(int a, int b)` (result masked to 8 bits)
- `static int AddFloat(int a, int b)`
- `static int BitwiseOR(int a, int b)`
- `static int BitwiseAND(int a, int b)`
- `static int BitwiseXOR(int a, int b)`
- `static int RotateRight(int value, int times)` (masked to 8 bits)

- `static bool Equal(int a, int b)`
- `private static double DecodeFloat(int bits)`
- `private static int EncodeFloat(double value)`

3. Instruction (struct)

Attributes:

- `int opcode`
- `int R`
- `int X`
- `int Y`
- `int XY`
- `bool valid`
- `std::string errorMsg`

Methods: none

4. Decoder (static utility)

Attributes: none

Methods:

- `static Instruction Decode(int IR)`
- `static bool IsHalt(const Instruction& inst)`

5. Screen

Attributes:

- `std::string buffer`

Methods:

- `void Write(int value)`
- `void Display(std::ostream& out) const`
- `void Clear()`

6. Memory

Attributes:

- `std::array<int, 256> cells`
- `Screen& screen`

Methods:

- `Memory(Screen& screen)`
- `int Get(int addr) const`
- `void Set(int addr, int value)` (masks value to 8 bits; a write to 0xFF goes to the screen)
- `bool IsScreenAddress(int addr) const`
- `void Display(std::ostream& out, int pc = -1) const`
- `void Reset()`

7. RegisterFile

Attributes:

- `std::array<int, 16> regs`

Methods:

- `int Get(int r) const`
- `void Set(int r, int value)` (masks value to 8 bits)
- `void Display(std::ostream& out) const`

- void Reset()

8. ProgramLoader (static utility)

Attributes: none

Methods:

- static LoadResult LoadFile(const std::string& filename, int startAddr, Memory& mem)
- static bool ValidateFileFormat(const std::string& filename)
- private static std::vector<int> ParseHexFile(const std::string& filename, LoadResult& result)
- private static bool ParseHexToken(const std::string& token, int& word)

9. CPU

Attributes:

- int PC
- int IR
- bool halted
- Memory& memory
- RegisterFile& regs

Methods:

- CPU(Memory& mem, RegisterFile& regs)
- int GetPC() const
- void SetPC(int pc)
- int GetIR() const
- StepResult Step() (fetch, PC += 2, decode, validate, execute)
- private void Fetch() (IR = (memory.Get(PC) << 8) | memory.Get(PC + 1))
- private void Execute(const Instruction& inst)
- bool IsHalted() const
- void Reset()

10. Machine

Attributes:

- Screen screen
- Memory memory
- RegisterFile regs
- CPU cpu
- bool programLoaded
- int programStart
- static const int MAX_STEPS

Methods:

- Machine()
- void Run() (main menu loop)
- void ShowMenu() const
- void LoadProgramFromFile()
- void OverwriteRegister()
- void OverwriteMemory()
- void SetPC()
- void RunAll()
- void RunStep()

- `void DisplayState() const`
- `bool ValidStartAddress(int addr) const` (even and <= 0xFE)
- `int ReadHexInput(const std::string& prompt, int minVal, int maxVal)`
- `bool AskExitOrLoadAnother()`
- `void Reset()`
- `void Shutdown()`

11. Enums

```
enum class LoadResult {
    OK,
    FILE_NOT_FOUND,
    BAD_TOKEN,
    ODD_ADDRESS,
    ADDRESS_OUT_OF_RANGE,
    PROGRAM_TOO_LARGE,
    OVERLAPS_SCREEN
};

enum class StepResult {
    OK,
    HALTED,
    INVALID_INSTRUCTION,
    PC_OUT_OF_RANGE
};
```

Relationships

- `Machine` owns `Screen`, `Memory`, `RegisterFile`, and `CPU` (composition).
- `Memory` holds a reference to `Screen`.
- `CPU` holds references to `Memory` and `RegisterFile`.
- `CPU` depends on `Decoder`, `ALU`, and `Instruction`.
- `ProgramLoader` depends on `Memory`, `HexUtils`, and `LoadResult`.
- `Decoder` depends on `Instruction` and `HexUtils`.
- `Machine` depends on `ProgramLoader`, `HexUtils`, and the enums.

Reminder: declare members in `Machine` in the order `screen, memory, regs, cpu,` since C++ initializes them in declaration order and each later one holds a reference to an earlier one.

9. User

Give me UML Design of the final classes distribution we reached

10. Claude

Final UML Class Diagram (Mermaid)

```
classDiagram
    direction LR

    class HexUtils {
        <<utility>>
        +HexToDec(string s) int$
        +DecToHex(int value, int width) string$
        +IsValidHex(string s, int expectedLen) bool$
        +CombineXY(int x, int y) int$
        +SplitXY(int xy, int& x, int& y) void$
        +Mask8(int v) int$
        +Mask16(int v) int$
    }
```

```
class ALU {
    <<utility>>
    +AddInt(int a, int b) int$
    +AddFloat(int a, int b) int$
    +BitwiseOR(int a, int b) int$
    +BitwiseAND(int a, int b) int$
    +BitwiseXOR(int a, int b) int$
    +RotateRight(int value, int times) int$
    +Equal(int a, int b) bool$
    -DecodeFloat(int bits) double$
    -EncodeFloat(double value) int$
}

class Instruction {
    <<struct>>
    +int opcode
    +int R
    +int X
    +int Y
    +int XY
    +bool valid
    +string errorMsg
}

class Decoder {
    <<utility>>
    +Decode(int IR) Instruction$
    +IsHalt(Instruction inst) bool$
}

class Screen {
    -string buffer
    +Write(int value) void
    +Display(ostream& out) void
    +Clear() void
}

class Memory {
    -int cells[256]
    -Screen& screen
    +Memory(Screen& screen)
    +Get(int addr) int
    +Set(int addr, int value) void
    +IsScreenAddress(int addr) bool
    +Display(ostream& out, int pc) void
    +Reset() void
}

class RegisterFile {
    -int regs[16]
    +Get(int r) int
    +Set(int r, int value) void
    +Display(ostream& out) void
    +Reset() void
}

class ProgramLoader {
    <<utility>>
    +LoadFile(string filename, int startAddr, Memory& mem) LoadResult$
    +ValidateFileFormat(string filename) bool$
    -ParseHexFile(string filename, LoadResult& result) vector~int~$
    -ParseHexToken(string token, int& word) bool$
}

class CPU {
    -int PC
    -int IR
```

```
    -bool halted
    -Memory& memory
    -RegisterFile& regs
    +CPU(Memory& mem, RegisterFile& regs)
    +GetPC() int
    +SetPC(int pc) void
    +GetIR() int
    +Step() StepResult
    +IsHalted() bool
    +Reset() void
    -Fetch() void
    -Execute(Instruction inst) void
}

class Machine {
    -Screen screen
    -Memory memory
    -RegisterFile regs
    -CPU cpu
    -bool programLoaded
    -int programStart
    -int MAX_STEPS$
    +Machine()
    +Run() void
    +ShowMenu() void
    +LoadProgramFromFile() void
    +OverwriteRegister() void
    +OverwriteMemory() void
    +SetPC() void
    +RunAll() void
    +RunStep() void
    +DisplayState() void
    +ValidStartAddress(int addr) bool
    +ReadHexInput(string prompt, int minVal, int maxVal) int
    +AskExitOrLoadAnother() bool
    +Reset() void
    +Shutdown() void
}

class LoadResult {
    <<enumeration>>
    OK
    FILE_NOT_FOUND
    BAD_TOKEN
    ODD_ADDRESS
    ADDRESS_OUT_OF_RANGE
    PROGRAM_TOO_LARGE
    OVERLAPS_SCREEN
}

class StepResult {
    <<enumeration>>
    OK
    HALTED
    INVALID_INSTRUCTION
    PC_OUT_OF_RANGE
}

%% Composition (Machine owns these)
Machine *-- Screen
Machine *-- Memory
Machine *-- RegisterFile
Machine *-- CPU

%% Associations (held references)
Memory --> Screen : writes to (addr FF)
CPU --> Memory : reference
CPU --> RegisterFile : reference
```

```
    %% Dependencies
    CPU ..> Decoder : uses
    CPU ..> ALU : uses
    CPU ..> Instruction : executes
    CPU ..> StepResult : returns
    Decoder ..> Instruction : creates
    Decoder ..> HexUtils : uses
    ALU ..> HexUtils : Mask8
    ProgramLoader ..> Memory : writes to
    ProgramLoader ..> HexUtils : uses
    ProgramLoader ..> LoadResult : returns
    Machine ..> ProgramLoader : uses
    Machine ..> HexUtils : uses
    Machine ..> LoadResult : checks
    Machine ..> StepResult : checks
```

Reading the diagram

Arrow	Meaning
`*--`	Composition: Machine owns and creates these objects
`-->`	Association: the object stores a reference to another
`..>`	Dependency: used inside methods, not stored
`$` suffix	Static method
`-` / `+`	Private / public

Implementation notes

- **Member order in `Machine`:** `screen`, `memory`, `regs`, `cpu`. Each later member takes a reference to an earlier one, and C++ initializes in declaration order.
- **Invariants:** cells and registers hold 0-255, PC holds 0-255, and IR holds 0-65535. Masking is enforced in `Memory::Set`, `RegisterFile::Set`, the `ALU` return values, and `Decoder::Decode`.
- **Screen write:** `Memory::Set(0xFF, v)` forwards to `Screen::Write(v)` instead of storing in `cells`.
- **Step flow:** `CPU::Step()` runs Fetch, then PC += 2, then Decode, then validate, then Execute. A jump overwrites the already-incremented PC.

You can paste the code block into the Mermaid Live Editor (mermaid.live) or any Markdown tool that supports Mermaid to render it. If you'd like, I can also draw a sequence diagram of the fetch-decode-execute cycle or write the C++ header files.

11. User

Give me mermaid uml design

12. Claude

Mermaid UML Class Diagram (final design)

I removed the `&` characters from the signatures because some Mermaid versions fail to render them. In the C++ code, `ostream`, `Memory`, `Screen` and similar parameters are still passed by reference.

```
classDiagram
```

```
direction LR

class HexUtils {
    <<utility>>
    +HexToDec(string s) int$
    +DecToHex(int value, int width) string$
    +IsValidHex(string s, int expectedLen) bool$
    +CombineXY(int x, int y) int$
    +SplitXY(int xy, int x, int y) void$
    +Mask8(int v) int$
    +Mask16(int v) int$
}

class ALU {
    <<utility>>
    +AddInt(int a, int b) int$
    +AddFloat(int a, int b) int$
    +BitwiseOR(int a, int b) int$
    +BitwiseAND(int a, int b) int$
    +BitwiseXOR(int a, int b) int$
    +RotateRight(int value, int times) int$
    +Equal(int a, int b) bool$
    -DecodeFloat(int bits) double$
    -EncodeFloat(double value) int$
}

class Instruction {
    <<struct>>
    +int opcode
    +int R
    +int X
    +int Y
    +int XY
    +bool valid
    +string errorMsg
}

class Decoder {
    <<utility>>
    +Decode(int IR) Instruction$
    +IsHalt(Instruction inst) bool$
}

class Screen {
    -string buffer
    +Write(int value) void
    +Display(ostream out) void
    +Clear() void
}

class Memory {
    -array~int,256~ cells
    -Screen screenRef
    +Memory(Screen screen)
    +Get(int addr) int
    +Set(int addr, int value) void
    +IsScreenAddress(int addr) bool
    +Display(ostream out, int pc) void
    +Reset() void
}

class RegisterFile {
    -array~int,16~ regs
    +Get(int r) int
    +Set(int r, int value) void
    +Display(ostream out) void
    +Reset() void
}
```

```
class ProgramLoader {
    <<utility>>
    +LoadFile(string filename, int startAddr, Memory mem) LoadResult$
    +ValidateFileFormat(string filename) bool$
    -ParseHexFile(string filename, LoadResult result) vector~int~$
    -ParseHexToken(string token, int word) bool$
}

class CPU {
    -int PC
    -int IR
    -bool halted
    -Memory memoryRef
    -RegisterFile regsRef
    +CPU(Memory mem, RegisterFile regs)
    +GetPC() int
    +SetPC(int pc) void
    +GetIR() int
    +Step() StepResult
    +IsHalted() bool
    +Reset() void
    -Fetch() void
    -Execute(Instruction inst) void
}

class Machine {
    -Screen screen
    -Memory memory
    -RegisterFile regs
    -CPU cpu
    -bool programLoaded
    -int programStart
    -int MAX_STEPS$
    +Machine()
    +Run() void
    +ShowMenu() void
    +LoadProgramFromFile() void
    +OverwriteRegister() void
    +OverwriteMemory() void
    +SetPC() void
    +RunAll() void
    +RunStep() void
    +DisplayState() void
    +ValidStartAddress(int addr) bool
    +ReadHexInput(string prompt, int minVal, int maxVal) int
    +AskExitOrLoadAnother() bool
    +Reset() void
    +Shutdown() void
}

class LoadResult {
    <<enumeration>>
    OK
    FILE_NOT_FOUND
    BAD_TOKEN
    ODD_ADDRESS
    ADDRESS_OUT_OF_RANGE
    PROGRAM_TOO_LARGE
    OVERLAPS_SCREEN
}

class StepResult {
    <<enumeration>>
    OK
    HALTED
    INVALID_INSTRUCTION
    PC_OUT_OF_RANGE
}
```

```
    Machine *-- Screen
    Machine *-- Memory
    Machine *-- RegisterFile
    Machine *-- CPU

    Memory --> Screen : writes to on addr FF
    CPU --> Memory : reference
    CPU --> RegisterFile : reference

    CPU ..> Decoder : uses
    CPU ..> ALU : uses
    CPU ..> Instruction : executes
    CPU ..> StepResult : returns
    Decoder ..> Instruction : creates
    Decoder ..> HexUtils : uses
    ALU ..> HexUtils : uses Mask8
    ProgramLoader ..> Memory : writes to
    ProgramLoader ..> HexUtils : uses
    ProgramLoader ..> LoadResult : returns
    Machine ..> ProgramLoader : uses
    Machine ..> HexUtils : uses
    Machine ..> LoadResult : checks
    Machine ..> StepResult : checks
```

Paste it into mermaid.live to render and export it as PNG or SVG.

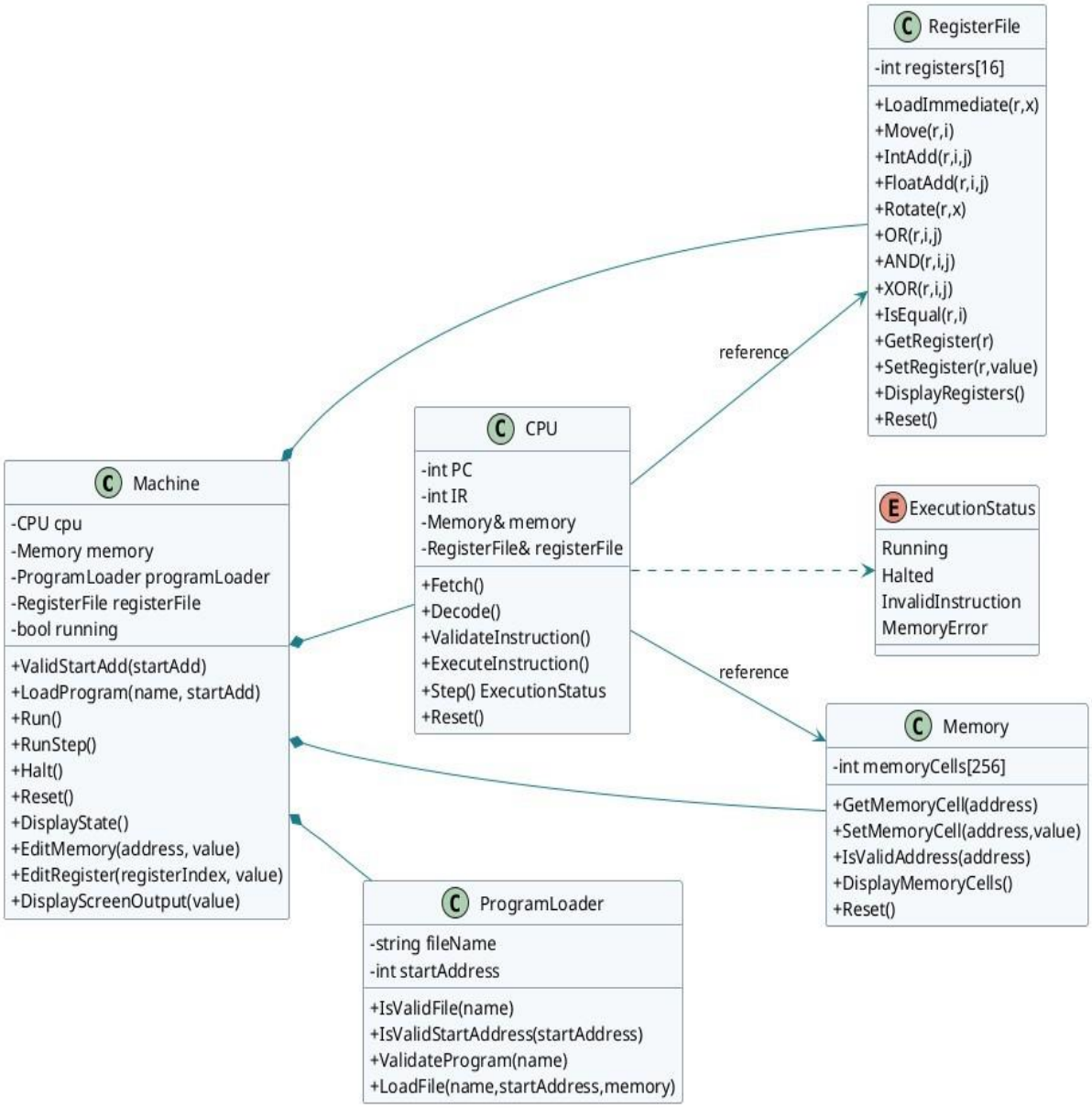


Figure 1: Rendered Mermaid UML class diagram of the earlier final design (before further merging)

I can also give you a Mermaid sequence diagram of the fetch-decode-execute cycle, or a flowchart of the menu flow (load, run all or step by step, exit or load another).

13. User

Give me the dialogue transcript of this chat as a whole including tge image in a .docx fike

14. Claude

The transcript is in Vole_Simulator_Chat_Transcript.docx. It has all 13 turns in order, with each speaker labelled and shaded, and it runs to 23 pages.

- **Code and tables:** The Mermaid diagrams, class listings and the arrow-legend table are kept in monospace blocks and a real table.
- **Image:** I rendered the final Mermaid class diagram as a PNG and placed it in turn 12, right where

the diagram was given. It is small at page width, so zoom in to read it.

- **First message:** I reflowed the opcode table into labelled paragraphs, and your original Mermaid diagram is reproduced as a code block.
- **Last turn:** This request is included as the final user turn, with no reply after it.

The chat contained no uploaded picture, so I took "the image" to mean the UML diagram. If you meant something else, tell me and I'll add it.

15. User

(Translated from Arabic) Explain to me every method / attribute in every class of the final design we reached: what exactly it is supposed to do and what its role is in my simulator, without writing any code.

16. Claude

Explanation of the Final Design (no code)

The classes are ordered from the simplest helper tools up to the most complex, i.e. `Machine`, which ties everything together.

1. HexUtils (static helper tools)

The class's role: translates between hex and decimal and protects values from leaving their range.

- `HexToDec(s)`: converts a hex string like "A3" into a number (163). Used when reading the file and the user's input.
- `DecToHex(value, width)`: the reverse; converts a number into a hex string with a fixed number of digits (for example 5 becomes "05"). Used for all printing.
- `IsValidHex(s, expectedLen)`: checks that the whole string is valid hex characters and that its length is the required one. Essential for checking the file and the inputs.
- `CombineXY(x, y)`: joins two nibbles into one 8-bit number. Example: X=3 and Y=C gives 3C.
- `SplitXY(xy, x, y)`: the reverse; splits the number into X and Y.
- `Mask8(v)`: returns only the last 8 bits (0-255). This is what solves the problem of using int instead of uint8_t.
- `Mask16(v)`: the same idea but for 16 bits, used with the IR.

2. ALU (static arithmetic and logic operations)

The class's role: executes the operations of opcodes 5 to B on plain values, without knowing anything about the registers or the memory.

- `AddInt(a, b)`: two's complement addition (opcode 5). The result is cut to 8 bits, so if it exceeds 255 it wraps around.
- `AddFloat(a, b)`: floating-point addition (opcode 6). Converts the two values to real numbers, adds them, and returns the result in the 8-bit format.
- `BitwiseOR / AND / XOR`: opcodes 7, 8 and 9. A bit-by-bit operation on the two values.
- `RotateRight(value, times)`: opcode A. Rotates the bits to the right, and the bit that falls off the end goes back in at the start. It takes `times` modulo 8, since the rotation repeats every 8.
- `Equal(a, b)`: compares two values; used in opcode B (the jump when R equals R0).
- `DecodeFloat(bits)` (private): unpacks the 8 bits into a real value: a sign bit, 3 exponent bits using excess-4, and 4 mantissa bits.
- `EncodeFloat(value)` (private): the reverse; converts a real value into 8 bits, normalizes it, and handles overflow and underflow.

3. Instruction (struct, data only)

The struct's role: a container holding the result of decoding one instruction.

- `opcode`: the first hex digit (1 to C), which determines the type of operation.
- `R`: the number of the main register (the second digit).
- `X` and `Y`: the last two digits separately; used as S and T in the operations that work on registers.
- `XY`: the last two digits together as one number; used as an address or an immediate value.
- `valid`: whether the instruction is well-formed or not.
- `errorMsg`: the reason for rejection if it is not valid, so a clear message can be shown to the user.

4. Decoder (static)

The class's role: converts the IR (16 bits) into a ready-to-use Instruction; this is the decode step.

- `Decode(IR)`: splits the 4 digits, checks that the opcode is from 1 to C, and checks the special constraints (opcode 4 needs a zero in its second digit, opcode A needs a zero in its third digit, opcode C needs 000). It returns an Instruction with `valid` and the error message if there is a problem.
- `IsHalt(inst)`: says whether the instruction is C000 (HALT), so the CPU can stop.

5. Screen

The class's role: represents the simulator's screen, which gets written to when the program stores to address FF.

- `buffer`: a string that accumulates everything written to the screen, in order, so it can be shown after the run.
- `Write(value)`: converts the value to an ASCII character and appends it to the buffer.
- `Display(out)`: prints the buffer's content to the stream you give it.
- `Clear()`: empties the buffer; called when a new program starts.

6. Memory

The class's role: the main memory (256 cells, each 8 bits).

- `cells`: the array itself, holding the program and the data. The values in it are always from 0 to 255.
- `screen`: a reference to the screen, so that writing to FF is turned into screen output.
- `Memory(screen)` (constructor): links the memory to the screen and zeroes the cells.
- `Get(addr)`: reads a cell's value. The CPU uses it in the fetch and in opcode 1.
- `Set(addr, value)`: writes into a cell after cutting the value to 8 bits. If the address is FF, it sends the value to the screen instead of storing it.
- `IsScreenAddress(addr)`: says whether the address is FF.
- `Display(out, pc)`: prints the memory as a matrix, and can highlight where the PC points.
- `Reset()`: zeroes all cells before a new program.

7. RegisterFile

The class's role: the 16 general-purpose registers (0 to F).

- `regs`: an array of 16 values, each 0-255.
- `Get(r)`: reads a register, after checking that the number is between 0 and F.
- `Set(r, value)`: writes into a register after cutting the value to 8 bits.
- `Display(out)`: prints the 16 registers with their hex values.
- `Reset()`: zeroes all of them.

8. ProgramLoader (static)

The class's role: reads the program file, checks it, and puts it in memory.

- `LoadFile(filename, startAddr, mem)`: the main function. It checks that the address is even and below FF, that the file exists, and that the program fits from this address without touching FF, and then writes each instruction into two consecutive cells (the high byte then the low byte). It returns a `LoadResult` that shows success or the reason for failure.
- `ValidateFileFormat(filename)`: checks the file without loading it, i.e. that every word is 4 valid hex digits.
- `ParseHexFile(filename, result)` (private): reads the whole file and returns a list of 16-bit words, and records the error result if it finds something wrong.
- `ParseHexToken(token, word)` (private): checks one word and converts it to a number, returning false if it is not 4 hex digits.

9. CPU

`The class's role: the heart of the simulator; it executes the fetch-decode-execute cycle.`

- `PC`: the address of the next instruction.
- `IR`: the instruction fetched at the moment (16 bits).
- `halted`: becomes true when HALT is executed.
- `memory` and `regs`: references to the memory and the registers it works on.
- `CPU(mem, regs)` (constructor): links the CPU to the components.
- `GetPC()` and `SetPC(pc)`: read and modify the PC. SetPC is used when the user specifies the program's start.
- `GetIR()`: returns the IR so it can be displayed.
- `Step()`: executes one whole step: fetch the instruction, add 2 to the PC, decode, check that it is valid, execute, and return a `StepResult`. It also checks that the PC has not reached FF.
- `Fetch()` (private): reads two bytes from PC and PC+1 and assembles them into IR.
- `Execute(inst)` (private): executes the instruction according to its opcode. It moves values between memory and registers, calls the ALU for the arithmetic, and changes the PC if a jump is taken.
- `IsHalted()`: says whether the program has stopped.
- `Reset()`: returns the PC and IR to zero and clears the halted state.

10. Machine

`The class's role: the conductor; it owns the components and runs the menu and the interaction with the user.`

- `screen`, `memory`, `regs` and `cpu`: the four components. This order matters because each one takes a reference to the ones before it.
- `programLoaded`: prevents running before a program has been loaded.
- `programStart`: the program's start address, so the PC can be returned to it.
- `MAX_STEPS`: a maximum number of steps in Run All, so that an infinite loop does not hang the program.
- `Machine()`: prepares everything.
- `Run()`: the main menu loop.
- `ShowMenu()`: prints the choices.
- `LoadProgramFromFile()`: asks for the file name and the address, checks the address, calls the loader, sets the PC, and shows any error.
- `OverwriteRegister()` and `OverwriteMemory()`: ask for the number and the new value, check the range, and write it.
- `SetPC()`: lets the user change the PC.
- `RunAll()`: repeats `Step()` until HALT, an error, or `MAX_STEPS`, and then shows the whole state.
- `RunStep()`: executes one step, shows the state, and waits for Enter.
- `DisplayState()`: shows the memory, the PC, the IR, the registers and the screen.

- `ValidStartAddress(addr)`: checks that the address is even and less than or equal to FE.
- `ReadHexInput(prompt, min, max)`: asks the user and repeats the question until valid hex in range is entered.
- `AskExitOrLoadAnother()`: the last question: exit or another program.
- `Reset()`: clears the memory, the registers, the CPU and the screen before a new program.
- `Shutdown()`: ends the program cleanly.

11. The Enums

- **LoadResult**: the result of loading the file (success, file not found, bad word, odd address, address out of range, program too big, overlap with the screen). It makes the message to the user precise.
- **StepResult**: the result of one step (normal, HALT, invalid instruction, PC out of range). It lets `RunAll` and `RunStep` behave correctly.

If you like, I can next explain the execution sequence step by step on a real example, or how each opcode is executed inside `Execute`.

17. User

(Translated from Arabic) Now tell me what things we could do without or merge together, and give me a listing after these modifications.

18. Claude

Suggested Simplification

What we drop or merge

1. Remove the Decoder class and merge it into Instruction

The Decoder had only two functions. Make `Decode` a static function inside `Instruction`, and drop `IsHalt`, since the CPU can compare the opcode with C directly.

2. Remove the Screen class and merge it into Memory

The screen is just a string. Make it an attribute inside Memory, and writing to FF appends to it. This also removes the member-order problem and the need for Memory to take a reference.

The cost: Memory now has some I/O in it. But that is logical, because the specification itself says the screen is memory-mapped.

3. Reduce HexUtils

- Remove `CombineXY` and `SplitXY`, because the Decoder splits the digits with shifts anyway.
- Remove `Mask16`, because the IR is assembled from two masked bytes, so it can never exceed 16 bits.

4. Reduce ALU

Remove `BitwiseOR`, `AND`, `XOR` and `Equal`, because each is one line and can be written directly inside `Execute`. What stays in the ALU is what has real logic: addition, float, and rotation.

5. Simplify CPU

Remove `IsHalted`, because `Step()` returns `HALTED` and that is enough.

6. Simplify Memory

Remove `IsScreenAddress`, because it is only a comparison with FF.

7. Simplify ProgramLoader

- Remove `ValidateFileFormat`, because `LoadFile` already checks the format while loading.
- Merge `ParseHexToken` into `ParseHexFile`.

8. Simplify Machine

- Merge `RunAll` and `RunStep` into one function `RunProgram(stepMode)`, because the loop is the same.
- Remove `programStart` (the PC is set at load time), `SetPC` (not required by the specification), and `Shutdown` (exiting happens when `Run` returns).
- Merge `AskExitOrLoadAnother` into `Run`.

9. Reduce LoadResult

Merge `ODD_ADDRESS` and `ADDRESS_OUT_OF_RANGE` into `INVALID_ADDRESS`, and merge `OVERLAPS_SCREEN` into `PROGRAM_TOO_LARGE`.

The Listing After the Modification (int types)

Invariants: cells, registers and the PC are from 0 to 255, and the IR is from 0 to 65535.

1. HexUtils (static)

Attributes: none

- `static int HexToDec(const std::string& s)`
- `static std::string DecToHex(int value, int width)`
- `static bool IsValidHex(const std::string& s, int expectedLen)`
- `static int Mask8(int v)`

2. ALU (static)

Attributes: none

- `static int AddInt(int a, int b)`
- `static int AddFloat(int a, int b)`
- `static int RotateRight(int value, int times)`
- `private static double DecodeFloat(int bits)`
- `private static int EncodeFloat(double value)`

3. Instruction (struct)

Attributes:

- `int opcode`
- `int R`
- `int X`
- `int Y`
- `int XY`
- `bool valid`
- `std::string errorMsg`

Methods:

- `static Instruction Decode(int IR)` (splits the IR and checks validity)

4. Memory

Attributes:

- `std::array<int, 256> cells`
- `std::string screenBuffer`

Methods:

- `Memory()`
- `int Get(int addr) const`
- `void Set(int addr, int value)` (cuts to 8 bits, and address FF goes to the screen)
- `void Display(std::ostream& out, int pc = -1) const`
- `void DisplayScreen(std::ostream& out) const`
- `void Reset()` (zeroes the cells and empties the screen)

5. RegisterFile

Attributes:

- `std::array<int, 16> regs`

Methods:

- `int Get(int r) const`
- `void Set(int r, int value)`
- `void Display(std::ostream& out) const`
- `void Reset()`

6. ProgramLoader (static)

Attributes: none

- `static LoadResult LoadFile(const std::string& filename, int startAddr, Memory& mem)`
- `private static std::vector<int> ParseHexFile(const std::string& filename, LoadResult& result)`

7. CPU

Attributes:

- `int PC`
- `int IR`
- `bool halted`
- `Memory& memory`
- `RegisterFile& regs`

Methods:

- `CPU(Memory& mem, RegisterFile& regs)`
- `int GetPC() const`
- `void SetPC(int pc)` (used by `Machine` at load time)
- `int GetIR() const`
- `StepResult Step()`
- `void Reset()`
- `private void Fetch()`
- `private void Execute(const Instruction& inst)`

8. Machine

Attributes:

- `Memory memory`
- `RegisterFile regs`
- `CPU cpu`
- `bool programLoaded`
- `static const int MAX_STEPS`

Methods:

- `Machine()`

- `void Run()` (the main menu, and at the end it asks exit or another program)
- `void ShowMenu() const`
- `void LoadProgramFromFile()`
- `void OverwriteRegister()`
- `void OverwriteMemory()`
- `void RunProgram(bool stepMode)`
- `void DisplayState() const`
- `bool ValidStartAddress(int addr) const`
- `int ReadHexInput(const std::string& prompt, int minVal, int maxVal)`
- `void Reset()`

Member order: memory, then regs, then cpu.

9. Enums

- **LoadResult:** `OK`, `FILE_NOT_FOUND`, `BAD_TOKEN`, `INVALID_ADDRESS`, `PROGRAM_TOO_LARGE`
- **StepResult:** `OK`, `HALTED`, `INVALID_INSTRUCTION`, `PC_OUT_OF_RANGE`

Summary: from 10 classes down to 8 (we removed `Screen` and `Decoder`), and we removed about 20 methods, without losing any requirement of the specification.

Note: if the instructor wants a clear separation between the screen and the memory, bring `Screen` back as a small class and leave the rest as it is.

Would you like me to update the Mermaid diagram to the new shape?

19. User

(Translated from Arabic) Write everything: how it is supposed to work and what it does exactly, but again without code.

20. Claude

Detailed Explanation of the Design After Simplification (8 classes)

For each class I write its role, and what every attribute and method does exactly. At the end there is a full run-through scenario that ties everything together. Any design decision that is not in the specification is stated explicitly, so you can change it if you don't like it.

1. HexUtils

Role: small tools called from everywhere, with no state.

- **HexToDec(s):** takes a hex string (upper or lower case) and returns its value. It assumes the string was checked before, and if it finds a wrong character it returns -1.
- **DecToHex(value, width):** returns a hex string in upper case, padded with zeros on the left up to the required number of digits. For example 10 with width 2 becomes "0A". All the printing in the program goes through it.
- **IsValidHex(s, expectedLen):** returns true only if the string is not empty, its length is exactly the required one, and all its characters are 0-9 or A-F or a-f. Any prefix such as "0x" is treated as an error. If `expectedLen` is zero, the length is not checked.
- **Mask8(v):** returns the last 8 bits of the number, so the value is always from 0 to 255. It is what compensates for using int instead of uint8_t.

2. ALU

Role: calculations only. It takes values and returns a value, and it knows nothing about the registers or the memory.

- **AddInt(a, b):** adds the two values and cuts the result to 8 bits. Binary addition in two's complement comes out right with no special handling, because the difference between signed and unsigned appears only in interpretation, not in the addition. So if the result exceeds 255 it wraps around (FF + 01 = 00).
- **RotateRight(value, times):** rotates the 8 bits to the right. Each rotation takes the bit at the far right and puts it at the far left. The number of rotations is taken modulo 8 (because 8 rotations return the same value). If the remainder is zero, it returns the value as it is.
- **AddFloat(a, b):** unpacks the two values from the 8-bit format into real numbers, adds them, and converts the result into 8 bits again.
- **DecodeFloat(bits)** (private): the 8 bits are divided into: a sign bit, 3 bits of exponent, and 4 bits of mantissa. The exponent uses excess-4, so the real exponent = the stored value - 4. The mantissa is interpreted as 0.xxxx (the first bit has a weight of one half). The value = sign x mantissa x 2^(exponent). If all the bits are zeros, the result is zero.
- **EncodeFloat(value)** (private): zero returns 00. Otherwise it takes the sign, computes the exponent that makes the mantissa between one half and one (normalized), and takes the first 4 bits of the mantissa by truncation, with no rounding. If the exponent after adding is larger than possible (overflow), it puts the largest possible value with the same sign. If it is smaller than possible (underflow), it returns zero.

A decision you must confirm: truncate or round? The original Vole book uses truncate, so stay with it unless the instructor said otherwise.

3. Instruction

Role: represents one instruction after it is decoded, and it also contains the validity check.

An instruction is 4 hex digits: the first is the opcode, and the other three are divided like this: R is the second digit, X the third, Y the fourth.

- **opcode:** the first digit (from 1 to C).
- **R:** the second digit, usually the target register.
- **X, Y:** the third and fourth digits, used as registers in the arithmetic operations, or as the number of rotations.
- **XY:** the last two digits together as a number from 0 to 255, used as an address or an immediate value.
- **valid:** true if the instruction is well-formed.
- **errorMsg:** the reason for rejection as clear text (for example "opcode D is unknown").
- **Decode(IR)** (static): splits the four digits and then checks:
- The opcode must be from 1 to C, otherwise it is invalid.
- Opcode 4 (the form 0RS): the second digit must be zero. The source is the third digit, and the destination is the fourth.
- Opcode A (the form R0X): the third digit must be zero. The number of rotations is the fourth digit, and the register is the second digit.
- Opcode C: the other three digits must be zeros (C000 only).
- For the other opcodes, any value in the digits is accepted.

It returns a ready Instruction, which is the one the CPU uses. The function changes nothing outside, and it gives the same result for the same input.

4. Memory

Role: the memory (256 cells) and the screen.

- **cells:** 256 cells, each between 0 and 255. They start at zeros.
- **screenBuffer:** a string that accumulates everything written to the screen in order.
- **Memory():** prepares the cells with zeros and empties the buffer.
- **Get(addr):** returns the cell's content. If the address is outside 0-255 it returns zero. If the address is FF it also returns zero, because the screen is write-only and there is no real cell.
- **Set(addr, value):** cuts the value to 8 bits. If the address is FF it appends the character to `screenBuffer` without changing any cell. Otherwise it writes into the cell. If the address is outside the range the write is ignored.
- **How a value is converted to a character:** numbers from 0x20 to 0x7E are appended as a normal character. The value 0x0A (newline) appends a new line. Any other value is appended as text like `<4F>` so that it stays visible.
- **Display(out, pc):** prints a 16x16 table. The header row on top has 0 to F (the last digit of the address), and the header column on the left has 0 to F (the first digit). So cell 3C is in row 3 and column C. Each value takes two hex digits. If `pc` is given, the two cells the PC points at are marked with a clear sign (brackets, for example) so you can see the next instruction.
- **DisplayScreen(out):** prints a "Screen" title and under it the content of the buffer. If it is empty it says so.
- **Reset():** zeroes all cells and empties the buffer.

A decision: if the user does an Overwrite on FF from the menu, it is treated as a write to the screen, because that is the only defined behaviour for this address. You could instead reject it with a message; both are acceptable.

5. RegisterFile

Role: the 16 registers (0 to F).

- **regs:** 16 values, each between 0 and 255, starting at zeros.
- **Get(r):** returns the register's value. If the number is outside 0-15 it returns zero.
- **Set(r, value):** cuts the value to 8 bits and writes it. If the number is outside the range it is ignored. All writing must go through here.
- **Display(out):** prints the 16 registers in the form "R0=00 R1=3A ..." arranged in two or four rows.
- **Reset():** zeroes all of them.

6. ProgramLoader

Role: reads the program file and puts it into memory after making sure it is valid.

File format: hex words separated by spaces or lines. Each word is exactly 4 digits, upper or lower case. Empty lines are ignored. Any word that is not 4 valid hex digits counts as an error.

- **ParseHexFile(filename, result)** (private): opens the file (if it is not found: `FILE_NOT_FOUND`). It reads word by word and checks each one with `IsValidHex` with length 4. If a word is wrong: `BAD_TOKEN`. If the file has no words at all, it is also treated as `BAD_TOKEN`. It returns the list of numbers (each 16 bits).
- **LoadFile(filename, startAddr, mem):** works in this order:

1. The address must be even and between 00 and FE. Otherwise `INVALID_ADDRESS`.
2. It calls `ParseHexFile`, and any error is returned as it is.
3. It computes the size: each instruction takes two cells. If the address + (2 x number of words) is larger than FE, the program would reach cell FF, so it refuses with `PROGRAM_TOO_LARGE`.
4. If all the checks pass, it writes each word into two cells: the high byte (the first two digits) at the address, and the low byte at the next one.

The most important point: nothing is written into memory until all the checks succeed, so half a program is never left loaded if an error occurs in the

middle. It returns OK on success.

7. CPU

Role: executes the fetch-decode-execute cycle.

- **PC:** the address of the next instruction (from 0 to 255).
- **IR:** the last instruction fetched (16 bits), shown on the screen.
- **halted:** becomes true after HALT is executed.
- **memory, regs:** references to the memory and the registers.
- **CPU(mem, regs):** sets the references and starts the PC and IR at zero and halted at false.
- **GetPC():** returns the PC.
- **SetPC(pc):** puts a new value into the PC after cutting it to 8 bits. It is called by `Machine` after loading the program.
- **GetIR():** returns the IR.
- **Fetch()** (private): reads two bytes: one from PC and one from PC+1, and assembles them into IR (the first is the high one).
- **Execute(inst)** (private): executes according to the opcode:
- **1:** put into register R the content of cell XY.
- **2:** put into register R the value XY itself.
- **3:** write the value of register R into cell XY. If XY = FF it goes to the screen (this happens automatically inside `Memory::Set`).
- **4:** copy register X into register Y (the third digit is the source and the fourth is the destination).
- **5:** register R = the sum of register X and register Y using `AddInt`.
- **6:** the same idea using `AddFloat`.
- **7, 8, 9:** register R = (OR or AND or XOR) between register X and register Y. These operations are written directly here.
- **A:** rotate register R to the right a number of times = the fourth digit (Y).
- **B:** if the value of register R equals the value of register 0, put XY into the PC. If R itself is 0, the condition is always true. Otherwise nothing happens.
- **C:** set `halted` to true.
- **Step():** one complete step in this order:

1. If `halted` is already true: return `HALTED` without any execution.
2. If PC is larger than FE (so two bytes cannot be read): return `PC_OUT_OF_RANGE`.
3. Call `Fetch`.
4. Add 2 to the PC immediately. This is important so that the jump (opcode B) overwrites the incremented value, not the other way round.
5. Decode the IR with `Instruction::Decode`. If it is invalid: return the PC to the location of the bad instruction (subtract 2) so the user can see it, and return `INVALID_INSTRUCTION`.
6. Call `Execute`.
7. If the instruction was C return `HALTED`, otherwise `OK`.

- **Reset():** PC and IR to zero and halted to false.

A decision: a jump to an odd address is technically accepted. If you want to be strict you could treat it as an error, but the specification did not say so.

8. Machine

Role: the conductor. It owns the components and deals with the user.

- **memory, regs, cpu:** the components. The order matters (the CPU must take references to the two before it).
- **programLoaded:** becomes true after a successful load, and prevents running without a program.

- **MAX_STEPS:** a maximum number of steps in the full run, for example 10000, so an infinite loop does not hang the program.
- **Machine():** prepares everything and starts with `programLoaded = false`.
- **ShowMenu():** prints the choices: 1 load a program, 2 change a register, 3 change a memory cell, 4 run all, 5 run step by step, 6 show the state, 7 exit.
- **ReadHexInput(prompt, min, max):** prints the prompt and reads a line. It removes the spaces, checks that it is hex of one or two digits, converts it, and checks that it is in the range. If it is wrong it prints a message and asks again, and it only returns with a valid value. If the input ends (EOF) it must leave the loop so the program does not fall into an endless cycle.
- **ValidStartAddress(addr):** true if the address is even and between 0 and FE.
- **LoadProgramFromFile():** asks for the file name, then for the start address with `ReadHexInput`. If the address is not valid it prints a message and goes back to the menu. It calls `LoadFile`. If the result is not OK it prints a message that fits the reason and returns without changing anything. If OK: it does `cpu.Reset()`, sets the PC to the start address, empties the screen, sets `programLoaded = true`, and shows the state (step 5 of the specification). It does not clear the memory here, so the user can load more than one program in different places.
- **OverwriteRegister():** asks for the register number (0-F) and then the value (00-FF), writes with `Set`, and shows the state.
- **OverwriteMemory():** asks for the address (00-FF) and the value (00-FF), writes with `Set`, and shows the state.
- **DisplayState():** prints in order: the memory table, the PC (two hex digits), the IR (4 hex digits), the sixteen registers, and then the screen.
- **RunProgram(stepMode):** if `programLoaded` is false it prints a warning and returns. If the CPU has already stopped, it tells the user to load another program. Then:
- **Full run:** repeats `Step()` until a result other than `OK` comes out, or `MAX_STEPS` is exceeded, in which case it prints a warning that it is probably an infinite loop. When it finishes it shows the state once (step 8).
- **Step by step:** executes one `Step()`, shows the state, and if the result is `OK` it says "press Enter" and waits, and then the next step (step 9). If the result is not `OK` it stops.

In both cases it prints a message that fits the reason for stopping: a normal HALT, an invalid instruction (with the errorMsg), or the PC going out of range.

- **Reset():** clears the memory (and the screen), the registers and the CPU, and sets `programLoaded` to false.
- **Run():** the main menu loop: it shows the menu, reads the choice, and executes it. After any run ends (full or step by step) it asks: "exit or load another program?". If another program, it calls `Reset()` first and continues. If exit, it leaves the loop and so the program ends.

9. The Enums

- **LoadResult:** `OK` (succeeded), `FILE_NOT_FOUND`, `BAD_TOKEN` (a word that is not 4 hex digits, or an empty file), `INVALID_ADDRESS` (odd or larger than FE), `PROGRAM_TOO_LARGE` (will not fit before FF). Each value has a different message for the user.
- **StepResult:** `OK` (continue), `HALTED` (finished), `INVALID_INSTRUCTION`, `PC_OUT_OF_RANGE`. It keeps the running logic in one place.

A full scenario that ties everything together

1. The user chooses 1 and writes the file name and the address 00.
2. `LoadFile` checks and writes the program into memory, and `Machine` sets the PC to 00 and shows the state.
3. The user chooses 5 (step by step).
4. On each step: `Step` fetches the instruction from the PC, adds 2 to the PC, decodes it, and executes it. `Machine` shows the state and waits for Enter.
5. When it reaches C000, `Step` returns `HALTED` and the finish message is printed.

6. `Machine` asks: exit or another program? If another program, it does `Reset()` and repeats from step 1.

Would you like me to apply this design to an example program (such as one that prints a character on the screen) and explain the state after each step with the numbers?

21. User

(Translated from Arabic) I don't understand mask8... is it even important?

22. Claude

Yes, it is important, but the idea is very simple.

The problem

A register in the Vole is 8 bits in size, meaning the largest value in it is FF (255). But `int` in C++ is 32 bits, so it does not know that you want it to be only 8.

For example: if you add FF + 01 in an `int`, the result will be 100 hex (that is 256), and that is a number bigger than what the register can hold. In the real Vole the result must be `00`, because the extra bit is lost.

What Mask8 does

It keeps only the last 8 bits and throws away anything above them. In practice it gives the remainder of dividing by 256.

- 256 becomes 0
- 257 becomes 1
- 300 becomes 44
- 255 stays 255

And if a negative number comes out (like -1), it turns it into 255, which is what is required in two's complement.

Where exactly is it needed?

- **In the `Set` of Memory and RegisterFile.** This is the most important place, and if you do it here only, it will protect the values that get stored.
- **In `RotateRight`**, because rotating produces extra bits above the 8, so they must be removed for the result to be right.
- **In `AddInt`**, and this duplicates what is in `Set`, but it is useful if you want the ALU to give a correct result on its own.

Can we do without it?

If you use `uint8_t` instead of `int`, C++ cuts by itself and you will not need it. And since we decided to continue with `int`, you can write `& 0xFF` directly in those three places. The `Mask8` function is just a convenient name for that operation so you don't repeat it, so if you don't want it, remove it from HexUtils and the design stays sound.

If you like, I can give you a table of numbers before and after the cut for addition and rotation so the idea sticks.

23. User

(Translated from Arabic) If I remove it, what exactly should I watch out for? And why do I add the addresses of the registers in the first place?

24. Claude

If you remove Mask8, watch out for 3 things

The function itself is not important, what matters is that the cut happens. So instead of it you write `& 0xFF` in these places:

1. **In Set of the RegisterFile and of the Memory.** This is the most important place. And after that it is forbidden to write into the array directly from any other place; all writing goes through `Set` only.
2. **In AddInt.** So that FF + 01 comes out 00, not 100.
3. **In RotateRight.** Rotation produces bits above the 8, so you must cut the result after combining the two parts.

How you know you forgot one: you will find in the memory or registers table a value of 3 digits like `100`, and the table gets broken, or a negative number appears in a strange way.

You do not need the cut in `Get`, nor in the IR (because it is assembled from two already-cut bytes), nor in the XY (because it is split from the IR after it was cut).

Why do we add the addresses of the registers?

We are not adding the addresses, we are adding their contents. Perhaps the confusion came from the shape of the instruction.

Take this example: `5726`

- The digits 7 and 2 and 6 are **register numbers** (just names, like the number of a drawer).
- The CPU goes to register 2 and register 6 and reads **the value inside them**.
- It adds the two values and puts the result in register 7.

So if register 2 has 03 and register 6 has 05, the result 08 is put in register 7. The numbers 7 and 2 and 6 themselves are not added and do not enter the calculation.

This is the reason `AddInt(a, b)` takes **two values**, not register numbers. The CPU is the one that first gets the values from the RegisterFile and then gives them to the ALU.

The same idea applies to opcodes 6 and 7 and 8 and 9. As for opcode 4, it moves the content of one register to another register without any calculation.

If you like, I can make a full example on the instruction `5726` with numbers and show you the registers before and after.

25. User

(Translated from Arabic) Didn't you say that its importance is when we add FF and 01?

26. Claude

Yes, I said that, and there is no contradiction. The confusion is in the word "addresses".

FF and 01 are not register addresses, they are values inside the registers.

An example with `5726`:

- register 2 holds the value **FF**.

- register 6 holds the value **01**.
- The CPU reads these two values and adds them: FF + 01 = **100** (256).
- The register takes 8 bits only, so the cut must happen and the result becomes **00**. And that is the job of `& 0xFF` or `Mask8`.
- The result 00 is written into register 7.

The numbers 7 and 2 and 6 in the instruction just say "read from which register and write into which register". They are names, and they do not enter the addition.

So the two sentences complete each other:

- What we add is the **content** of the registers (like FF and 01).
- That addition can produce a result larger than 255, so we need the cut.

If you like, I can make a small table with the registers before and after executing `5726` with different values so the picture sticks.

27. User

(Translated from Arabic) I still feel the classes and their contents are too many, merge them more.

28. Claude

More Merging: from 8 classes to 4

What was merged

- **ALU + HexUtils become `Utils`**: the only things that have real logic are the hex and the float. The ordinary addition (`& 0xFF`) and the rotation became lines inside `Execute`, so they were removed from the class. And `Mask8` was removed completely.
- **Memory + RegisterFile + CPU + ProgramLoader become `CPU`**: they all work on the same state, so we keep it in one place. The two arrays become ordinary attributes, and the screen is a string inside them.
- **LoadResult was removed**: `LoadProgram` returns an error message as text, and empty means success (like `Instruction` has `errorMsg`).
- **Machine was reduced to the user interface only**: the menu and the questions. Everything else goes to the CPU.

The Final Listing

1. Utils (static)

Attributes: none

- `HexToDec(s)`: a hex string to a number.
- `DecToHex(value, width)`: a number to a hex string with a fixed number of digits.
- `IsValidHex(s, expectedLen)`: checks that the string is valid hex and its length is right.
- `AddFloat(a, b)`: floating-point addition of two 8-bit values.
- `DecodeFloat(bits)` (private): from 8 bits to a real value.
- `EncodeFloat(value)` (private): the reverse, truncating the mantissa.

2. Instruction (struct)

Attributes: `opcode, R, X, Y, XY, valid, errorMsg`

- `Decode(IR)` (static): splits the four digits and checks validity (opcode from 1 to C, and the special

constraints of 4 and A and C).

3. CPU

Attributes:

- `memory[256]`: the memory.
- `regs[16]`: the registers.
- `screenBuffer`: the screen text.
- `PC`, `IR`, `halted`

Methods:

- `LoadProgram(filename, startAddr)`: checks the address (even and below FF), reads the file and checks each word, makes sure the program fits before FF, and only then writes into memory and sets the PC. It returns the error text, or an empty text if it succeeded.
- `Step()`: a complete step (fetch, add 2 to PC, decode, execute) and returns `StepResult`.
- `WriteMemory(addr, value)`: cuts the value, and address FF goes to the screen. It is called from `Execute` and from the menu.
- `WriteRegister(r, value)`: cuts the value and writes.
- `Display(out)`: prints the 16x16 memory, the PC, the IR, the registers and the screen all at once.
- `Reset()`: zeroes everything.
- `Fetch()` and `Execute(inst)` (private): as explained before.

4. Machine

Attributes: `cpu, programLoaded, MAX_STEPS`

- `Run()`: the main menu, and at the end it asks exit or another program (and if another program it calls `cpu.Reset()`).
- `ShowMenu()`: prints the choices.
- `LoadProgramFromFile()`: asks for the file and the address, calls `cpu.LoadProgram`, and shows the error if there is one.
- `OverwriteRegister()` and `OverwriteMemory()`: ask and call `WriteRegister` or `WriteMemory`.
- `RunProgram(stepMode)`: full run or step by step, and after each time it calls `cpu.Display`.
- `ReadHexInput(prompt, min, max)`: asks until a valid value is entered.

Only one Enum

- **StepResult:** `OK`, `HALTED`, `INVALID_INSTRUCTION`, `PC_OUT_OF_RANGE`

The result

- From **8 classes to 4**, and from two enums to one.
- All ten requirements in the specification are still covered.

The concession: the CPU became a big class with a lot in it. If the instructor wants Memory and RegisterFile as separate classes, bring them back (both are very small) and leave the rest as it is.

Would you like me to update the Mermaid diagram to this shape?

29. User

(Translated from Arabic) What should I make decode float and encode float do?

30. Claude

DecodeFloat and EncodeFloat

First: the shape of the 8 bits

Every register in the Vole can be interpreted as a decimal number in this format:

- **One bit (far left):** the sign. 0 is positive and 1 is negative.
- **The 3 bits after it:** the exponent, using excess-4. So the real exponent = the stored value minus 4. The stored value is from 0 to 7, so the real exponent is from -4 to +3.
- **The last 4 bits:** the mantissa. It is interpreted as 0.xxxx in binary, so its value = the number divided by 16.

The final value = sign x mantissa x 2^(real exponent)

The sign here is sign-magnitude, not two's complement. So the negative is the same as the positive, only with the first bit set to 1.

DecodeFloat: from 8 bits to a real number

It takes the number (0-255) and returns its decimal value, with these steps:

1. **Separate the 3 parts:** the sign is the first bit, the exponent is the 3 bits after it, and the mantissa is the last 4 bits.
2. **The zero case:** if the mantissa is all zeros, return zero right away. This covers 00, and also 80 (negative zero), which we treat as an ordinary zero.
3. **Compute the mantissa as a number:** the value of the 4 bits divided by 16.
4. **Compute the real exponent:** the stored exponent minus 4.
5. **Multiply:** the mantissa times 2 raised to the exponent. And if the sign is negative, multiply by -1.

Examples:

- 68 = 0 110 1000: the exponent 6-4=2, the mantissa 8/16=0.5, the value 0.5 x 4 = **2.0**
- 58 = 0 101 1000: the exponent 1, the value 0.5 x 2 = **1.0**
- 5C = 0 101 1100: the mantissa 12/16=0.75, the value 0.75 x 2 = **1.5**
- 38 = 0 011 1000: the exponent -1, the value 0.5 x 0.5 = **0.25**
- D8 = 1 101 1000: the same as 58 but negative, so **-1.0**

EncodeFloat: from a real number to 8 bits

It takes a decimal number and returns the 8 bits. This one is harder than the first because it must do **normalization**:

1. **Zero:** if the number is zero, return 00.
2. **The sign:** if the number is negative, remember that the sign is 1 and work with the absolute value from then on.
3. **Find the exponent:** you want to convert the number to the form (a number between 0.5 and less than 1) x 2^exponent. The way:
   - If the number is 1 or larger: divide it by 2 and increase the exponent by 1, and repeat until it is less than 1.
   - If the number is less than 0.5: multiply it by 2 and decrease the exponent by 1, and repeat until it is 0.5 or more.
4. **Extract the mantissa:** the number that is between 0.5 and 1, multiply it by 16 and take the integer part only (cut without rounding). The result is

always from 8 to 15, meaning its first bit is 1 (and that is the meaning of normalized).

5. **Convert the exponent to the stored one:** the real exponent + 4.
6. **Check the limits:**
   - If the stored exponent is larger than 7 (overflow): return the largest possible value with the same sign, which is exponent=7 and mantissa=1111.
   - If the stored exponent is less than 0 (underflow): return 00.
7. **Assemble the 8 bits:** the sign, then the exponent (3 bits), then the mantissa (4 bits).

Example: encoding 1.75

- 1.75 / 2 = 0.875 (exponent 1).
- 0.875 x 16 = 14, which is 1110.
- The stored exponent 1+4 = 5, which is 101.
- The result: 0 101 1110 = **5E**.

And then how AddFloat uses them

1. DecodeFloat of the first value.
2. DecodeFloat of the second value.
3. Add the two ordinary numbers.
4. EncodeFloat of the result and return it.

A full example: `5C` + `38` means 1.5 + 0.25 = 1.75, and the result is `5E` as in the example above.

Things you must watch out for

- **Truncation loses precision:** for example 1.8125 will not be stored exactly because the mantissa has only 4 bits. This is normal in the Vole, so when you compute the expected result in a test, use truncation, not rounding.
- **A decision you must confirm:** does the instructor want truncate or round? The original Vole book uses truncate, so ask if you are not sure.
- **A `double` in C++ holds any value from the 8 bits exactly**, so there is no risk of rounding errors in the decoding. The only problem in the addition itself is if the result is not representable.
- **A zero result:** if you add 1.0 and -1.0 the result is zero, and the encoding returns 00.

Would you like a table of 10 examples (values, the sum, and the expected hex) to use as test cases?

31. User

(Translated from Arabic) Then why? I mean, if I don't include them, doesn't C++ add the float / double anyway?

32. Claude

Because C++ adds a float in its own format (32-bit IEEE), but the register you have holds 8 bits in the specific Vole format. The two are not the same shape, so C++ does not know that these 8 bits mean a decimal number.

If you add the values directly

Take this example: a register holds `5C` (which is 1.5 in the Vole format), and another register holds `38` (which is 0.25).

- **If you add them as ints:** 5C + 38 = 94. And in the Vole format that means a very small negative number, so it is **completely wrong**.
- **If you put them in a float or a double:** C++ will see them as 92 and 56 (the numeric value of the bits), and add them to get 148. And that is also **wrong**, because 92 and 56 have no relation to 1.5 and 0.25.

The right answer is 1.75 and it is stored as `5E`.

The two functions are the "translator"

Like writing a number in Arabic and converting it to English so someone else understands it:

1. **DecodeFloat:** translates the 8 bits into a real number (92 becomes 1.5, and 56 becomes 0.25). **This is what makes C++ understand that this number is a decimal.**
2. **The addition:** only here does C++ work on its own (1.5 + 0.25 = 1.75). **This is the only part C++ gives you.**
3. **EncodeFloat:** translates the number back into the 8-bit format so it can be stored in the register (1.75 becomes 5E).

So what do you really get for free?

The addition itself you take ready from C++ (one line). What you must write by hand is the translation to and from the Vole format, because there is nothing ready in C++ that knows this format.

A note

If opcode 6 will not be tested and will not run in any program, you can postpone the two functions and make opcode 6 be treated as unsupported. But if the instructor requires that all the opcodes be executed (and in the specification opcode 6 exists), then you must write them.

Would you like a table of the 8-bit values and the decimal numbers they equal (5 or 6 examples) to test the two functions with after you write them?

33. User

(Translated from Arabic) Don't we already have dectohex and the reverse?

34. Claude

No, they do something completely different. `HexToDec` and `DecToHex` change the **way of writing** only, not the **meaning** of the bits.

The difference

- **HexToDec("5C")** returns **92**. This is the same number but written in decimal instead of hex.
- **DecodeFloat(92)** returns **1.5**. This is a **new interpretation** of the bits according to the Vole format.

So the bits `01011100` are the same in both cases, but:

- as an **integer** they mean 92.
- as a **float in the Vole** they mean 1.5.

`HexToDec` knows how to do the first only, and it has no information about the sign and the exponent and the mantissa.

An example to clarify

If the register holds `5C`:

1. `HexToDec("5C")` gives 92. Up to here there is no problem, this is an ordinary

integer and we use it in all opcodes other than 6.
2. If you add 92 + 56 (which is 38) you get 148, which has no relation to 1.75.
3. What gets you to 1.75 is DecodeFloat first, then the addition, then EncodeFloat.

Why the two have different roles

The function	Works on	Converts from/to
HexToDec / DecToHex	**text**	text <-> an integer
DecodeFloat / EncodeFloat	**bits**	an integer <-> a decimal number in the Vole format

The first pair is for reading the file, the input, and printing. The second pair is for one operation only: opcode 6.

So what is the result of the question?

You will not be able to do without DecodeFloat and EncodeFloat if you want opcode 6 to work correctly. If you don't want it, consider it unsupported and that's it.

Would you like me to draw the whole journey (from the text "5C" to the result) in numbered steps on a full float-addition example?

35. User

(Translated from Arabic) Look, before the last modifications you made, I want to make them 5 or 6 classes. I want ALU, Memory (mainmemory, register, screen), Machine, Utils, CPU.

36. Claude

The Design with 5 Classes (+ a small struct)

I brought back ALU, and I combined the memory, the registers and the screen into Memory, and I kept file loading inside Machine because it is part of dealing with the user and the files.

1. Utils (static)

Attributes: none

- HexToDec(s): a hex string to a number.
- DecToHex(value, width): a number to a hex string with a fixed number of digits.
- IsValidHex(s, expectedLen): checks that the string is valid hex and its length is right.

The class's role: reading and printing in hex only. We removed Mask8 and we will write the cut (& 0xFF) directly in the places that need it.

2. ALU (static)

Attributes: none

- AddInt(a, b): two's complement addition and cutting the result to 8 bits (opcode 5).
- AddFloat(a, b): floating-point addition in the Vole format (opcode 6).
- RotateRight(value, times): a circular right rotation in 8 bits (opcode A).
- Or(a, b) and And(a, b) and Xor(a, b): the bit operations (opcodes 7 and 8 and 9).
- DecodeFloat(bits) (private): from 8 bits to a real number.

- `EncodeFloat(value)` (private): from a real number to 8 bits.

The class's role: all the calculations in one place, and it is what guarantees that the result stays between 0 and 255. It takes values and returns a value, and it knows nothing about the registers or the memory.

3. Memory

Attributes:

- `cells[256]`: the main memory.
- `regs[16]`: the registers.
- `screenBuffer`: the screen text.

Methods:

- `GetCell(addr)`: reads a cell.
- `SetCell(addr, value)`: cuts the value to 8 bits and writes. If the address is FF it appends the character to the screen instead of the cell.
- `GetReg(r)`: reads a register.
- `SetReg(r, value)`: cuts the value and writes into a register.
- `DisplayMemory(out, pc)`: prints a 16x16 table and marks the location of the PC.
- `DisplayRegisters(out)`: prints the 16 registers.
- `DisplayScreen(out)`: prints the content of the screen.
- `Reset()`: zeroes the cells and the registers and empties the screen.

The class's role: all the storage in one place, and all the writing goes through `SetCell` or `SetReg`, so the cut is guaranteed.

4. CPU

Attributes:

- `PC`, `IR`, `halted`
- `memory`: a reference to the Memory.

Methods:

- `GetPC()`, `SetPC(pc)`, `GetIR()`
- `Step()`: one complete step and returns `StepResult`.
- `Reset()`: zeroes PC and IR and halted.
- `Fetch()` (private): reads two bytes from PC and PC+1 and assembles them into IR.
- `Decode()` (private): splits the four digits of the IR and checks validity (opcode from 1 to C, and the special constraints of 4 and A and C). It returns an `Instruction`.
- `Execute(inst)` (private): executes according to the opcode, and it calls `Memory` for reading and writing and `ALU` for the calculation.

The class's role: the fetch-decode-execute cycle only. `Step()` checks that the PC is not larger than FE, and adds 2 to the PC before the execution so that the jump (opcode B) overwrites it.

5. Machine

Attributes:

- `memory`, `cpu`: in this order because the CPU takes a reference to the Memory.
- `programLoaded`, `MAX_STEPS`

Methods:

- `Run()`: the main menu, and at the end it asks exit or another program (and if another program it calls `Reset`).
- `ShowMenu()`

- `LoadProgramFromFile()`: asks for the file and the address, checks the address (even and below FF), reads the file and checks each word with `IsValidHex`, makes sure the program fits before FF, and only then writes into memory with `SetCell` and sets the PC.
- `OverwriteRegister()` and `OverwriteMemory()`
- `RunProgram(stepMode)`: full run or step by step, and after each time it calls `DisplayState`.
- `DisplayState()`: calls the three display functions in `Memory` and prints the PC and the IR.
- `ReadHexInput(prompt, min, max)`: asks until a valid value is entered.
- `Reset()`: calls `memory.Reset()` and `cpu.Reset()` and sets `programLoaded` to false.

The class's role: all the dealing with the user and the files, and it ties the other classes together.

6. Instruction (struct, optional)

Attributes: opcode, R, X, Y, XY, valid, errorMsg

This is just a data container that **Decode** returns. If you want exactly 5 classes, you can cancel it and make **Decode** store the values in attributes inside the CPU, but the code would be less clean. Consider it class number 6.

7. Enum

- **StepResult:** OK, HALTED, INVALID_INSTRUCTION, PC_OUT_OF_RANGE

Who calls whom

- `Machine` -> `Memory` and `CPU` and `Utils`.
- `CPU` -> `Memory` and `ALU`.
- `ALU` and `Utils` do not call anyone.

What changed from the previous shape

- `ALU` came back as a class on its own instead of being inside `Utils`.
- `Memory` now holds the memory and the registers and the screen together.
- `ProgramLoader` was removed and its role went to `Machine`.
- `LoadResult` was removed, and `LoadProgramFromFile` prints the error message directly.

Would you like me to update the Mermaid diagram to this shape?

37. User

(Translated from Arabic) Can we separate the program loader and remove the instruction?

38. Claude

The Design with 6 Classes

I brought **ProgramLoader** back on its own, and I removed **Instruction**. Instead, the CPU keeps the decoded digits as its own private attributes.

1. Utils (static)

Attributes: none

- `HexToDec(s)`: a hex string to a number.
- `DecToHex(value, width)`: a number to a hex string with a fixed number of digits (for printing).
- `IsValidHex(s, expectedLen)`: checks that the string is valid hex and its length is right. The Loader and the Machine use it.

2. ALU (static)

Attributes: none

- `AddInt(a, b)`: two's complement addition and cutting the result to 8 bits (opcode 5).
- `AddFloat(a, b)`: floating-point addition in the Vole format (opcode 6).
- `Or(a, b)` and `And(a, b)` and `Xor(a, b)`: opcodes 7 and 8 and 9.
- `RotateRight(value, times)`: a circular right rotation in 8 bits (opcode A).
- `DecodeFloat(bits)` (private): from 8 bits to a real number.
- `EncodeFloat(value)` (private): the reverse, by truncation with no rounding.

3. Memory

Attributes:

- `cells[256]`: the main memory.
- `regs[16]`: the registers.
- `screenBuffer`: the screen text.

Methods:

- `GetCell(addr)` and `SetCell(addr, value)`: reading and writing. Writing cuts to 8 bits, and address FF goes to the screen.
- `GetReg(r)` and `SetReg(r, value)`: the same idea for the registers.
- `DisplayMemory(out, pc)`: a 16x16 table with the PC location marked.
- `DisplayRegisters(out)` and `DisplayScreen(out)`
- `Reset()`: zeroes everything.

4. ProgramLoader (static)

Attributes: none

- `LoadFile(filename, startAddr, memory)`: returns an error text, and empty means it succeeded. It works in this order:

1. The address must be even and between 00 and FE.
2. The file must open.
3. Every word must be 4 hex digits (with `IsValidHex`), and the file must not be empty.
4. The number of words x 2 with the address must end before FF.
5. Only after all the checks succeed does it write into memory with `SetCell`, two bytes for each word (the high one first).

- `ParseFile(filename, words)` (private): reads the words, checks them, and returns an error text if there is a problem.

5. CPU

Attributes:

- `PC`, `IR`, `halted`
- `memory`: a reference to the Memory.
- `opcode`, `R`, `X`, `Y`, `XY`: the decoded digits of the current instruction.
- `errorMsg`: the reason for rejecting the last invalid instruction.

Methods:

- `GetPC()`, `SetPC(pc)`, `GetIR()`, `GetErrorMsg()`
- `Step()`: checks that PC is less than or equal to FE, calls `Fetch`, adds 2 to PC, calls `Decode`, and if it is not valid it returns `INVALID_INSTRUCTION` (and the PC goes back to its place), otherwise it calls `Execute`. It returns `StepResult`.
- `Reset()`

- `Fetch()` (private): assembles IR from two bytes.
- `Decode()` (private): splits the IR into opcode and R and X and Y and XY and stores them in the attributes. It returns true/false according to validity (opcode from 1 to C, opcode 4 the second digit is zero, opcode A the third digit is zero, opcode C all zeros) and puts the reason in `errorMsg`.
- `Execute()` (private): executes according to the `opcode` in the attributes, and it calls `Memory` and `ALU`.

6. Machine

Attributes: `memory, cpu (in this order), programLoaded, MAX_STEPS`

Methods:

- `Run()`: the main menu, and at the end it asks exit or another program.
- `ShowMenu()`
- `LoadProgramFromFile()`: asks for the file and the address, calls `ProgramLoader::LoadFile`, and if it returned an error it prints it, and if it succeeded it sets the PC and shows the state.
- `OverwriteRegister()` and `OverwriteMemory()`
- `RunProgram(stepMode)`
- `DisplayState()`
- `ReadHexInput(prompt, min, max)`
- `Reset()`

Enum

- **StepResult:** `OK, HALTED, INVALID_INSTRUCTION, PC_OUT_OF_RANGE`

Who calls whom

- `Machine` -> `Memory` and `CPU` and `ProgramLoader` and `Utils`.
- `CPU` -> `Memory` and `ALU`.
- `ProgramLoader` -> `Memory` and `Utils`.
- `ALU` and `Utils` do not call anyone.

A note

We removed the `LoadResult` enum and replaced it with an error text. If the instructor wants an enum, you can bring it back without changing the rest of the design.

Would you like me to update the Mermaid diagram to this shape?

39. User

(Translated from Arabic) Yes

40. Claude

Mermaid Diagram for the 6-Class Design

```
classDiagram
    direction LR

    class Utils {
        <<utility>>
        +HexToDec(string s) int$
        +DecToHex(int value, int width) string$
        +IsValidHex(string s, int expectedLen) bool$
    }

    class ALU {
        <<utility>>
```

```
    +AddInt(int a, int b) int$
    +AddFloat(int a, int b) int$
    +Or(int a, int b) int$
    +And(int a, int b) int$
    +Xor(int a, int b) int$
    +RotateRight(int value, int times) int$
    -DecodeFloat(int bits) double$
    -EncodeFloat(double value) int$
}

class Memory {
    -array~int,256~ cells
    -array~int,16~ regs
    -string screenBuffer
    +GetCell(int addr) int
    +SetCell(int addr, int value) void
    +GetReg(int r) int
    +SetReg(int r, int value) void
    +DisplayMemory(ostream out, int pc) void
    +DisplayRegisters(ostream out) void
    +DisplayScreen(ostream out) void
    +Reset() void
}

class ProgramLoader {
    <<utility>>
    +LoadFile(string filename, int startAddr, Memory mem) string$
    -ParseFile(string filename, vector~int~ words) string$
}

class CPU {
    -int PC
    -int IR
    -bool halted
    -Memory memory
    -int opcode
    -int R
    -int X
    -int Y
    -int XY
    -string errorMsg
    +GetPC() int
    +SetPC(int pc) void
    +GetIR() int
    +GetErrorMsg() string
    +Step() StepResult
    +Reset() void
    -Fetch() void
    -Decode() bool
    -Execute() void
}

class Machine {
    -Memory memory
    -CPU cpu
    -bool programLoaded
    -int MAX_STEPS$
    +Run() void
    +ShowMenu() void
    +LoadProgramFromFile() void
    +OverwriteRegister() void
    +OverwriteMemory() void
    +RunProgram(bool stepMode) void
    +DisplayState() void
    +ReadHexInput(string prompt, int minVal, int maxVal) int
    +Reset() void
}

class StepResult {
```

```
    <<enumeration>>
    OK
    HALTED
    INVALID_INSTRUCTION
    PC_OUT_OF_RANGE
}

Machine *-- Memory
Machine *-- CPU
CPU --> Memory : reference

CPU ..> ALU : uses
CPU ..> StepResult : returns
ProgramLoader ..> Memory : writes to
ProgramLoader ..> Utils : uses
Machine ..> ProgramLoader : uses
Machine ..> Utils : uses
Machine ..> StepResult : checks
```

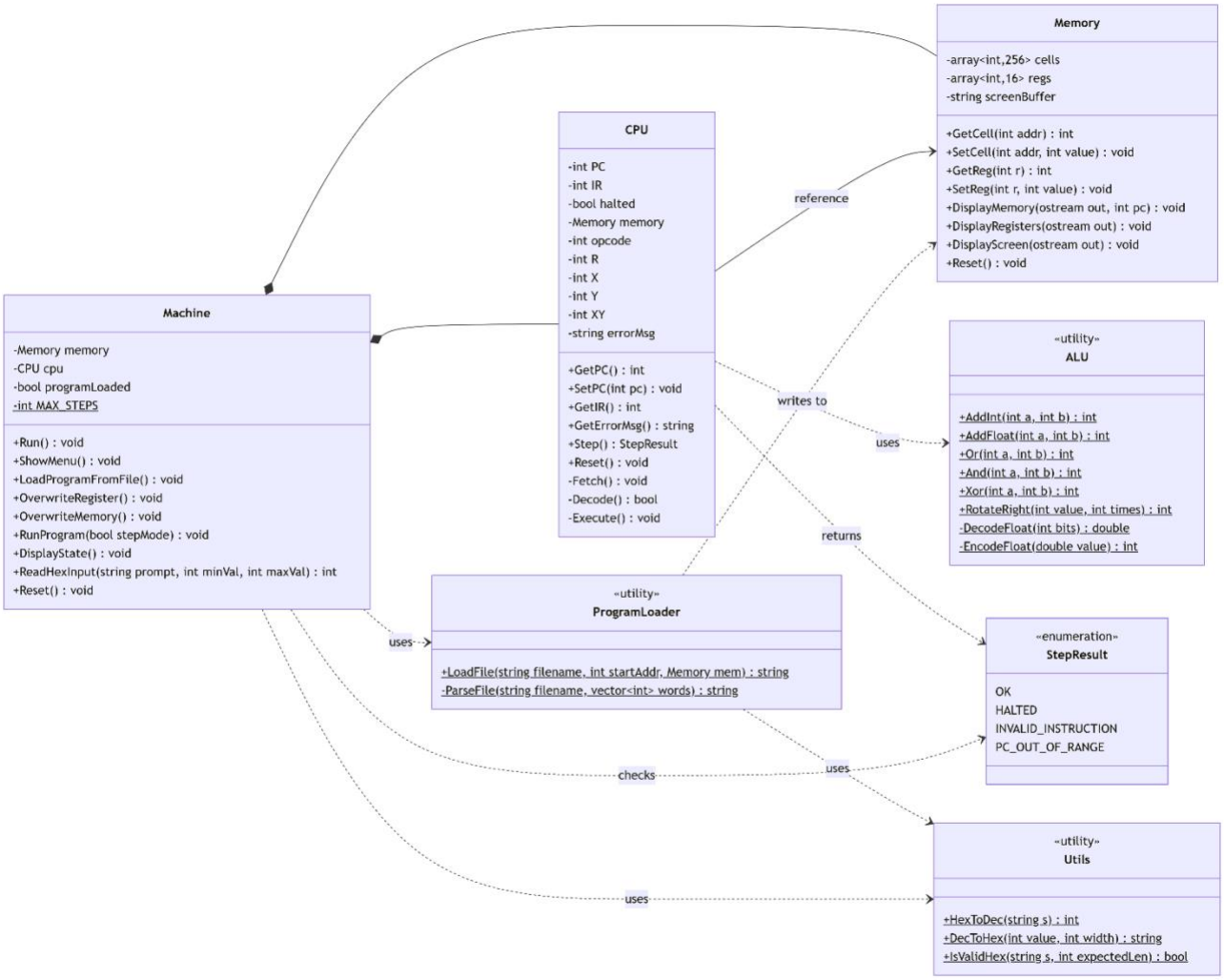


Figure 2: Rendered Mermaid UML class diagram of the 6-class design

Notes

- I removed the `&` sign from the signatures because some Mermaid versions fail with it. In the real code `Memory` and `ostream` are still passed by reference.

- `*--` means the `Machine` is the one that owns the object. And `-->` means the CPU keeps a reference to the Memory. And `..>` means use inside a method without storing.
- The `$` sign means static.
- The order of the members in `Machine` must be `memory` first and then `cpu`.

You can paste it into mermaid.live and export it as an image. If you want, I can put the ready image in a docx file for you, or make a sequence diagram of the fetch-decode-execute cycle.

Programs Testing Screenshots

Program 1 Testing

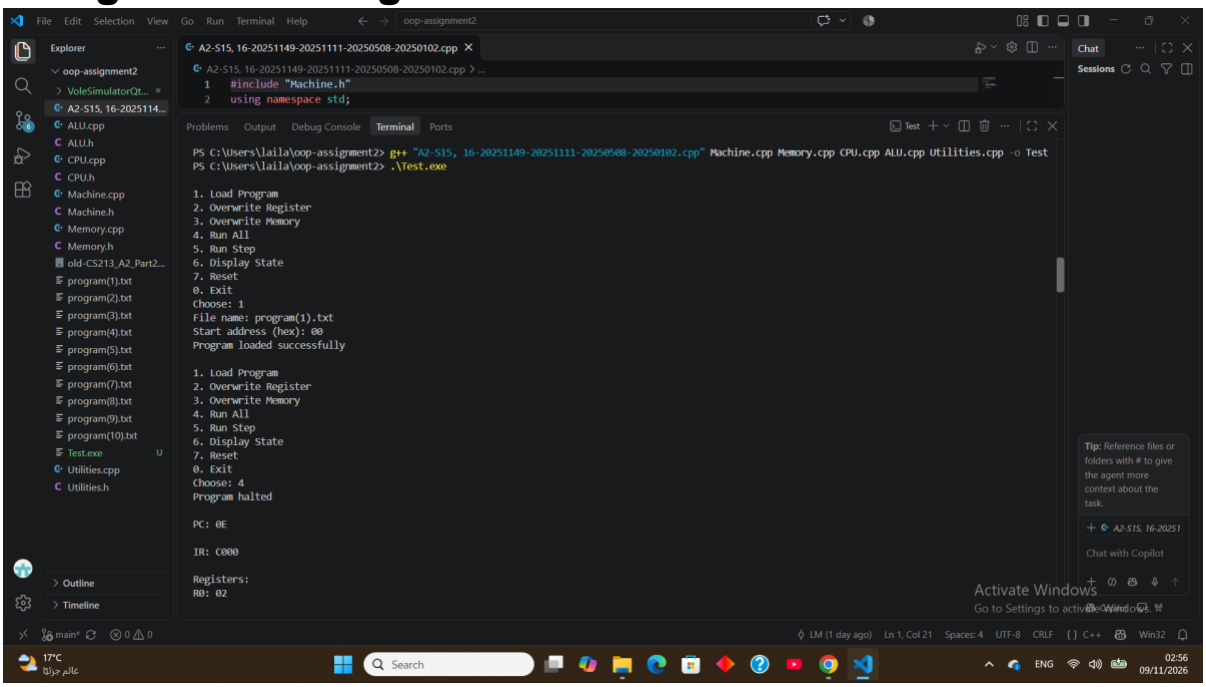


Figure: Program 1 Testing – Screenshot 1

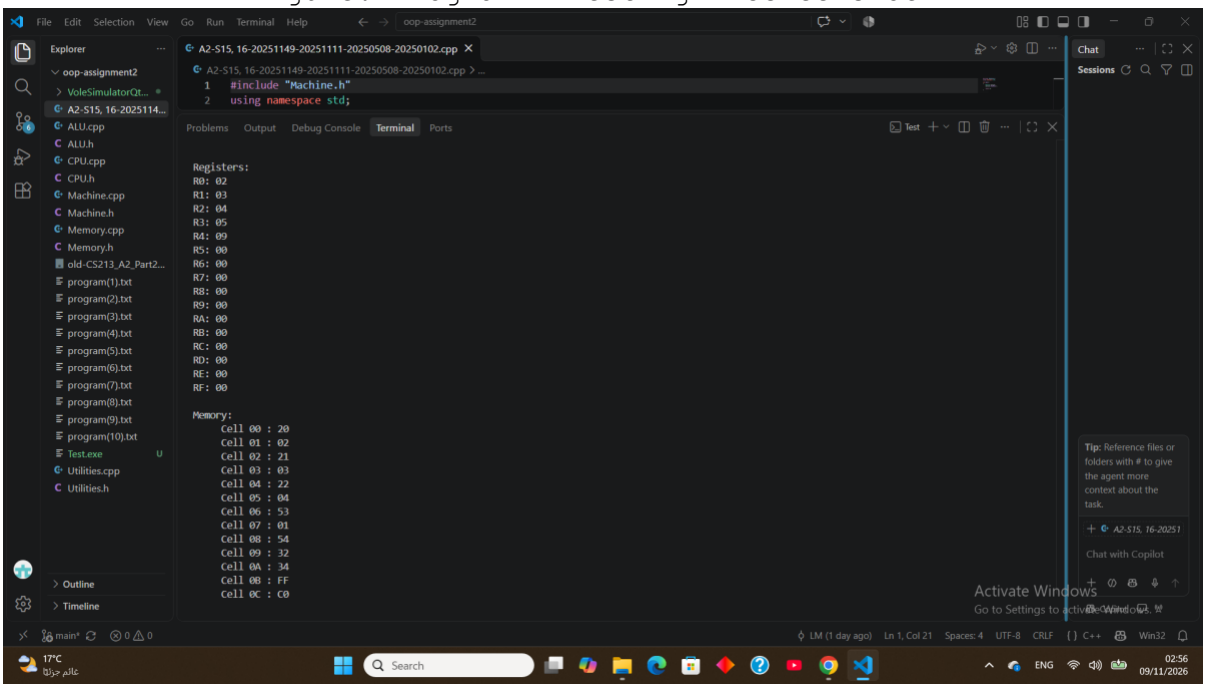


Figure: Program 1 Testing – Screenshot 2

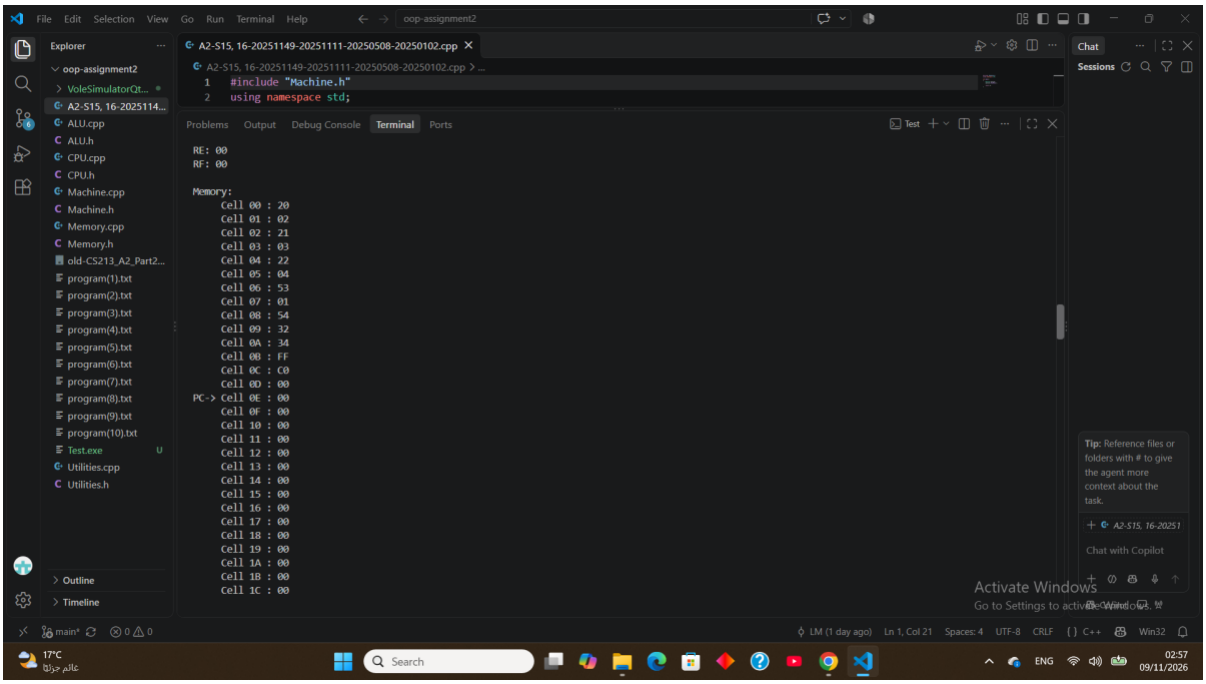


Figure: Program 1 Testing – Screenshot 3

Program 2 Testing

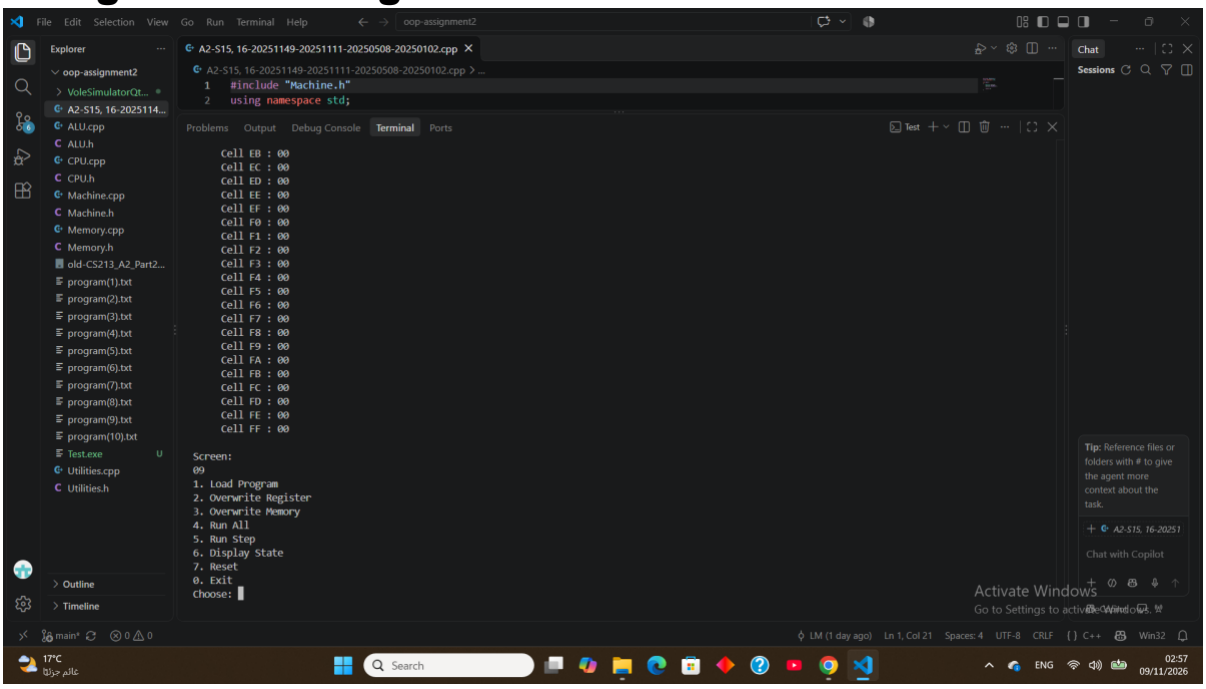


Figure: Program 2 Testing – Screenshot 1

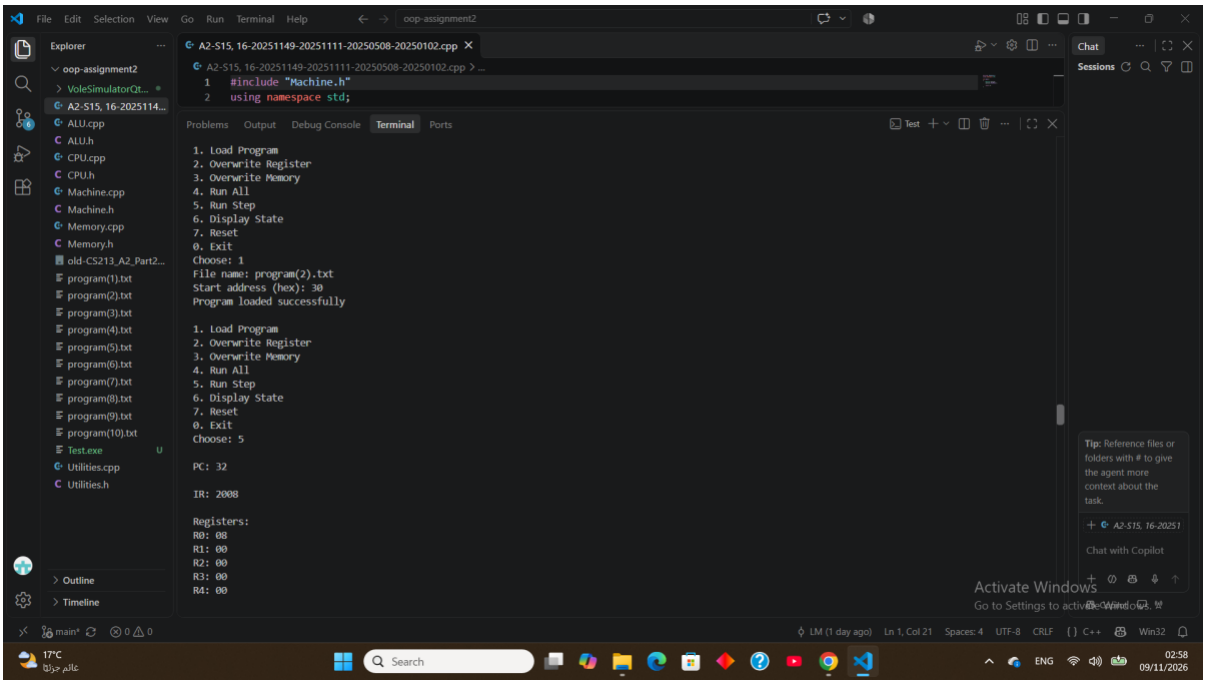


Figure: Program 2 Testing – Screenshot 2

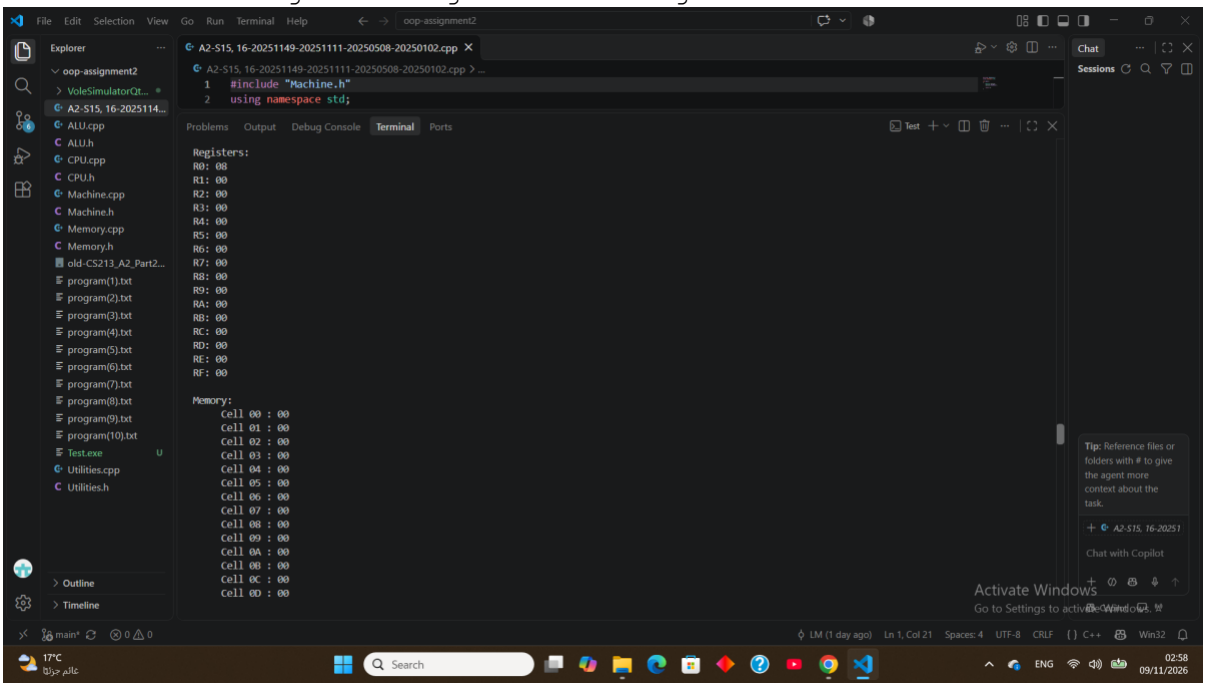


Figure: Program 2 Testing – Screenshot 3

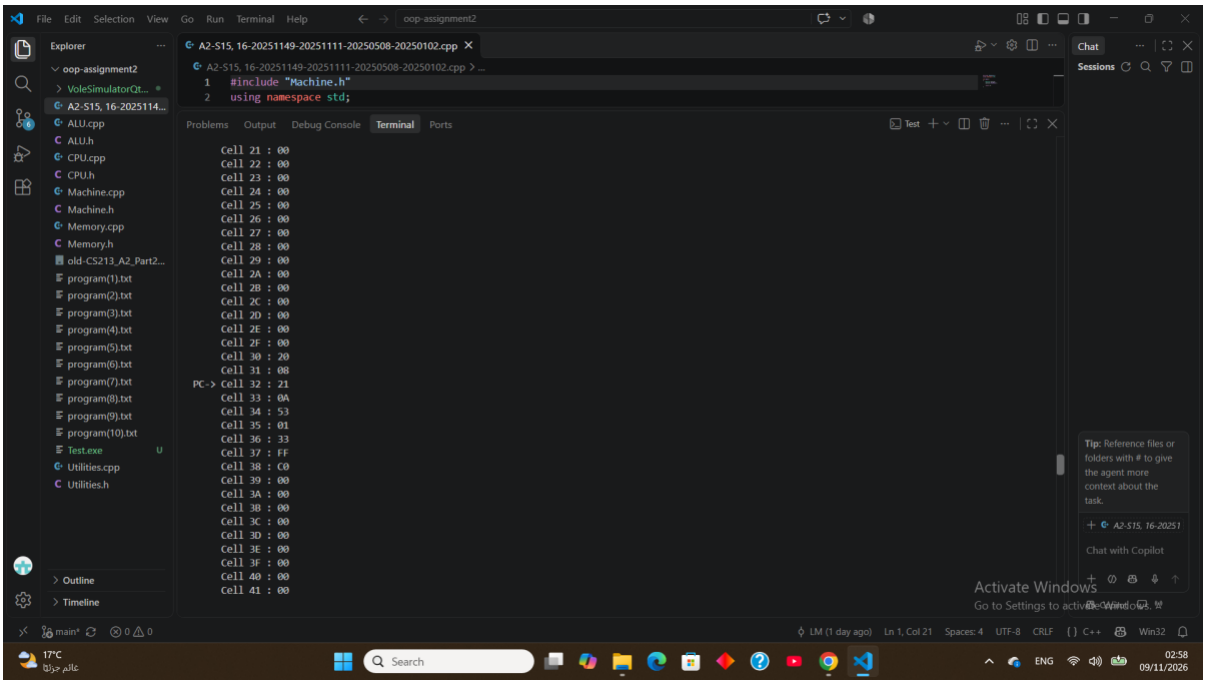


Figure: Program 2 Testing – Screenshot 4

Program 3 Testing

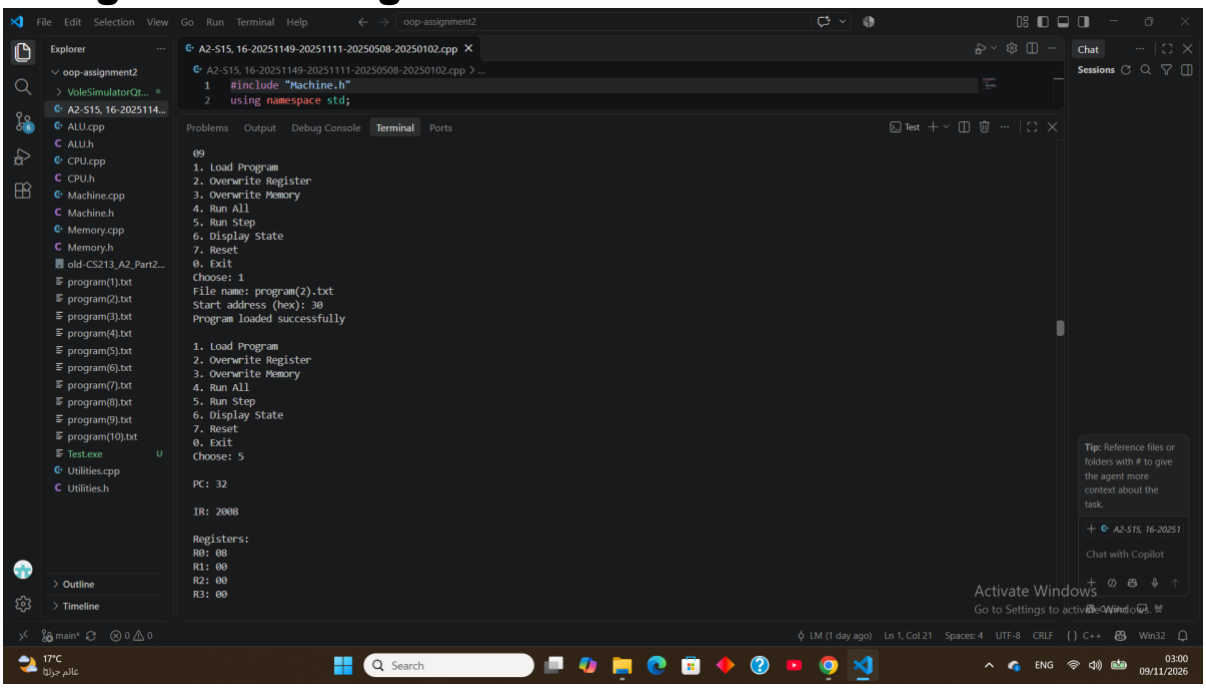


Figure: Program 3 Testing – Screenshot 1

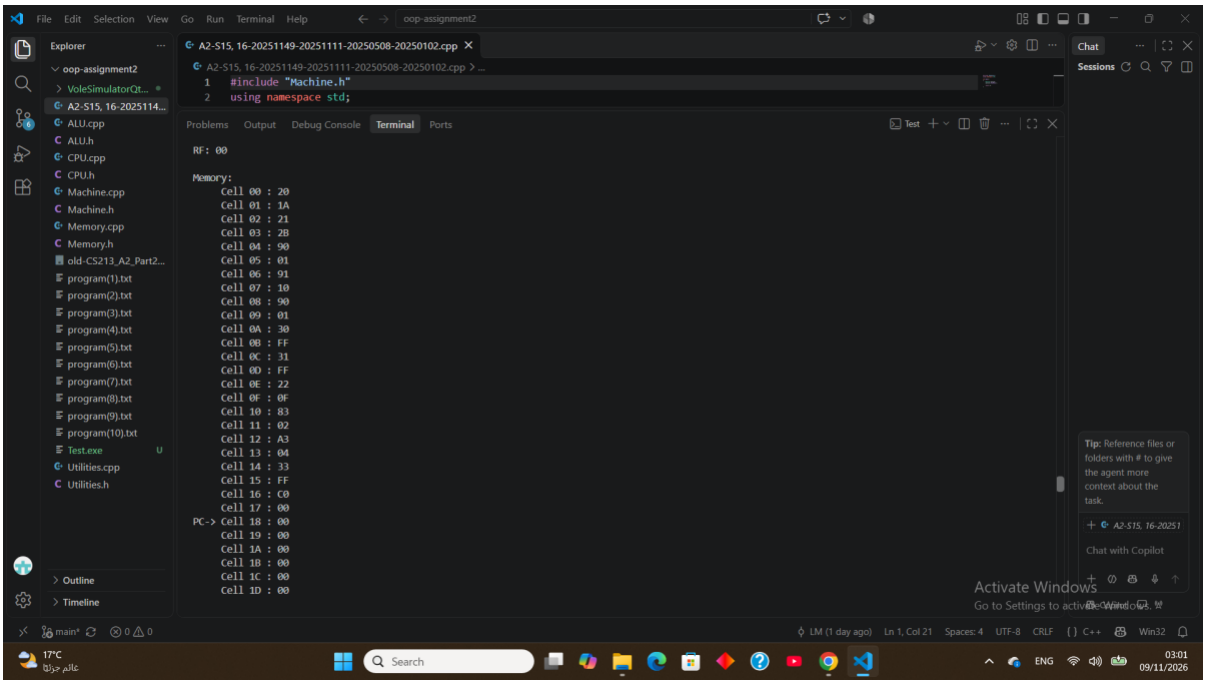
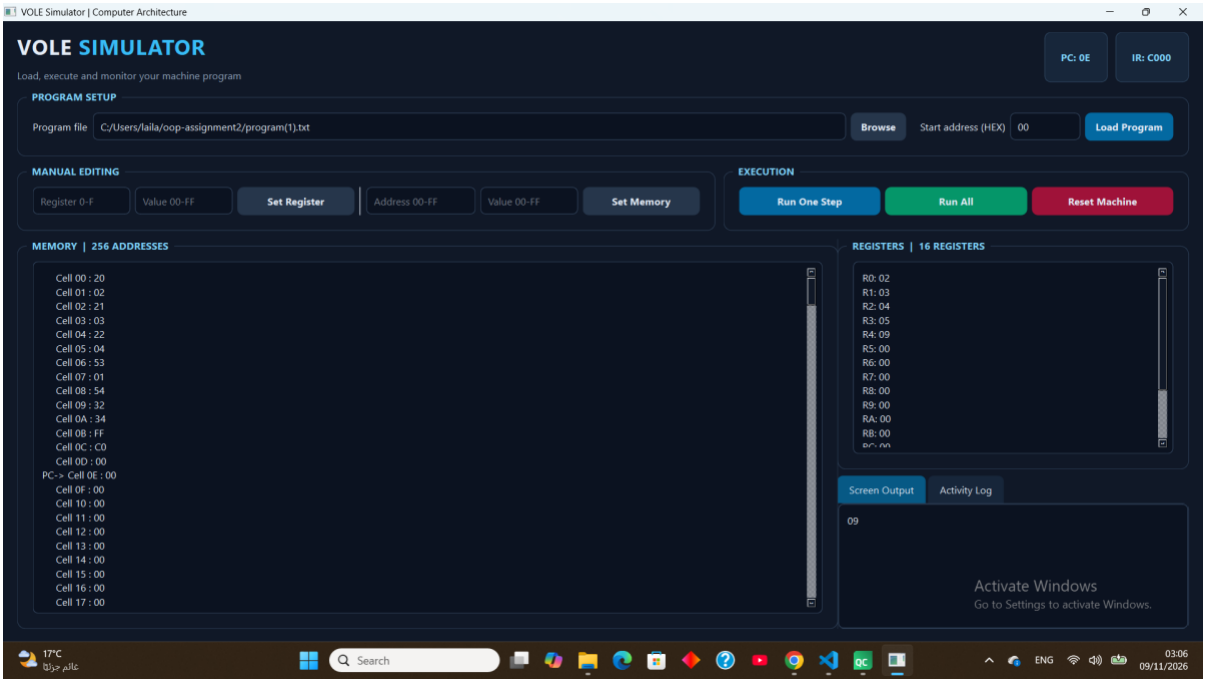
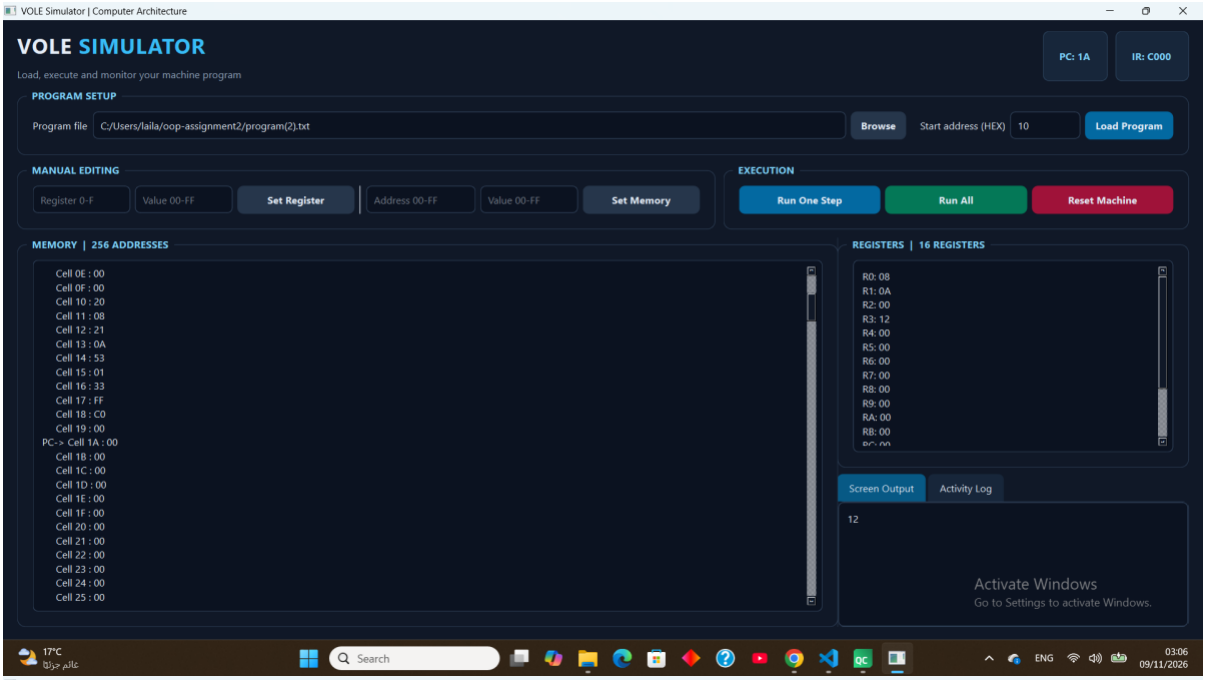
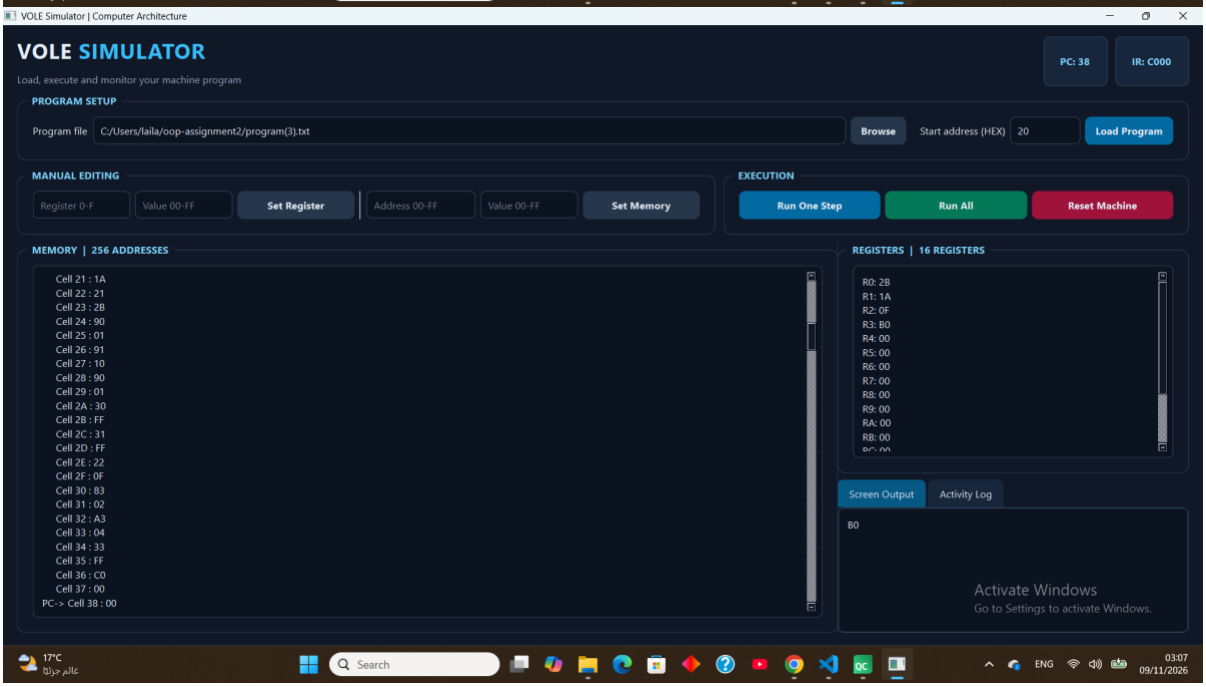


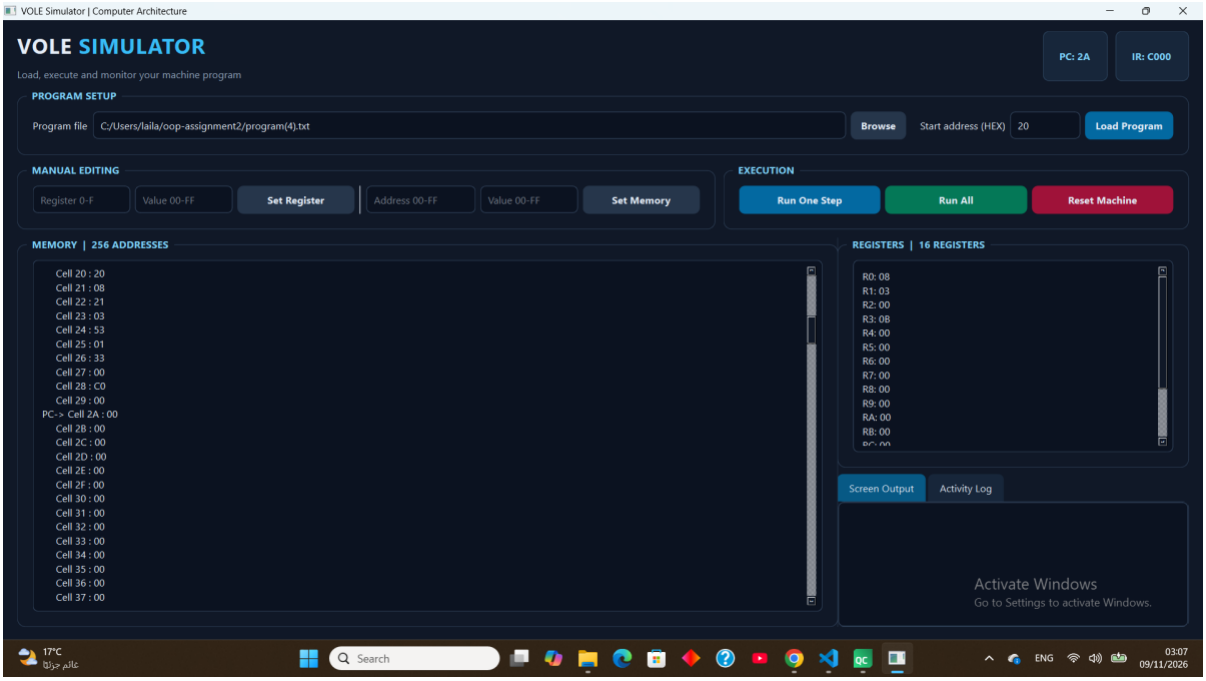
Figure: Program 3 Testing – Screenshot 2

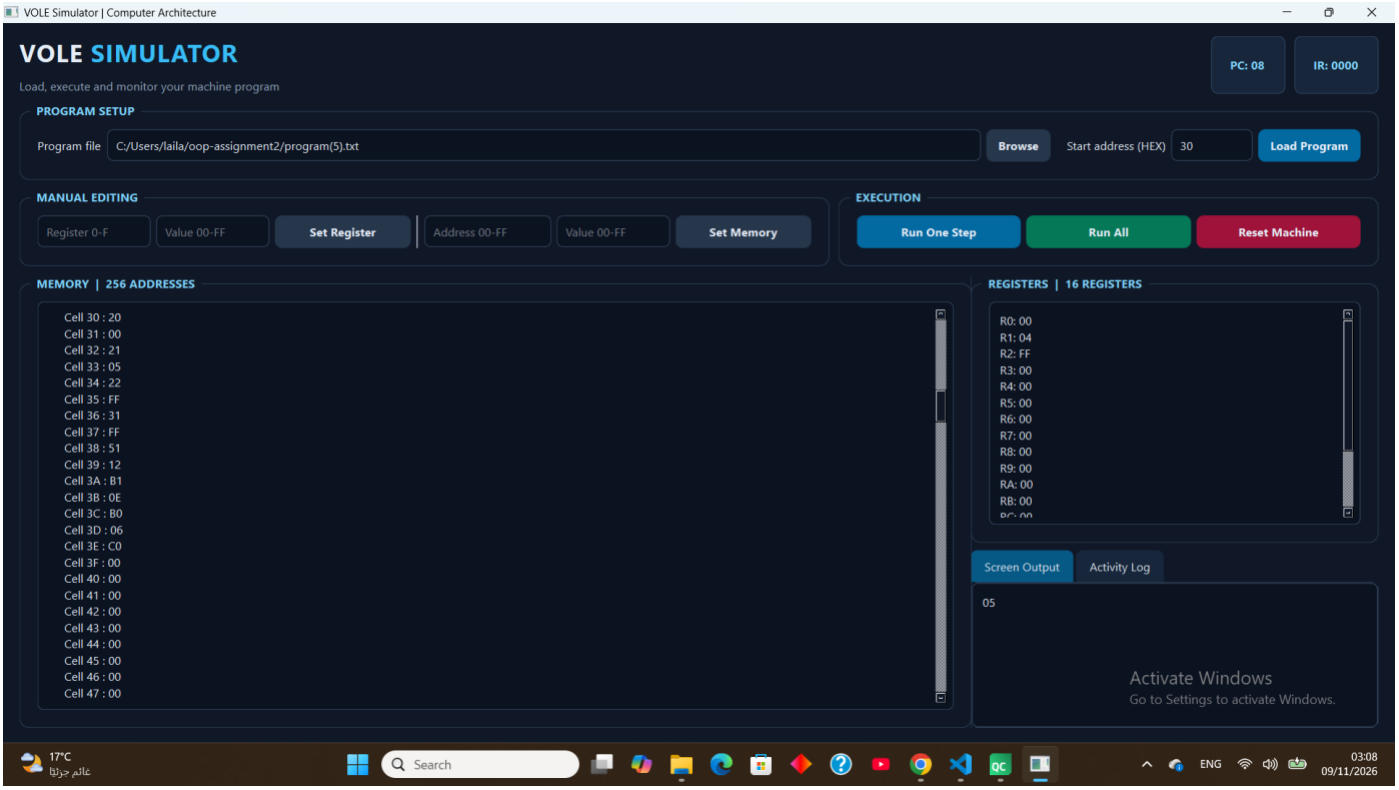
Testing 10 programs GUI Screenshots

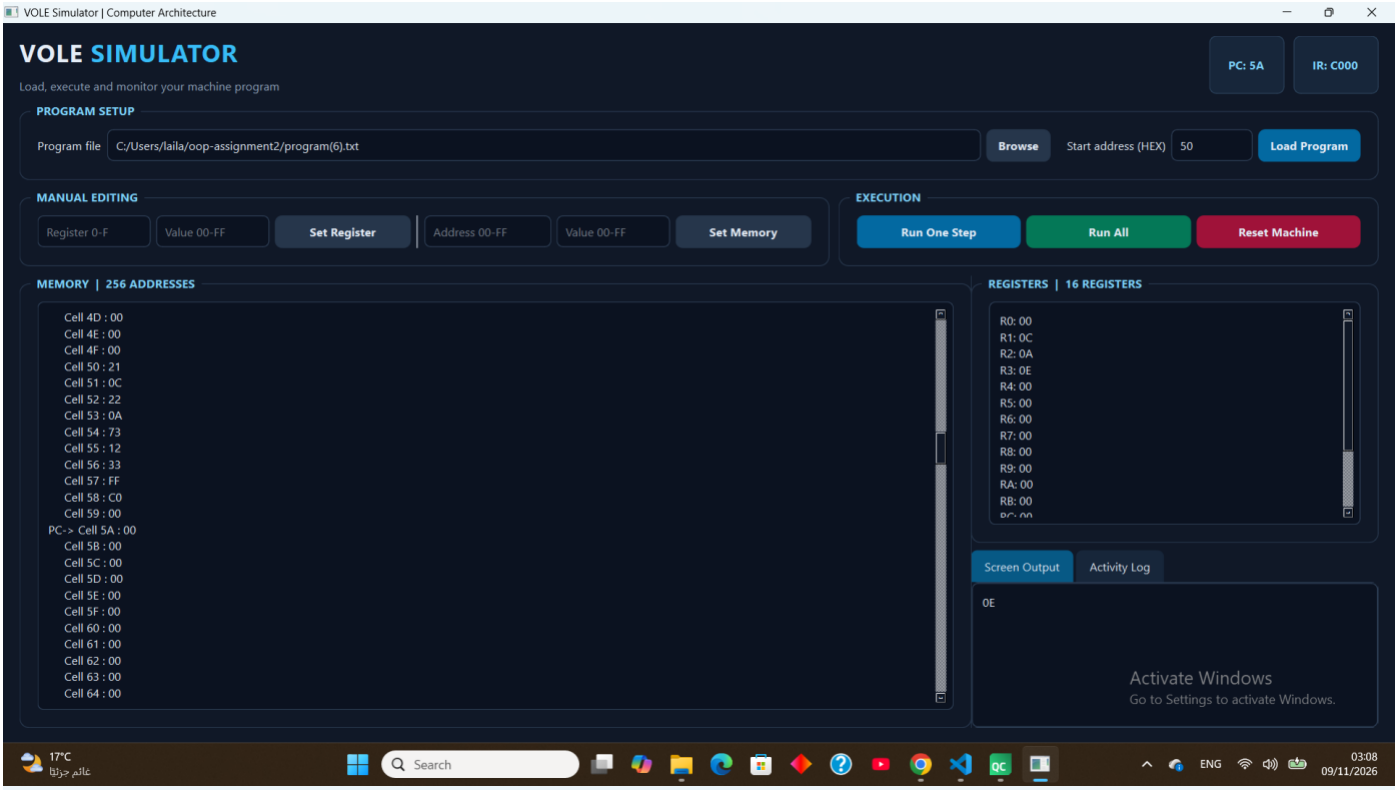


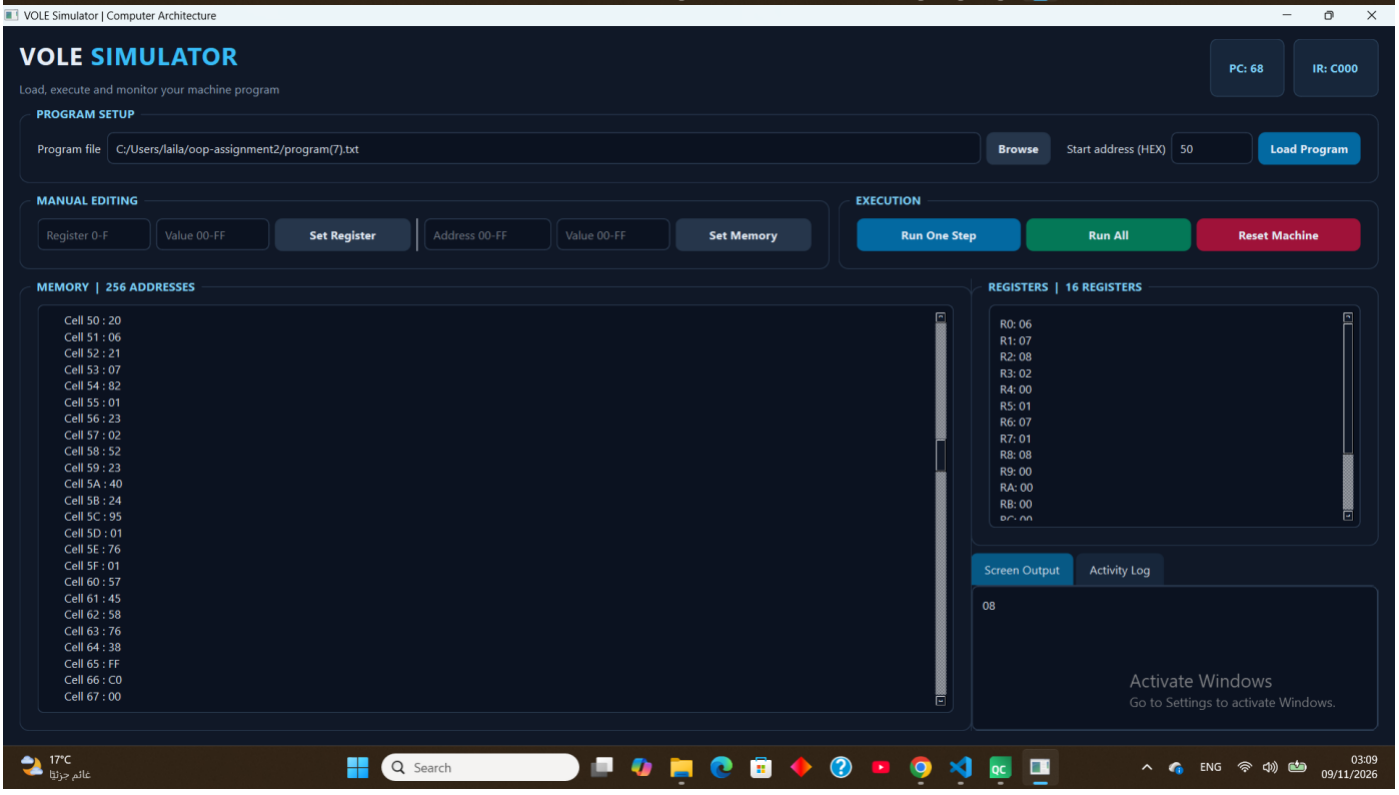

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 1A
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(2).txt
Browse
Start address (HEX)
10
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 0E : 00
Cell 0F : 00
Cell 10 : 20
Cell 11 : 08
Cell 12 : 21
Cell 13 : 0A
Cell 14 : 53
Cell 15 : 01
Cell 16 : 33
Cell 17 : FF
Cell 18 : C0
Cell 19 : 00
PC-> Cell 1A : 00
Cell 1B : 00
Cell 1C : 00
Cell 1D : 00
Cell 1E : 00
Cell 1F : 00
Cell 20 : 00
Cell 21 : 00
Cell 22 : 00
Cell 23 : 00
Cell 24 : 00
Cell 25 : 00
REGISTERS | 16 REGISTERS
R0: 08
R1: 0A
R2: 00
R3: 12
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
12
Activate Windows
Go to Settings to activate Windows.
17°C
عالم جزئيا
Search
ENG
03:06
09/11/2026

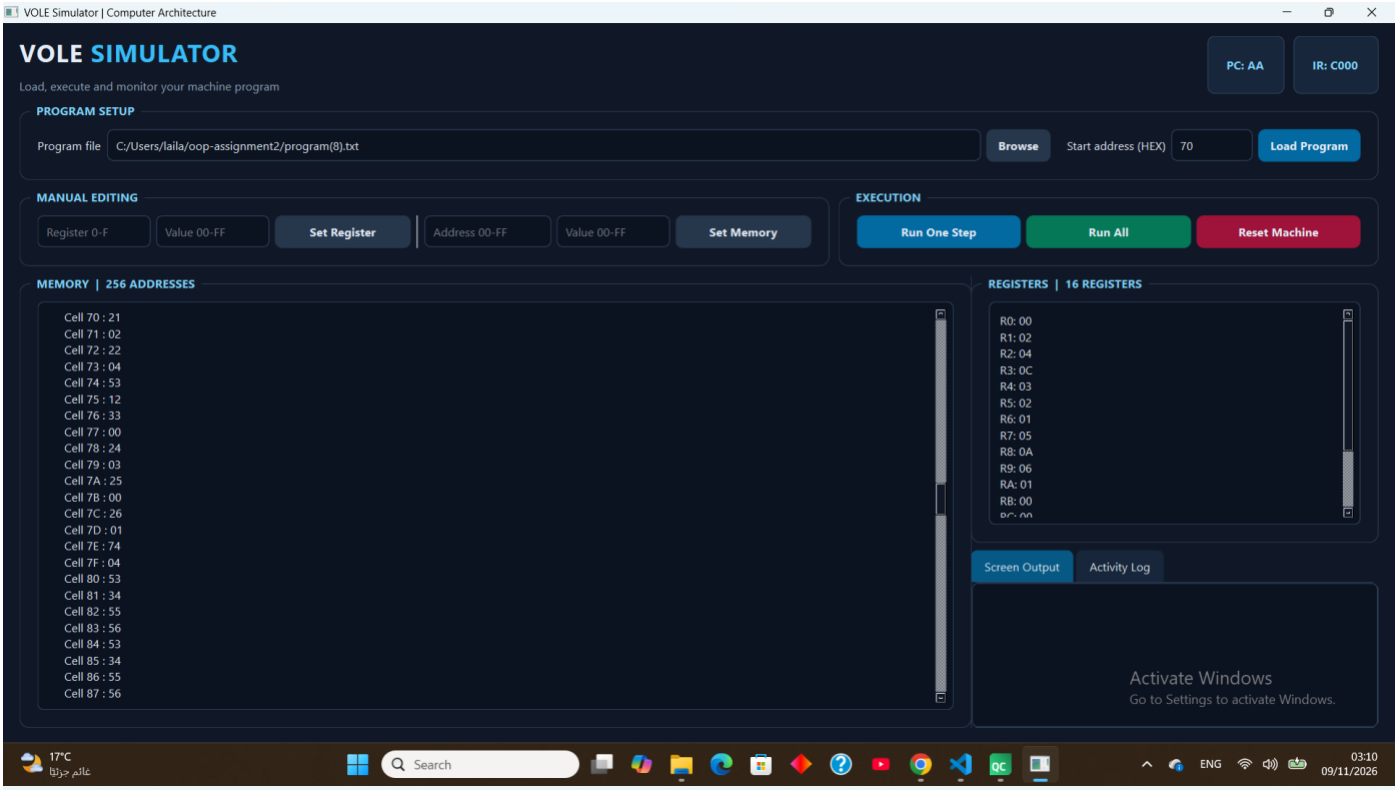

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 38
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(3).txt
Browse
Start address (HEX)
20
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 21 : 1A
Cell 22 : 21
Cell 23 : 2B
Cell 24 : 90
Cell 25 : 01
Cell 26 : 91
Cell 27 : 10
Cell 28 : 90
Cell 29 : 01
Cell 2A : 30
Cell 2B : FF
Cell 2C : 31
Cell 2D : FF
Cell 2E : 22
Cell 2F : 0F
Cell 30 : 83
Cell 31 : 02
Cell 32 : A3
Cell 33 : 04
Cell 34 : 33
Cell 35 : FF
Cell 36 : C0
Cell 37 : 00
PC-> Cell 38 : 00
REGISTERS | 16 REGISTERS
R0: 2B
R1: 1A
R2: 0F
R3: B0
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
B0
Activate Windows
Go to Settings to activate Windows.
17°C
عالم جزئيا
Search
ENG
03:07
09/11/2026

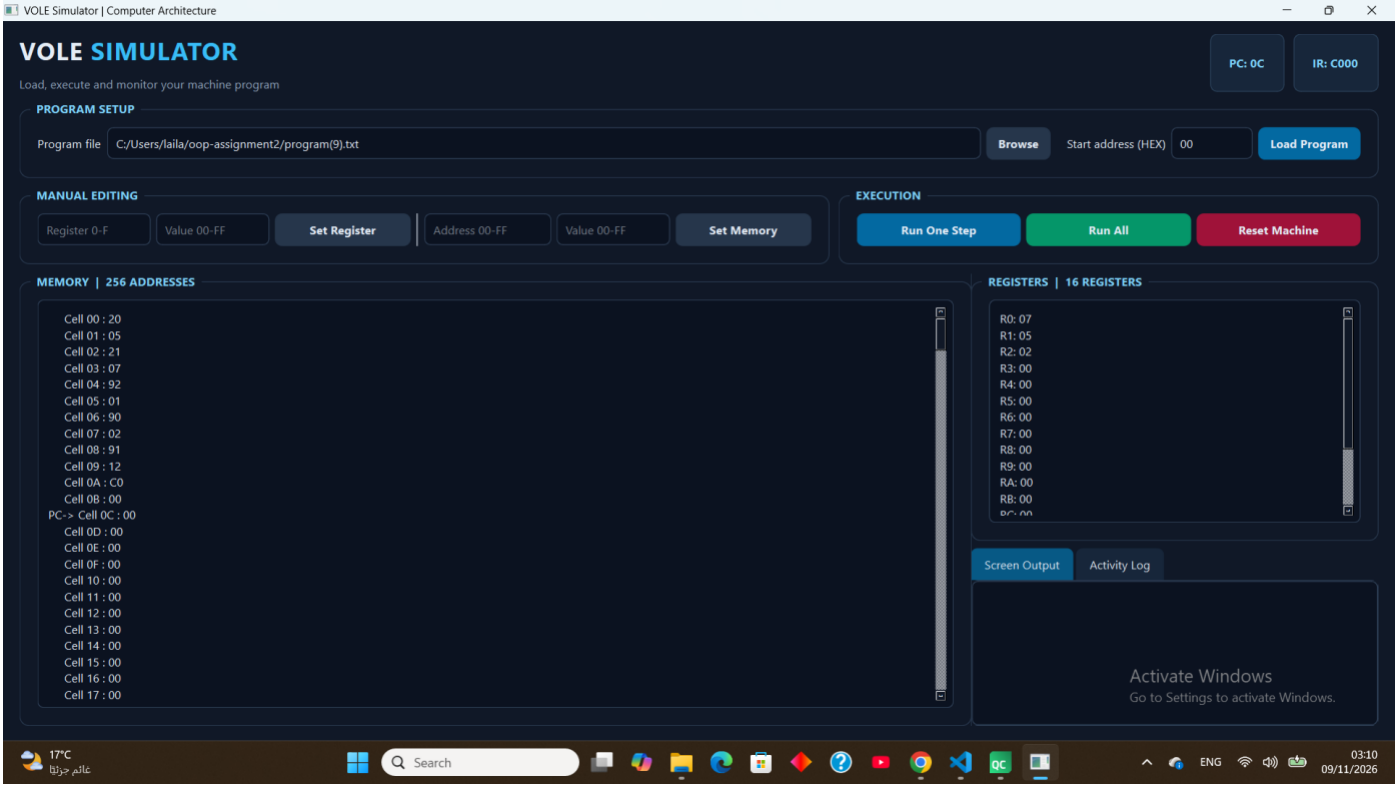

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 2A
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(4).txt
Browse
Start address (HEX)
20
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 20 : 20
Cell 21 : 08
Cell 22 : 21
Cell 23 : 03
Cell 24 : 53
Cell 25 : 01
Cell 26 : 33
Cell 27 : 00
Cell 28 : C0
Cell 29 : 00
PC-> Cell 2A : 00
Cell 2B : 00
Cell 2C : 00
Cell 2D : 00
Cell 2E : 00
Cell 2F : 00
Cell 30 : 00
Cell 31 : 00
Cell 32 : 00
Cell 33 : 00
Cell 34 : 00
Cell 35 : 00
Cell 36 : 00
Cell 37 : 00
REGISTERS | 16 REGISTERS
R0: 08
R1: 03
R2: 00
R3: 08
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
Activate Windows
Go to Settings to activate Windows.
17°C
عالم جزئيا
Search
ENG
03:07
09/11/2026

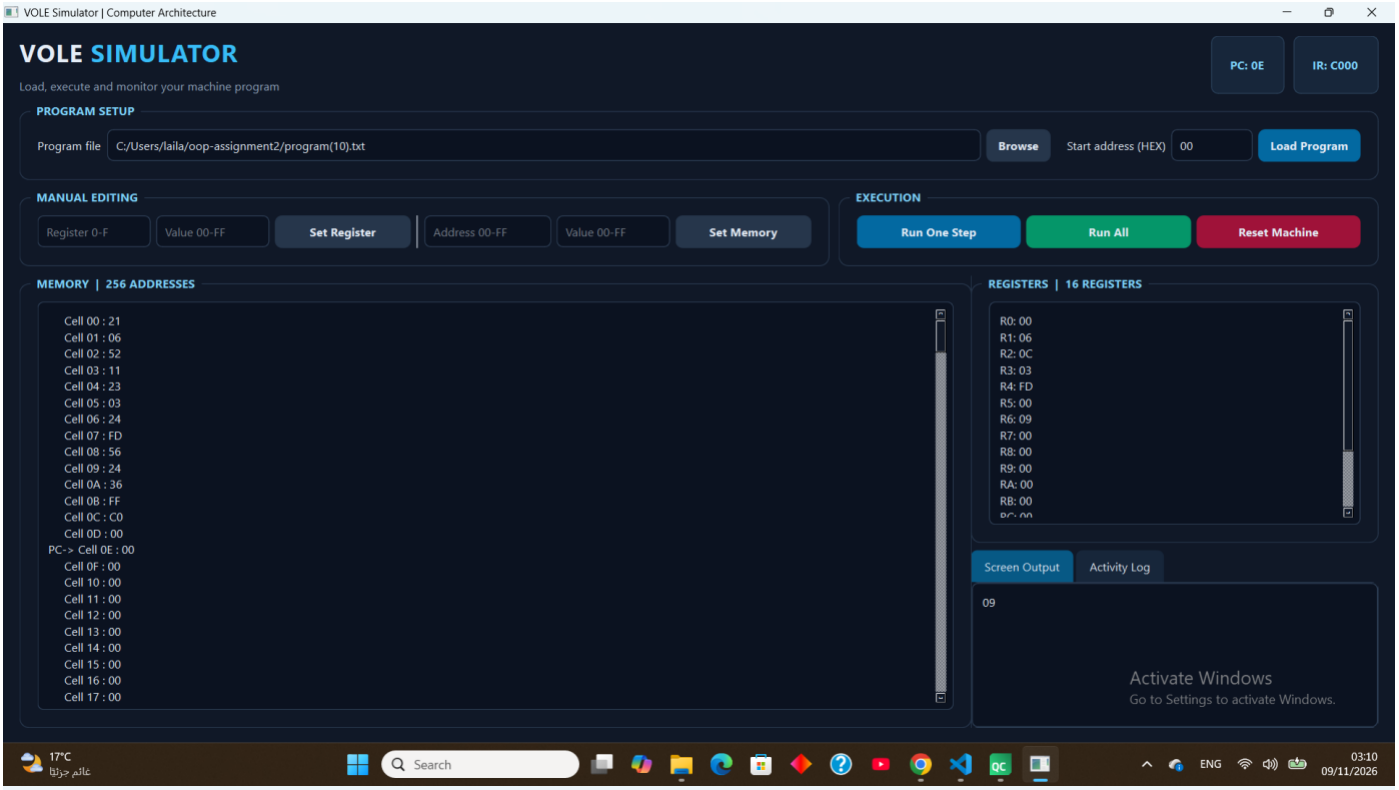

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 08
IR: 0000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(5).txt
Browse
Start address (HEX)
30
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 30 : 20
Cell 31 : 00
Cell 32 : 21
Cell 33 : 05
Cell 34 : 22
Cell 35 : FF
Cell 36 : 31
Cell 37 : FF
Cell 38 : 51
Cell 39 : 12
Cell 3A : B1
Cell 3B : 0E
Cell 3C : B0
Cell 3D : 06
Cell 3E : C0
Cell 3F : 00
Cell 40 : 00
Cell 41 : 00
Cell 42 : 00
Cell 43 : 00
Cell 44 : 00
Cell 45 : 00
Cell 46 : 00
Cell 47 : 00
REGISTERS | 16 REGISTERS
R0: 00
R1: 04
R2: FF
R3: 00
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
05
Activate Windows
Go to Settings to activate Windows.
17°C
عالم جزئيا
Search
ENG
03:08
09/11/2026

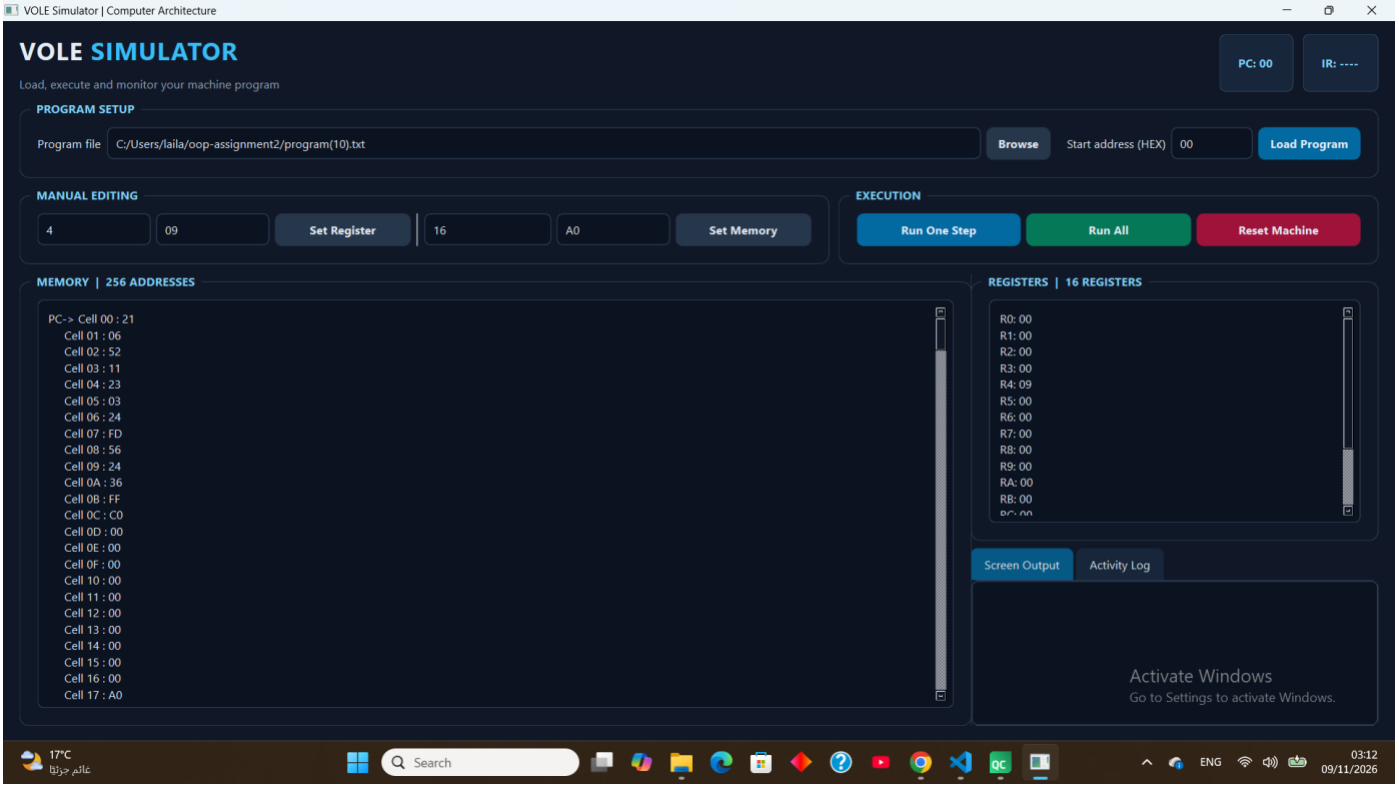

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 5A
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(6).txt
Browse
Start address (HEX)
50
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 4D : 00
Cell 4E : 00
Cell 4F : 00
Cell 50 : 21
Cell 51 : 0C
Cell 52 : 22
Cell 53 : 0A
Cell 54 : 73
Cell 55 : 12
Cell 56 : 33
Cell 57 : FF
Cell 58 : C0
Cell 59 : 00
PC-> Cell 5A : 00
Cell 5B : 00
Cell 5C : 00
Cell 5D : 00
Cell 5E : 00
Cell 5F : 00
Cell 60 : 00
Cell 61 : 00
Cell 62 : 00
Cell 63 : 00
Cell 64 : 00
REGISTERS | 16 REGISTERS
R0: 00
R1: 0C
R2: 0A
R3: 0E
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
0E
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:08
09/11/2026


VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 68
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(7).txt
Browse
Start address (HEX)
50
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 50 : 20
Cell 51 : 06
Cell 52 : 21
Cell 53 : 07
Cell 54 : 82
Cell 55 : 01
Cell 56 : 23
Cell 57 : 02
Cell 58 : 52
Cell 59 : 23
Cell 5A : 40
Cell 5B : 24
Cell 5C : 95
Cell 5D : 01
Cell 5E : 76
Cell 5F : 01
Cell 60 : 57
Cell 61 : 45
Cell 62 : 58
Cell 63 : 76
Cell 64 : 38
Cell 65 : FF
Cell 66 : C0
Cell 67 : 00
REGISTERS | 16 REGISTERS
R0: 06
R1: 07
R2: 08
R3: 02
R4: 00
R5: 01
R6: 07
R7: 01
R8: 08
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
08
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:09
09/11/2026

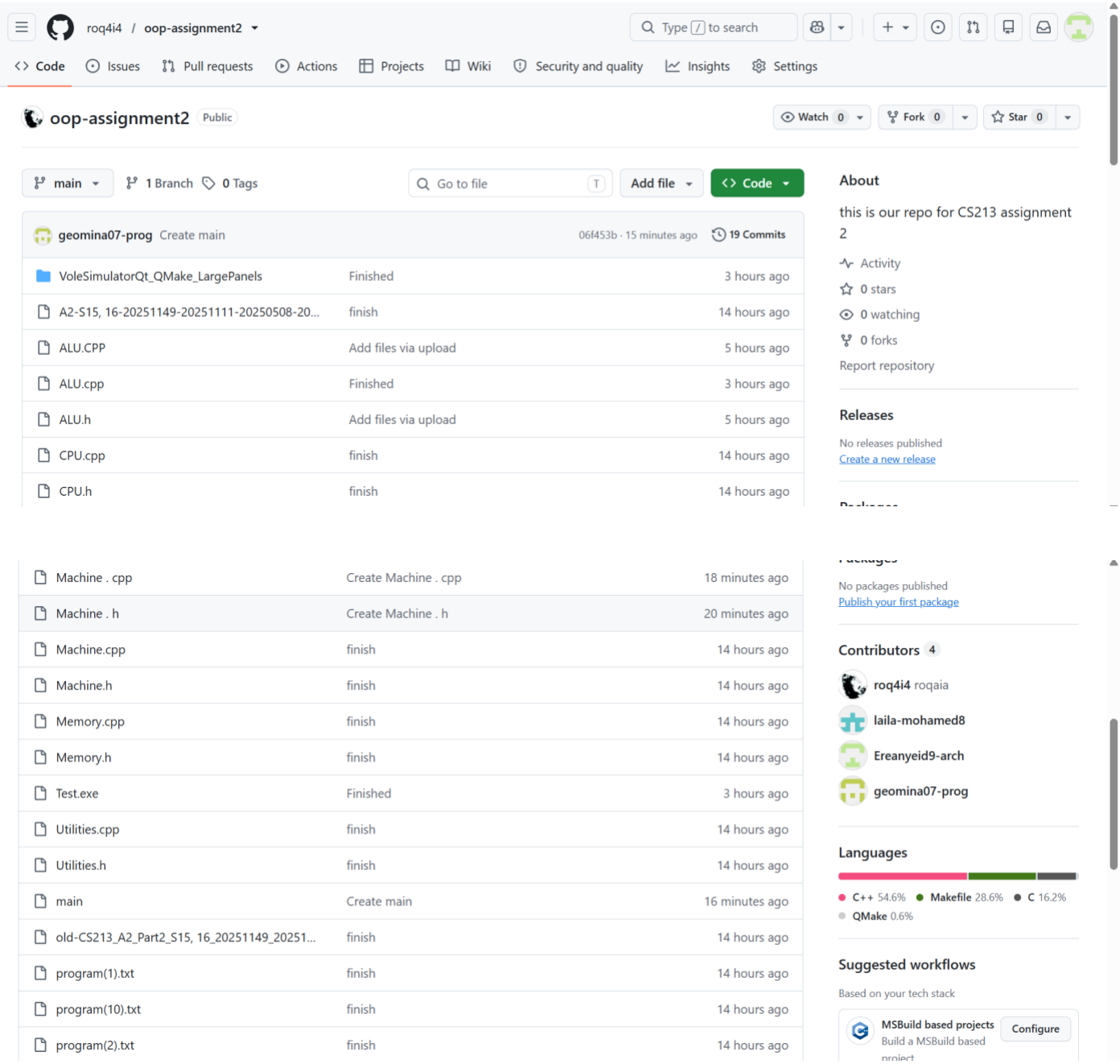

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: AA
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(8).txt
Browse
Start address (HEX)
70
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 70 : 21
Cell 71 : 02
Cell 72 : 22
Cell 73 : 04
Cell 74 : 53
Cell 75 : 12
Cell 76 : 33
Cell 77 : 00
Cell 78 : 24
Cell 79 : 03
Cell 7A : 25
Cell 7B : 00
Cell 7C : 26
Cell 7D : 01
Cell 7E : 74
Cell 7F : 04
Cell 80 : 53
Cell 81 : 34
Cell 82 : 55
Cell 83 : 56
Cell 84 : 53
Cell 85 : 34
Cell 86 : 55
Cell 87 : 56
REGISTERS | 16 REGISTERS
R0: 00
R1: 02
R2: 04
R3: 0C
R4: 03
R5: 02
R6: 01
R7: 05
R8: 0A
R9: 06
RA: 01
RB: 00
Screen Output
Activity Log
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:10
09/11/2026

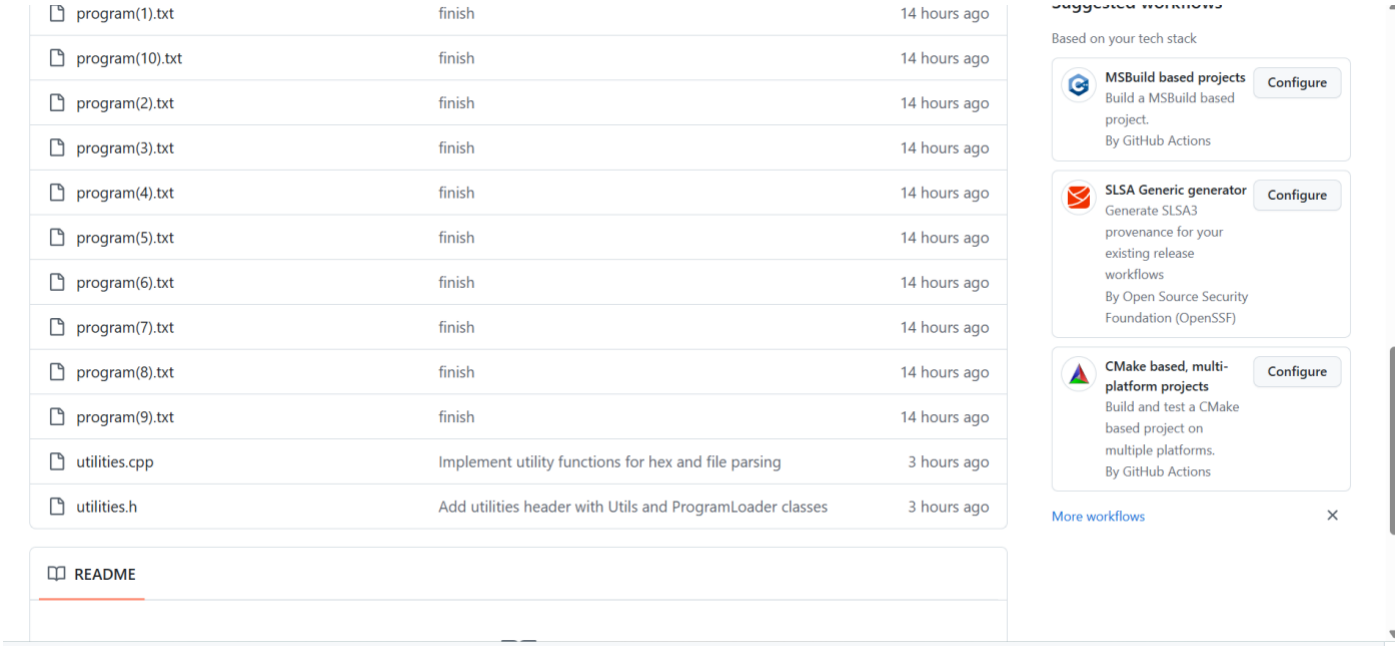

VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 0C
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(9).txt
Browse
Start address (HEX)
00
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 00 : 20
Cell 01 : 05
Cell 02 : 21
Cell 03 : 07
Cell 04 : 92
Cell 05 : 01
Cell 06 : 90
Cell 07 : 02
Cell 08 : 91
Cell 09 : 12
Cell 0A : C0
Cell 0B : 00
PC-> Cell 0C : 00
Cell 0D : 00
Cell 0E : 00
Cell 0F : 00
Cell 10 : 00
Cell 11 : 00
Cell 12 : 00
Cell 13 : 00
Cell 14 : 00
Cell 15 : 00
Cell 16 : 00
Cell 17 : 00
REGISTERS | 16 REGISTERS
R0: 07
R1: 05
R2: 02
R3: 00
R4: 00
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:10
09/11/2026


VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 0E
IR: C000
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(10).txt
Browse
Start address (HEX)
00
Load Program
MANUAL EDITING
Register 0-F
Value 00-FF
Set Register
Address 00-FF
Value 00-FF
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
Cell 00 : 21
Cell 01 : 06
Cell 02 : 52
Cell 03 : 11
Cell 04 : 23
Cell 05 : 03
Cell 06 : 24
Cell 07 : FD
Cell 08 : 56
Cell 09 : 24
Cell 0A : 36
Cell 0B : FF
Cell 0C : C0
Cell 0D : 00
PC-> Cell 0E : 00
Cell 0F : 00
Cell 10 : 00
Cell 11 : 00
Cell 12 : 00
Cell 13 : 00
Cell 14 : 00
Cell 15 : 00
Cell 16 : 00
Cell 17 : 00
REGISTERS | 16 REGISTERS
R0: 00
R1: 06
R2: 0C
R3: 03
R4: FD
R5: 00
R6: 09
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
09
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:10
09/11/2026


VOLE Simulator | Computer Architecture
VOLE SIMULATOR
Load, execute and monitor your machine program
PC: 00
IR: ----
PROGRAM SETUP
Program file
C:/Users/laila/oop-assignment2/program(10).txt
Browse
Start address (HEX)
00
Load Program
MANUAL EDITING
4
09
Set Register
16
A0
Set Memory
EXECUTION
Run One Step
Run All
Reset Machine
MEMORY | 256 ADDRESSES
PC-> Cell 00 : 21
Cell 01 : 06
Cell 02 : 52
Cell 03 : 11
Cell 04 : 23
Cell 05 : 03
Cell 06 : 24
Cell 07 : FD
Cell 08 : 56
Cell 09 : 24
Cell 0A : 36
Cell 0B : FF
Cell 0C : C0
Cell 0D : 00
Cell 0E : 00
Cell 0F : 00
Cell 10 : 00
Cell 11 : 00
Cell 12 : 00
Cell 13 : 00
Cell 14 : 00
Cell 15 : 00
Cell 16 : 00
Cell 17 : A0
REGISTERS | 16 REGISTERS
R0: 00
R1: 00
R2: 00
R3: 00
R4: 09
R5: 00
R6: 00
R7: 00
R8: 00
R9: 00
RA: 00
RB: 00
Screen Output
Activity Log
Activate Windows
Go to Settings to activate Windows.
17°C
عائم جزئيًا
Search
ENG
03:12
09/11/2026

GitHub Screenshots:





..		
.qtcreator	Finished	4 hours ago
build/Desktop_Qt_6_12_0_MinGW_64_bit_Debug	Finished	4 hours ago
A2-S15_16-20251149-20251111-20250508-202...	Finished	4 hours ago
A2-S15_16-20251149-20251111-20250508-202...	Finished	4 hours ago
ALU.cpp	finish	15 hours ago
ALU.h	finish	15 hours ago
CPU.cpp	finish	15 hours ago
CPU.h	finish	15 hours ago
CS213_A2_Part2_S15, 16_20251149_20251111_2...	Finished	4 hours ago
Machine.cpp	finish	15 hours ago
Machine.h	finish	15 hours ago
MainWindow.cpp	finish	15 hours ago

Memory.cpp	finish	15 hours ago
Memory.h	finish	15 hours ago
README.txt	finish	15 hours ago
TEAM.txt	finish	15 hours ago
Utilities.cpp	finish	15 hours ago
Utilities.h	finish	15 hours ago
old main.cpp	finish	15 hours ago

What we learned:

Through building this assignment, we gained hands-on experience in applying Object-Oriented Programming (OOP) principles to design a modular and scalable software solution. We learned how to structure code using core OOP concepts—such as encapsulation to safeguard data, inheritance to promote code reuse, and polymorphism to allow flexible component behavior. Additionally, working on this project deepened our understanding of class design, memory management, and input validation, while improving our ability to collaborate, debug complex logical flows, and translate real-world requirements into clean, maintainable code.